



Overseer: The Roadmap

How a Rust harness becomes the best agent harness there is:
an Alfred to your Batman

EDITION 1 · OCTOBER 2026

Contents

Part 0 — Before You Read	3
How to Read This Roadmap	4
The Contract	7
Executive Summary	9
Part I — Foundations	13
1. The North Star: What “Alfred” Means	14
2. The Constitution: Principles and Invariants	17
3. Where We Stand: The Baseline Audit	21
4. The Competitive Bar	27
Part II — The Pillars	30
5. Reliability and Long-Horizon Execution	31
6. Orchestration and Subagents	47
7. Memory	54
8. Context Management	58
9. Tool Efficiency	62
10. Token and Cache Efficiency	65
11. Policy: Principles, Rules, AGENTS.md and the Instruction Compiler	69
12. Automation: Durable Workflows, Schedules and Triggers	76
Part III — Supporting Systems	82
13. Security and Trust	83
14. Computer Use and the Browser	87
15. The Model Layer	89
16. Observability	91
17. Evaluation and Proof	92
18. Frontends and Experience	94
19. Extensibility and Ecosystem	96
20. Runtime, Performance and Distribution	97
21. Overseer Life: How the Harness Carries the Dashboard	99
Part IV — Execution	101
22. What Could Break: The Failure Atlas	102
23. Sequencing: Waves, Dependencies and Gates	107
24. The Verification System	112
25. The Scoreboard	114
26. Decisions for the Owner, and the Risk Register	116
Appendices	118
Appendix A — New Event Kinds	119
Appendix B — Configuration Surface	120
Appendix C — Workstream Index	121
Appendix D — Glossary	126
Appendix E — Sources	127
Appendix F — How This Edition Was Reviewed	128



Part 0 — Before You Read

How to Read This Roadmap

This is the master plan for turning Overseer from a careful, well-instrumented agent harness into the best agent harness that exists: the one an owner can hand a hundred-hour job to, walk away from, and come back to a finished, verified, fully audited result. It is written to be acted on. Every chapter ends in workstreams with identifiers, acceptance gates, and the failure modes that would defeat them.

Who this is for

- **The owner.** You set direction, make the decisions listed in Chapter 26, and approve what reaches review.
- **Any engineer or agent working on Overseer.** Each workstream is specified closely enough to start from, with pointers into the code as it stands.
- **Reviewers.** The gates and the failure atlas (Chapter 22) are the checklist that a change is measured against.

What this document is, and is not

It is a roadmap: current state, target state, the path between them, and the evidence required to say each step is done. It is not a design record for any single feature. When a workstream starts, it gets its own decision record under `docs/research/` in the existing style (the memory v2, memory v3, tool economy and subagent tier records are the model), and where the decision record disagrees with this roadmap, the decision record wins.

It is also not a calendar. Work is sequenced by dependency and by gate, never by date. A wave closes when its gates pass, not when a week ends.

Baseline

Everything described as “today” refers to this exact state:

- `feat/fortify` at `cfc56a6`, stacked on `feat/memory-v3` at `f46ba1b` (PRs #54 and #55, open, not yet merged into review).
- Plus five reviewed but unmerged fix branches: `fix2-memory` (`68c1d62`), `fix2-agent` (`4b4a2b3`), `fix2-gateway` (`627b63c`), `fix2-tools` (`0bf3719`) and the local `fix2-computer` (`8e53d27`, including the lean pass).

Where a fix branch changes the picture, the text says so, tagged `[fix2]`. A reader working after integration should treat `[fix2]` items as present.

Size of the baseline, measured on `feat/fortify`:

Crate	Source lines	Test lines	Role
<code>overseer-core</code>	58,695	12,903	Engine: IR, event log, providers, tools, permissions, memory, subagents
<code>overseer-tui</code>	8,500	1,900	Terminal UI and the localhost browser UI
<code>overseer-gateway</code>	8,175	1,002	Always-on daemon, channels, triggers, inbox and outbox
<code>overseer-life</code>	4,772	1,615	Overseer Life connectors (Phase 0)
<code>overseer-cli</code>	3,788	1,834	The <code>overseer</code> command
<code>overseer-proto</code>	207	0	Wire types

Conventions

Status tags mark how far a capability is from existing:

Tag	Meaning
[have]	Present on the baseline and covered by tests
[fix2]	Present on a reviewed fix branch, lands at integration
[partial]	Exists but incomplete, with the gap named
[deferred]	Explicitly parked in code with a DEFERRED(owner) marker
[new]	Does not exist; this roadmap proposes it

Evidence tags follow the research notes:

Tag	Meaning
[code]	Read in the Overseer source at the cited path
[doc]	Primary vendor documentation
[src]	Secondary source, recorded in an earlier research note
[infer]	Reasoning, not measurement

Workstream identifiers are a letter plus a number, and they never change once published. The letters are:

Letter	Pillar
R	Reliability and long-horizon execution
O	Orchestration and subagents
M	Memory
C	Context management
T	Tool efficiency
K	Token and cache efficiency
P	Policy: principles, rules, AGENTS.md, instruction architecture
A	Automation: workflows, schedules and triggers
S	Security and trust
U	Computer use and the browser
L	Model layer
B	Observability
E	Evaluation and proof
X	Frontends and experience
Y	Extensibility and ecosystem
Z	Runtime, performance and distribution
F	Overseer Life integration

Waves are numbered W0 to W10 (Chapter 23). That is why automation workstreams use the letter A rather than W.

Callouts have fixed meanings:

PRINCIPLE

A rule that holds across the whole system. Breaking it needs a decision record.

DECISION

A choice the owner has already made, quoted or paraphrased with its date.

GATE

The evidence that must exist before a workstream or wave is called done.

WHAT COULD BREAK

A specific way the plan, or the system it describes, could fail.

NOTE

Context that helps but does not bind.

Standing rules that govern all of this work

These come from the owner and from `AGENTS.md`. They apply to every workstream in this document.

1. **No GitHub Actions spend.** Pushing a branch is fine. A pull request's newest commit carries `[skip actions]`. Scratch branches are transport only and are never opened as PRs.
2. **One cloud session at a time, and only deliberately.** Stress testing happens on the owner's Mac first.
3. **Full line-by-line review, then an independent adversarial review,** before anything lands on a PR branch.
4. **Deferred-item comments everywhere.** Every PR body ends with a Deferred section; every merge commit carries the same list; every code site that defers work carries `// DEFERRED(<owner>): <what> – <gate>`.
5. **Branch ladder.** Feature branches fork from `review` and target it; `review` promotes to `dev` for stress releases; `dev` promotes to `main` only for tagged releases.
6. **No spending, no sign-ups, no writes to other apps' credentials or data** without the owner's explicit say.

The Contract

The owner’s request for this document, verbatim:

“design a pdf and throw it into the github. a multi step, extremely detailed roadmap to make this harness crush every other harness like dust. dont tell me no, aim for the moon and youll land amongst the stars. we want literally the best. a true alfred to batman. i dont care if the pdf is more than 100 pages long or however many. extreme details. dont miss out on anything. dont compromise on literally anything. absolutely no gap to be left. get into this zone of the future, what could potentially break? think twice, thrice before the final edition of the roadmap.”

And the pillar framing that preceded it, also verbatim:

“listen, we have a few pillars. memory. agent orchestration and everything subagent related. context management. tool efficiency. token efficiency. sub pillars would be things like principles, rules, agents.md, so so so many things more to discuss”

“reliability has to be a main pillar too. long horizon automations, workflows, everything needs to be bulletproof. tasks can go on for 100+ hours no issue and uninterrupted if needed”

What this contract requires of the document

Requirement	Where it is met
Multi-step and extremely detailed	Every pillar chapter breaks into numbered workstreams with design, interfaces, edge cases and gates; Chapter 23 sequences them into waves
Crush every other harness	Chapter 4 sets the competitive bar per capability; Chapter 25 turns “crush” into measurable claims
A true Alfred	Chapter 1 defines the Alfred standard operationally; every pillar is tested against it
Nothing missed, no gap left	Seventeen pillars and systems, plus a coverage matrix in Chapter 23 mapping every gap found in the baseline audit to a workstream
What could potentially break	Every workstream carries its own risks; Chapter 22 is a full failure atlas plus premortems of a 100-hour run
Reliability as a main pillar, 100+ hour runs	Chapter 5 is the longest chapter, and reliability is the spine every wave depends on
Memory, orchestration, context, tools, tokens	Chapters 6 through 10
Principles, rules, AGENTS.md	Chapters 2 and 11
Thought through more than once	Appendix F records the review rounds this edition went through and what each changed

Prior owner decisions this roadmap honours

These were made in earlier sessions and are binding here.

DECISION

Computer-use consent (2026-10-07). In the default per-action mode, screen reads and mutating actions ask **once per task**. Full automation is **truly never ask**, with no Rule-of-Two safety floor. There is no mandatory hard denylist in full automation: “in full auto mode, we’ll have dedicated permissions

for secrets and things like this up to the user to configure.” Consent modes ship on their own branch after the current integration.

DECISION

fix2-memory finding M8 (2026-10-07). Option (a): apply the ASCII-alphanumeric neighbour rule for U+200C/U+200D and change the two affected test rows to a letter split.

DECISION

--bare skips user skills (fix2-tools finding S1, 2026-10-07). Implemented as an AgentConfig field threaded through prompt.rs and tools/skill.rs, not a process-global switch.

DECISION

Cloud posture (2026-10-07). One session at a time; stress testing runs locally. Weekly cloud quota is scarce.

DECISION

Overseer Life (2026-10-05). Me first, product later. Rust core with a native SwiftUI shell on macOS, iOS soon, Windows and Linux later. Backend before frontend. Repo stays inside Overseer. X is the first social platform.

DECISION

No project-level API key (2026-09-25). Providers read their own key names only.

Executive Summary

The one-paragraph version

Overseer already gets the hard, invisible things right: an immutable hash-chained event log, engine-enforced budgets, deterministic compaction as a view, byte-stable prompt prefixes, a taint-tracked permission gate, a memory system that learns, and a subagent system whose budgets compose. Most harnesses never get those foundations right. What Overseer lacks is everything that turns a correct engine into a tireless one: a supervisor that resurrects dead runs, retry and failover that outlive a provider outage, durable intent that survives a week of compactions, a policy layer that can express “ask once per task” for a hundred hours, an orchestration layer with dependencies and persistent agents, and a workflow engine for automations that run for months. This roadmap builds those layers on the existing spine, in eleven waves gated by evidence, and it ends with a harness that is measurably better than every alternative on the axes that matter: completion on long tasks, cost per solved task, cache hit rate, unattended survival, and the size of the blast radius when something goes wrong.

Where we stand, pillar by pillar

The score is distance to the target defined in each chapter, not to today’s competitors. 100 means “the target in this roadmap is met and proven”; 0 means nothing exists. The scores are judgment over a code audit, not measurements, and they are honest rather than flattering.

Pillar	Score	What exists	What is missing
Memory	70	Two git-versioned stores, five layers, BM25F + activation ranking, recall, prospective reminders, learning review, threat scan, pending queue	Session history search, applied learned skills, dream pass, outcome memory, run journals, mounted repos
Cache discipline	75	Frozen segment order, prefix built once per agent, three Anthropic breakpoints, byte-stable specs, cache telemetry with an alert below 90%	Fourth breakpoint, 1-hour TTL, keepalive, cache-aware compaction timing
Token efficiency	65	Spend gate, TOON re-encoding, spill to file, read dedup, image caps, small-model aux tier, effort ladder	Real tokenizer, cross-turn result dedup, digest aging, cost-aware routing, effort decay
Tool layer	65	18 core tools, deferred catalog behind tools, QuickJS run_code, MCP with trust lanes	Composition reach, hashline edits, native deferral, project MCP config, LSP diagnostics
Security	65	Rule-of-Two latches, sandboxed bash, credential broker sentinels, hardened web surface, injection corpus	Egress proxy, consent modes, scoped grants, signed builds, long-run threat model
Context management	55	Deterministic compaction, keep-last-5 eviction, image caps, provenance wrapping	Token-aware eviction, retrieval-scoped context, notice aging, plan pinning, context planner
User experience	50	Full-screen TUI, inline mode, plain REPL, hardened localhost web UI, rewind and diff overlays	Long-run steering, inbox frontend, ACP/IDE bridge, mobile approvals, run timeline
Model layer	50	Anthropic, OpenAI-compatible, Responses, Gemini, opencode adapters; profiles; effort map	Failover ladders, retry, capability probes, deprecation handling, local models

Pillar	Score	What exists	What is missing
Evaluation	45	Rig with paired statistics, 40+5 tasks, SWE-bench/Terminal-Bench/ τ^2 adapters, control scaffold	No live claims yet, no long-horizon or chaos suites, eval hold in force
Computer use	45	Lean cua-driver backend, marks, modal gate, post-action diffs, model-sized images	Never run against a live desktop, consent modes, progress guard, settle waits
Orchestration	40	Task modes and tiers, worktrees, budget carving, verify, consult, control tree	DAG, artifacts, persistent agents, teams, merge queue, depth cap
Extensibility	40	Skills, MCP, hooks, microagents, modes	Plugin model, versioning, project MCP, user modes, trust tiers
Distribution	35	cargo-dist scaffold, lean dependencies, ~5.5 MB binary	Signed builds, installers, self-update, iOS gating
Reliability	30	Durable log with torn-tail repair, resumable agents, subagent reaping, timeouts on most calls	Supervisor, auto-resume, retry/backoff, failover, wall-clock budgets, durable intent, away policy
Observability	30	Per-call ledger rows, cache stats, overseer stats , run summary line	Cross-session rollups, traces, timeline, alerts, export
Rules and policy	25	<code>~/overseer/rules</code> , hooks, microagents, six modes, autonomy ladder	AGENTS.md loader, scoped grants, deny rules, project scope, policy compiler, management UI
Workflows	10	Single-shot gateway triggers with triage and an EV gate	Everything: durable workflows, chains, conditions, retries, compensation, schedules
Overall	~45	A correct engine	A tireless one

The twelve commitments

These are what “crush every other harness” means in practice. Each one is measurable, and Chapter 25 says how.

1. **A run never dies of a transient.** Rate limits, network drops, provider outages, crashes, reboots and laptop sleep pause a run; they do not end it. Only budget exhaustion, an owner decision, or a genuine dead end ends a run, and every end has a recorded reason.
2. **A hundred hours is routine.** A run can go for over a hundred hours of wall-clock time without human intervention, with cost, progress and health visible the whole time, and resume from any point after any failure.
3. **Intent outlives context.** What the agent is trying to do, what it has done and what comes next live in a durable journal, not in the context window. A run resumed after a week knows exactly where it stands.
4. **Every token earns its place.** In-session cache hit rate stays at or above 92% on Anthropic models, resident startup stays under 2,000 tokens, and cost per solved task is published and beats every competitor measured on the same model.
5. **Orchestration is a graph, not a call.** Subagents have dependencies, shared artifacts, persistent identities, and a merge queue; parallel writers never corrupt shared state.
6. **Policy is code, with a single compiler.** AGENTS.md, rules, grants, modes, hooks and memory-derived conventions compile into one ordered instruction and permission set, with provenance for every line and a user-visible explanation for every decision.

7. **Automations are durable workflows.** A trigger starts a versioned workflow whose every step is journaled, retried, compensated on failure, and resumable across upgrades.
8. **Memory learns outcomes, not just facts.** The harness remembers what worked, what failed, and why, per project, and uses it to choose approaches.
9. **Verification is mandatory and independent.** Nothing a run produces is called done without an executable check or a fresh-context verifier, and “unknown” is never a pass.
10. **The blast radius is engineered.** Every side effect is classified, gated, journaled, idempotent where possible and reversible where possible; secrets never enter context; untrusted content can never trigger exfiltration silently, except where the owner has explicitly chosen full automation.
11. **Everything is observable and explainable.** Any decision the harness made, from a permission verdict to a compaction cut to a retry, can be traced to its cause from the event log.
12. **Leanness is a gated property.** Binary size, memory use, startup time and resident tokens have budgets that the build enforces. Capability is added without bloat.

The waves at a glance

Chapter 23 has the full dependency graph. The short form:

Wave	Theme	Unlocks
W0	Land what exists: integrate the fix branches, merge #54 and #55, close the current milestone	A clean base
W1	Survival: retry, backoff, failover, wall-clock budgets, timeout holes, disk guards	Runs stop dying of transients
W2	The supervisor: heartbeats, liveness, auto-resume, re-adoption after reboot	Runs survive crashes and reboots
W3	Durable intent: run journal, plan artifact, progress ledger, resume briefings	Runs survive a week of compactions
W4	Policy compiler: AGENTS.md loader, scoped grants, deny rules, consent modes, away policy	Long runs can be safely unattended
W5	Context and token engine: token-aware eviction, dedup, aging, tokenizer, cache keepalive, 1-hour TTL	Cost per solved task drops sharply
W6	Orchestration graph: DAG, artifacts, merge queue, depth cap, persistent agents	Real parallel work
W7	Workflow engine: durable workflows on the gateway, schedules, compensation, inbox	Automations that run for months
W8	Memory v3 H2/H3 and outcome memory	The harness gets better with use
W9	Proof: long-horizon and chaos evaluation suites, published harness-contribution claims	“Best” becomes a measured fact
W10	Reach: ACP/IDE bridge, mobile approvals, signed distribution, Overseer Life integration	Everywhere the owner is

Computer use, security, observability and extensibility workstreams run alongside the waves at the points their dependencies allow; Chapter 23 places each one.

The ten things most likely to go wrong

Chapter 22 has the full atlas. These are the ones that would hurt most.

#	Risk	Mitigation in this plan
1	An auto-resumed run repeats a side effect it already performed (double send, double purchase, double push)	Side-effect journal with intent-before-act records and idempotency keys (R7); resume refuses to replay an act whose outcome is unknown without verification
2	A long unattended run drifts from its goal while looking busy	Durable intent journal (R5), progress ledger with stall detection (R9), periodic fresh-context goal audits (O9)
3	Cost runs away over a hundred hours	Rate budgets (\$ per hour and per day) with pause-not-kill breakers (R10), forecasting (B4), per-wave cost alerts
4	A provider changes or retires a model mid-run	Capability probes and failover ladders with recorded equivalence (L3, L4)
5	Prompt injection in content read on hour 60 steers the run	Taint latches persist across resume (have), workflow steps inherit taint (A6), full-auto mode remains an explicit owner choice with its own audit trail (S4)
6	An upgrade of the Overseer binary breaks an in-flight run	Event schema versioning and upgrade-safe resume (R13); the supervisor drains or pins the old binary
7	Event logs and spill files fill the disk over a week	Snapshots plus archival (R11), spill garbage collection, disk preflight and pause thresholds
8	Parallel writers produce conflicting changes	Merge queue with rebase-and-verify (O6), serialized integration, never parallel writers on shared state
9	Complexity erodes the leanness that made Overseer fast	Budgets enforced in the build (Z1), every subsystem must earn its tokens and bytes, measured on the eval rig
10	The plan is too big to finish	Waves are independently valuable, each closes on its own gates, and W1 through W3 alone remove most failure classes



Part I — Foundations

1. The North Star: What “Alfred” Means

Alfred Pennyworth is not a tool Bruce Wayne operates. He anticipates, he remembers, he runs the house for weeks while Bruce is away, he patches wounds at three in the morning without being asked how, he tells the truth when it is unwelcome, and he never once leaks the secret. The harness we are building has to meet that standard, translated into engineering.

1.1 The Alfred standard, operationally

Trait	What it means for the harness	Measured by
Tireless	Works for days without supervision. Pauses on failure, never dies of a transient.	Unattended survival rate on the long-horizon suite (E2); mean time between owner interventions
Remembers	Knows the owner, the projects, the conventions, what worked last time and what failed. Never asks the same thing twice.	Repeat-question rate; recall precision on the memory eval (E6)
Anticipates	Notifies what needs doing (reminders, failing CI, expiring certificates, quota running out) and prepares it, but only interrupts when the value of interrupting is high.	Inbox precision (items acted on ÷ items raised); interruption rate
Competent	Solves the task correctly, at the lowest cost that does it well.	Pass rate and cost per solved task on the rig, against competitors on the same model
Honest	Reports what it actually verified. “Unknown” is never “done”. Owns mistakes in the record.	Verified-claim rate; false-done rate in audits
Discreet	Secrets never enter context or logs; untrusted content cannot silently drive actions; every side effect is accountable.	Secret-leak tests; injection attack success rate (ASR) on the corpus; audit completeness
Obedient to the right person	Follows the owner’s rules and preferences over anything it reads, explains every refusal, and can be stopped instantly from anywhere.	Policy-violation rate in red team; kill-switch latency from each frontend
Unobtrusive	Lean in tokens, memory, CPU and attention.	Resident tokens, RSS, startup time, binary size budgets (Z1)
Self-improving	Gets measurably better at the owner’s work over time.	Learning-loop uplift: pass-rate delta between a cold and a warm memory store on repeated task families

1.2 Horizons: what the harness must handle at each timescale

A harness that is excellent at five-minute tasks can be useless at five-day tasks, because different failure modes dominate at each scale. The plan addresses every horizon explicitly.

Horizon	Typical job	Dominant failure modes	Pillars that carry it
Seconds	Answer a question, run a command	Latency, startup cost, wrong tool	Runtime (Z), tools (T)
Minutes	Fix a bug, write a feature	Wrong diagnosis, missing verification, loops	Verification (O), stuck detector, context (C)
Hours	Refactor a subsystem, research a topic	Context exhaustion, drift, cost, rate limits	Context (C), tokens (K), reliability (R1–R3)

Horizon	Typical job	Dominant failure modes	Pillars that carry it
Days	Migrate a codebase, run an audit across repos	Crashes, reboots, provider outages, lost intent, approvals while away	Reliability (R4–R10), policy (P4–P5), orchestration (O)
Weeks	Ongoing project management, a long research programme	Environment drift, upgrades, disk growth, stale memory	Reliability (R11–R16), memory (M), workflows (A)
Months	Standing automations: inbox triage, quota watching, renewals	Workflow versioning, trigger reliability, notification fatigue	Workflows (A), gateway, Life (F)

1.3 Five journeys the finished harness must handle

Each journey is a test the whole system has to pass end to end. Chapter 17 turns them into evaluation scenarios.

Journey A: the 100-hour migration

The owner asks for a migration of a large codebase from one framework to another, with every test passing at the end, and leaves for a long weekend.

1. The lead agent reads AGENTS.md, memory and the repository, writes a plan artifact to its run journal, and decomposes the work into a graph: inventory, per-module migrations, integration, verification.
2. Writers run in isolated worktrees under a merge queue. Each merge is rebased, built and verified before it lands. Readers fan out to answer questions in parallel.
3. On hour 9 the provider returns 529s for forty minutes. The run backs off, fails over to the configured second model for read-only subtasks, and resumes on the primary when it recovers. The journal records it all.
4. On hour 31 the laptop reboots for an OS update. The supervisor restarts with the login session, finds the run in `running` with a stale heartbeat, verifies the last side effect in the journal completed, and resumes.
5. On hour 52 a writer wants to run a command outside its grants. Under the owner’s away policy the request waits, notifies the owner’s phone, and after the deadline is denied once; the agent routes around it and records the gap for the owner.
6. On hour 97 the merge queue is empty, the verifier passes, and the run ends with a report: what changed, what was verified and how, what remains, what it cost, every decision it took on the owner’s behalf.

Journey B: the overnight research programme

The owner asks for a deep comparison of approaches to a technical problem, with sources. Readers fan out across topics, write findings as artifacts, a synthesis agent merges them, a fresh-context verifier checks every citation resolves and says what it claims, and the result lands in the inbox with confidence marked per claim.

Journey C: the standing automation

The owner defines a workflow: every morning at 07:00, check AI quota usage across accounts (Overseer Life connectors), check CI on open PRs, check certificate expiry on the owner’s domains, and send one digest. It runs for months. Its definition is upgraded twice without losing state. When a connector breaks, the digest says so instead of silently omitting the section.

Journey D: the incident at 3 a.m.

A webhook reports a failing deploy. The gateway’s triage classifies it as urgent; the EV gate decides it is worth waking the owner only if the agent cannot handle it. An agent run spawns with read access, diagnoses, prepares a rollback, and because rollback is an external side effect, asks. The owner approves from the phone. The run executes, verifies, and files a report.

Journey E: the life admin day

Through Overseer Life, a specialist agent notices a free trial converting to a paid plan tomorrow, drafts the cancellation, and asks; a second agent notices quota on one AI account is nearly gone and routes the day’s Overseer runs to another account; a third notices the X follower count dropped sharply and flags it. Each runs under its own scoped permissions on the shared engine.

1.4 What Overseer will not become

Saying no is part of the design. These are permanent non-goals unless a decision record reverses one.

- **Not a chatbot with tools bolted on.** The engine is the product; conversation is one frontend.
- **Not bloated.** No tokio in core, no heavyweight frameworks, no Chromium in the harness binary. Every dependency is budgeted.
- **Not a learned router.** Model routing stays deterministic and explainable (subagent tiers decision, 2026-09-30).
- **Not parallel writers on shared state.** Writers isolate and serialize through a merge queue.
- **Not self-critique as verification.** Verification is executable or fresh-context.
- **Not a cloud service.** Local-first. Cloud is an optional executor, never the source of truth.
- **Not a scraper of other apps’ secrets.** Overseer never refreshes another app’s login, never writes another app’s files.
- **Not opaque.** If the harness did it, the log says why.

2. The Constitution: Principles and Invariants

Overseer already has nine invariants in `AGENTS.md`. They are why the engine is trustworthy, and none of them changes. This chapter restates them, adds the invariants that long-horizon operation and the new pillars require, and sets the design principles and refusals that decide close calls.

2.1 The existing invariants (unchanged)

#	Invariant	Why it matters for the roadmap
1	Events are immutable; session state is a view over <code>events.jsonl</code> .	Every reliability, observability and replay feature in this plan is built on this.
2	Stable prompt prefixes: nothing volatile above the cache boundary.	The 92% cache target depends on it; policy compilation (P2) must respect it.
3	The engine enforces budgets, never the model.	Extends to wall-clock and rate budgets (R3, R10).
4	Tool results are budgeted: about 30K characters inline, then spill to file.	Extends to token-aware eviction (C1).
5	Read-before-edit is enforced by the harness, not the prompt.	Extends to hashline edits (T1).
6	Reasoning blocks are opaque: round-trip verbatim, never inspect or mutate.	Constrains reasoning shedding (C4) to dropping whole blocks where the provider allows it.
7	The raw provider <code>stop_reason</code> is preserved end to end.	Feeds the failure taxonomy (R1).
8	Compaction is a view over the event log, never a mutation; summaries are mechanical and never re-summarized; the recency tail starts on a model-response boundary.	Durable intent (R5) must not violate it: the journal is a separate artifact, not a summary of summaries.
9	Checkpoints live per user prompt; write and edit snapshot before first touch; <code>bash</code> side effects are not snapshotted.	The side-effect journal (R7) closes the <code>bash</code> blind spot as far as it can be closed.

2.2 New invariants

Each new invariant is introduced by the workstream named, and from then on is enforced the same way the existing ones are: by tests, by review, and where possible by construction.

#	Invariant	Introduced by
10	A run ends only for a recorded reason. Every <code>RunEnd</code> carries a reason from a closed set; “crashed” is not a reason, it is a state the supervisor resolves.	R1, R4
11	Transient failures pause; they never end a run. A failure classified transient is retried under a bounded, journaled policy.	R1, R2
12	Intent is journaled. The run’s goal, plan, progress and next step live in a durable journal that is not part of the context window and not derived from compaction.	R5
13	Side effects are recorded before they happen. Every classified side effect writes an intent record before execution and an outcome record after. Resume never re-executes an intent with an unknown outcome unless its family’s declared in-doubt policy permits it (an idempotency key the far side honours, or a declared safe replay) or a verification established the outcome. Effects with no observable read-back are detected and gated, never guessed.	R7
14	Every wait has a deadline and an escalation. No component waits forever, including on a human. Waiting on a human has an owner-configured policy.	R3, R8
15	Every budget has a rate form. Long runs are bounded by cost per hour and per day as well as totals, and breaching a rate pauses the run rather than killing it.	R10
16	Resource growth is bounded. Logs, spills, worktrees, caches and memory stores have caps and collection policies.	R11

#	Invariant	Introduced by
17	Policy has one compiler. Every instruction and permission source compiles through one ordered, provenance-preserving pipeline; no subsystem consults raw rules on its own.	P2
18	Every decision is explainable. Permission verdicts, compaction cuts, evictions, retries, failovers and routing choices are logged with their cause.	P11, B1
19	Taint flows with data. Content derived from untrusted input carries its taint through subagents, artifacts, workflow steps and memory.	A6, O3, S3
20	Writers never share a working tree. Parallel writers isolate; integration is serialized through a merge queue with verification.	O6
21	Old logs always replay. New event fields default; new event kinds are skipped by old readers; every schema change has a migration test.	R13
22	Leanness is enforced by the build. Binary size, RSS, startup and resident tokens have budgets that fail the gate when exceeded.	Z1
23	Claims are measured. A capability is not called better until the eval rig shows it, with paired statistics, on the same model.	E1
24	The owner can always stop everything. A kill switch reaches every running agent and workflow from every frontend within a bounded latency.	S10
25	Secrets the harness holds (brokered credentials, environment and keychain values) never enter model context, logs or child environments. Already true in practice; now an invariant every new subsystem must test. Secrets the owner pastes into a conversation do enter context; S7 detects them after the fact and drives redaction and rotation.	S2, S7

2.3 Design principles

These decide the close calls that invariants do not cover.

PRINCIPLE

Earn every token, byte and millisecond. A subsystem justifies its resident tokens, binary size and latency with a measured benefit. Minimal scaffolds capture more of each model generation's improvement; complexity that encodes assumptions about model weakness rots.

PRINCIPLE

Correct first, then durable, then fast, then clever. The order of work inside every pillar.

PRINCIPLE

Mechanism in the engine, policy in configuration. The engine provides retries, gates, budgets and graphs; the owner's configuration decides how they are used.

PRINCIPLE

Deterministic where possible, model-driven where necessary. Routing, compaction, triage rules, eviction and retry are deterministic. Model calls are reserved for judgment, and every model call is ledgered and bounded.

PRINCIPLE

Fail safe, fail loud, fail recoverable. An uncertain state refuses rather than guesses, says so visibly, and leaves the system resumable.

PRINCIPLE

One way to do each thing. One compiler for policy, one journal for intent, one ledger for spend, one log for events. Parallel mechanisms drift.

PRINCIPLE

The event log is the source of truth; everything else is a cache. Indexes, journals' derived views, dashboards and memory indexes must be rebuildable from durable sources.

PRINCIPLE

The harness enforces; the prompt informs. Prose rules shape what the model attempts. Only engine enforcement decides outcomes.

PRINCIPLE

Steal ideas, never dependencies blindly. Borrow the best idea from every harness with attribution; take a dependency only after it passes the budget and cargo deny.

PRINCIPLE

Every deferral is written down. Anything left out carries a `DEFERRED(owner)` marker with its gate, and appears in the PR and merge commit.

2.4 Refusals

The things we will not build, and why. Reversing one needs a decision record.

Refusal	Reason
Parallel writers on a shared tree	Corrupts state; Cognition's and Anthropic's evidence agrees writes should serialize
Self-critique as verification	Ungrounded self-correction degrades results (Huang et al. 2023)
A learned router	Needs training data we do not have, and is unexplainable
Full-transcript handoffs between agents	Briefs and results, not transcripts; transcripts waste tokens and leak taint
Vector search in the memory core	Size, poisoning surface, and BM25F + activation already meets the bar (memory v2/v3 records)
LLM query rewriting before every recall	Adds a model call to every turn
Automatic verification without an executable check	A verifier with nothing to run is self-critique by another name
Tokio or an async runtime in core	Leanness; the sync design with threads is sufficient and simpler to reason about
Reading or refreshing other apps' credentials	Standing Overseer Life rule
Silent downgrade of a pinned sandbox runtime	Already a rule for <code>--runtime</code> ; generalizes to every security setting

2.5 How decisions get made

1. A workstream that changes architecture starts with a decision record under `docs/research/YYYY-MM-DD-<slug>.md`, in the existing format: problem measured, evidence with tags, decisions, deferred items with owner and gate.
2. The record is reviewed before code starts. Disagreements between the record and this roadmap resolve in favour of the record, and the roadmap gets an erratum in its next edition.
3. Implementation follows the standing review rule: line-by-line review, then independent adversarial review, then the PR.
4. Every PR ends with its Deferred section; every merge commit carries it.
5. Owner decisions are recorded verbatim with their date, as in the Contract section of Part 0.

3. Where We Stand: The Baseline Audit

This chapter is the honest inventory the rest of the roadmap builds on. It was produced by reading the code on the baseline (Part 0), not by reading `AGENTS.md`. File and line references are to `feat/fortify` at `cfc56a6` unless tagged `[fix2]`. Line numbers drift; the symbol names are the durable pointer.

3.1 The engine loop

Mechanism	State	Where
ReAct loop with step and cost budgets	[have] <code>max_steps</code> default 100, <code>max_cost_usd</code> default \$5	<code>agent.rs</code> (budget check near the loop top)
Spend gate	[have] Worst-case pre-pricing (input at the higher of fresh and cache-write price plus <code>max_tokens</code>); clamps <code>max_tokens</code> and thinking budget to what is affordable; refuses below 1,024 output tokens; exactly one ledger row per call, answered or failed	<code>ledger.rs</code> Gate
Stuck detector	[have] Five patterns: same call pair 4×, same error 3×, three or more reasoning turns with no tool call or progress (monologue), ping-pong over 6, context errors 3×. First trip nudges and bumps effort; second trip ends the run	<code>stuck.rs</code> ; handling in <code>agent.rs</code>
Empty-response guard	[have] Three empty responses end the run	<code>agent.rs</code>
Malformed-response retry	[have] Exactly one retry on <code>Malformed</code>	<code>agent.rs</code>
Provider error handling	[partial] <code>RateLimit</code> , <code>Http</code> and <code>Transport</code> errors end the run as <code>provider_error</code> . <code>retry_after_ms</code> is parsed (<code>provider/mod.rs</code>) but only surfaced in the message	<code>agent.rs</code> , <code>provider/mod.rs</code>
Verify gate	[have] <code>verify_cmd</code> runs in the sandbox with a fixed 120 s timeout; output tail returns to context; capped at 8 blocks	<code>agent.rs</code>
Stop hooks	[have] stop hooks can veto a stop; they share the verify cap and fail open	<code>hooks.rs</code> , <code>agent.rs</code>
Interrupt and steer	[have] <code>Control</code> with interrupt and a steer queue, checked at tool-launch boundaries; skipped calls get synthetic results so pairing survives	<code>control.rs</code> , <code>agent.rs</code>
Microagents	[have] <code><cwd>/overseer/microagents/**/MICROAGENT.md</code> with <code>triggers:</code> (substring or <code>always</code>), injected per user turn as provenance-wrapped nudges, 8 KB per body, 16 KB per turn	<code>microagent.rs</code>
Modes	[have] Six built-in modes (code, architect, ask, debug, docs, orchestrator); user-defined modes deferred	<code>modes.rs</code>

3.2 Durability and recovery

Mechanism	State	Notes
Append-only event log	[have] JSONL, hash chain v2 (sha256 over previous hash plus canonical payload) on new logs, v1 structural hash on old	<code>event.rs</code>
Torn-tail repair	[have] <code>EventLog::open</code> repairs a torn final line; a corrupt earlier line is an error naming the line	<code>event.rs</code>
Crash mid-tool-call	[fix2] Reused call-id pairing and placeholder results; torn tail keeps complete events	<code>fix2-agent</code>

Mechanism	State	Notes
Checkpoints and rewind	[have] Per user prompt; write/edit snapshot before first touch; <code>bash</code> effects not recorded	<code>rewind.rs</code> , <code>session.rs</code>
Resume	[have] <code>Agent::resume</code> rebuilds from the log; manual via <code>--resume</code> , <code>--continue</code> , <code>--last</code>	<code>agent.rs</code> , CLI
Liveness lock	[have] <code>live.lock</code> held for an agent's life; refuses a second process's <code>learn</code> , <code>resume</code> and <code>rewind</code>	<code>live.rs</code>
Rewind lock	[fix2] Rewind holds the live lock	<code>fix2-tools</code>
Subagent reaping	[have] Sidecar nonce per process; <code>reap_dead</code> marks foreign-nonce running tasks dead and writes a synthetic marker; delivery set prevents double drain	<code>tools/task/sidecar.rs</code>
Automatic resume	[new] Nothing restarts a dead top-level run	—
Supervisor	[new] No process owns run liveness	—

3.3 Timeouts and deadlines

Site	Bound	On expiry
<code>bash</code>	120 s default, 600 s max	Kill; [fix2] now kills the whole process group, so <code>cmd</code> & no longer hangs the agent thread
<code>verify_cmd</code>	120 s fixed	Kill, tail into context
<code>diagnostics</code> , <code>struct_search</code>	120 s (+5 s pipe grace for the latter)	Kill, fail open
MCP calls (including initialize and <code>tools/list</code>)	30 s	Server killed and reaped; next use respawns
<code>computer</code> (cua-driver)	30 s per MCP call; <code>wait_for</code> ≤ 10 s, interrupt-aware	Error surfaces
<code>run_code</code>	30 s default, 120 s max; 5 s engine grace	Error result
Provider HTTP	600 s global (<code>ureq timeout_global</code>)	Run ends
Memory store lock	10 s acquire	BUSY
Live lock	2 s	BUSY
Ask dialog	None	Blocks the agent thread for as long as the human takes
Gateway control socket	10 s request deadline, 64 KiB lines, 16 connections; [fix2] reply shares the request deadline	Error
Telegram	15 s per call, long-poll 0	Error; runs inside the daemon tick, so a slow call stalls the tick
Desktop notifier	None (<code>Command::status()</code> on <code>osascript/notify-send</code>)	Can stall the daemon tick
Spawned gateway runs	600 s watchdog	Kill
Wall-clock run budget	None	—

3.4 Context and tokens

Mechanism	State
Prompt assembly	[have] <code>prompt::assemble</code> emits named segments in a frozen order: identity, contract, safety, memory, skills, MCP, persona. Assembled once per agent (start, resume, model switch). Fingerprint and hygiene lints in tests

Mechanism	State
Anthropic caching	[have] Three of four breakpoints: last cacheable system segment, tools tail, rolling breakpoint on the last eligible message block. 5-minute TTL only
OpenAI caching	[have] <code>prompt_cache_key</code> = session id, official hosts only; cache fields parsed for Responses and DeepSeek
Gemini caching	[have] Implicit only; <code>cachedContentTokenCount</code> parsed
Compaction	[have] Deterministic, view-only, two-turn tail on model-response boundaries, fixed-schema mechanical summary, triggered at a per-profile fraction (0.70–0.83) of context, by estimator, or on context-window errors
Result budgeting	[have] 30,000-character inline cap; TOON re-encoding of uniform JSON arrays first; credential redaction; spill to <code>tool-outputs/output-N.txt</code> (0600) with a 4K preview; middle truncation fallback
Eviction	[have] Keep the last 5 tool results; pin the last 3 <code>tools op=search</code> results; keep the last 2 images; reflection nudges keep last 1. Count-based, not token-based
Read dedup	[have] [unchanged] stub on a (path, mtime, range) hit
Token estimation	[partial] Characters-per-token heuristic calibrated per family (<code>tokens.rs</code>); no real tokenizer
Bounded prompt artifacts	[have] Memory resident $\leq 4,000$ bytes / 24 lines; INDEX ≤ 25 KB; CORE ≤ 1.5 KB; repo map $\leq 4,000$ bytes; subagent digest $\leq 8,000$ characters
Telemetry	[have] <code>UsageRecord</code> per call with cache hit rate; <code>CacheStats</code> with an alert below 0.90; per-run cache delta on <code>RunEnd</code> ; <code>overseer stats</code> ; TUI shows “NN% cached”

3.5 Tools

Mechanism	State
Registry	[have] 18 core names; specs name-sorted and byte-stable; optional tools detected once per registry (<code>computer</code> , <code>struct_search</code> , <code>diagnostics</code> , <code>skill</code>)
Advertised by default	[have] <code>bash</code> , <code>read</code> , <code>write</code> , <code>edit</code> , <code>grep</code> , <code>glob</code> , <code>task</code> , <code>memory</code> , <code>tools</code> , plus <code>run_code</code> (feature <code>code-mode</code>) and <code>skill</code> when a skill exists
Deferred behind <code>tools</code>	[have] <code>computer</code> , <code>diagnostics</code> , <code>mcp</code> , <code>plan</code> , <code>repo_map</code> , <code>struct_search</code> , <code>symbol</code> , and every MCP tool
Call pipeline	[have] disabled $\rightarrow$ unavailable $\rightarrow$ pre-tool hooks $\rightarrow$ broker sensitive latch $\rightarrow$ schema check $\rightarrow$ gate $\rightarrow$ dispatch $\rightarrow$ memory observe $\rightarrow$ policy notes $\rightarrow$ post-tool hooks $\rightarrow$ taint on raw text $\rightarrow$ credential sanitize $\rightarrow$ budget
<code>run_code</code>	[have] QuickJS with no <code>std/os/modules</code> ; 64 MB heap, 1 MB stack, 64 sub-calls, 8 MB sub-call bytes, 16K print, ≤ 120 s; cannot reach <code>computer</code> , <code>memory</code> , <code>plan</code> , <code>run_code</code> , <code>skill</code> , <code>task</code> , <code>tools</code>
MCP	[have] <code>~/overseer/mcp.json</code> only, <code>\${VAR}</code> expansion, per-server <code>trust: read ask</code> , lazy spawn, respawn on failure, env allowlist, shadowing names skipped
Computer use	[fix2] Lean pass: <code>cua-driver</code> is the only backend; model-visible output stripped of audit fields; model-sized screenshots with one refit retry; lean observation rendering; snapshot-gated post-reads; whole batch validated before dispatch
Web access	[new] No web fetch or search tool; bash network is denied by default

3.6 Orchestration

Mechanism	State
Task modes	[have] <code>read</code> (<code>read/grep/glob/memory</code>), <code>write</code> (full core in a worktree), <code>verify</code> (<code>read</code> plus sandboxed <code>bash</code>), <code>consult</code> (one heavy call, no tools)
Tiers	[have] <code>light/standard/heavy</code> derived from profiles, same transport only; defaults <code>read</code> $\rightarrow$ <code>light</code> , <code>write/verify</code> $\rightarrow$ <code>standard</code> , <code>consult</code> $\rightarrow$ <code>heavy</code>

Mechanism	State
Budget composition	[have] Cap = min(requested or tier default, parent remaining); reservations at launch; settlement into the parent ledger. Tier caps: light \$0.25, standard \$1, heavy \$2, consult \$0.50
Worktrees	[have] overseer/<sha8 of session path>/task-N branches; no-diff writers cleaned up; [fix2] gitignored writer output counts as a change
Background	[have] Max 4 in flight (config), digests $\leq$ 8,000 characters as SubagentDone notices; drained at the loop top
Escalation	[have] read/consult retry once one tier up on step cap, stuck, empty or error; writers never auto-retry
Verify	[have] Fresh-context verifier with JSON verdict and git tamper evidence; unparsable verdict is unknown, never pass
Cancellation	[have] task action=cancel at the next step boundary; parent interrupts fan out; [fix2] cancel holds the reservation until settlement
Subagent limits	[have] $\leq$ 20 steps, no auto-compact, no recall, no learning, filtered read-only memory
Task dependencies	[deferred] No after;; task.rs carries the DEFERRED marker
Shared artifacts	[new] Only digests and incidental files
Depth cap	[new] Writers see task and can spawn grandchildren; money is bounded by the account chain, depth is not
Persistent agents	[new] Every subagent is ephemeral
Foreground detach	[new] A foreground task blocks the parent for its whole run

3.7 Memory

Mechanism	State
Stores	[have] User and project, git-versioned, five layers with ADD-only INDEX files
Search	[have] In-memory BM25F fused by RRF with Petrov activation and confidence; activation and confidence vote only within the lexical top tier
Writes	[have] $\leq$ 5 per user turn; per-layer write bars; supersede refused under the untrusted latch; quarantine to proposals/
Recall and reminders	[have] $\leq$ 3 notes on user input with a trivial-prompt skip; prospective at:/kw:/path: reminders, fired once
Learning review	[have] Run-end review with signal lexicon and cadence, $\leq$ 14K digest, line protocol, write policy, pending queue, threat scan; [fix2] fixes for review findings M1–M7 and S2–S4 (finding numbers from the fix2-memory review, not workstreams in this roadmap)
Store IO safety	[have] No link following below the root, temp-file plus rename, O_NOFOLLOW reads, flock with 10 s acquire, drift check on unlocked model calls
History search, learned-skill application, dream pass, mounts, sync	[deferred] Memory v3 H2 and H3

3.8 Policy and rules

Mechanism	State
Presets and ladder	[have] ReadOnly, WorkspaceWrite, Plan; irreversibility classes Read, InternalWrite, ExternalComms, Money, Identity; per-domain autonomy from Observe to ActSilently
Hard deny	[have] Containment, bash deny globs, escalation tokens, ahead of every allow
Persisted rules	[partial] ~/ .overseer/rules: one tool:resource key per line, optional @turns=N TTL; exact match; global scope; append-only; no deny entries; no list or remove surface. [fix2] grant

Mechanism	State
	keys now exist for non-bash tools, and a persisted grant key can no longer smuggle an <code>@turns=</code> TTL (a hand-written trailing <code>@turns=N</code> still parses)
Rule-of-Two	[have] Untrusted $\times$ sensitive latches arm the exfil gate; <code>Tainted{latch}</code> events; re-armed on resume; [fix2] untrusted floor survives approval
Hooks	[have] <code>hooks.json</code> data rules: <code>pre_tool_use</code> blocks, <code>post_tool_use</code> annotates, <code>stop</code> vetoes
AGENTS.md	[new] Never enters the prompt. Project conventions reach the model only through microagents, memory, or the model reading the file itself
Instruction compiler	[new] Each source is consulted where it is used; no single ordering, provenance or explanation

3.9 Gateway and automation

Mechanism	State
Daemon	[have] Single-threaded tick: kill switch $\rightarrow$ control drain $\rightarrow$ trigger poll $\rightarrow$ process $\rightarrow$ reap $\rightarrow$ heartbeat
Triggers	[have] Interval, file watch (mtime), heartbeat, 5-field cron, webhook spool, Telegram poll
Pipeline	[have] Dedup $\rightarrow$ triage (first-match class rules: Ignore, Notify, DraftForReview, Act) $\rightarrow$ untrusted Act downgraded to DraftForReview $\rightarrow$ EV gate (benefit - cost $>$ 0, quiet hours, focus) $\rightarrow$ push, inbox or silent
Spawn	[have] <code>overseer exec --bare --max-steps N</code> child; max 2 concurrent; overflow to inbox; untrusted origins get <code>--autonomy external=approve</code> and an env marker; 600 s watchdog
Inbox and outbox	[have] Durable stores; draft $\rightarrow$ approve $\rightarrow$ send
Webhook replay	[fix2] Replay table never evicts young entries
Workflows	[new] One event $\rightarrow$ at most one run. No chains, conditions, retries, fan-out, compensation or versioned definitions

3.10 Security

Mechanism	State
Startup hardening	[have] <code>umask 0o077</code> , proxy-env scrub, owner-only roots
Sandbox	[have] <code>sandbox-exec</code> on macOS, <code>bwrap</code> on Linux, network denied, writes confined, secret dirs read-denied; pinned runtimes never downgrade
Credential broker	[partial] HMAC sentinels under a per-process key; consent recorded for audit; rate and window enforcement not wired
Web surface	[have] Per-install token in the URL fragment, Host allowlist, Origin and Sec-Fetch-Site checks, size and time limits, CSP
Injection corpus	[have] <code>tests/injection_asr.rs</code> gates the latches
Supply chain	[have] cargo deny
Egress proxy	[new] v1 denies all egress; a domain-allowlist proxy is still open
Signed builds	[new] Waits on the Apple Developer account

3.11 Evaluation

Mechanism	State
Rig	[have] Task spec v2 with mandatory oracle, deterministic graders, k-seed scheduler, immutable store, paired bootstrap and McNemar statistics, report cards, frozen bash-only control scaffold

Mechanism	State
Corpus	[have] 40 public tasks plus 5 canary held-out tasks, all oracle-verified (the corpus grew from 35 to 40 in commit <code>e875a8f</code> ; <code>eval/README.md</code> still says 35 and gets corrected in W0)
External adapters	[have] SWE-bench Verified (gold patch verified), Terminal-Bench 2.x and SWE-rebench via Harbor, τ^2 , LiveCodeBench
Live results	[new] None. The eval hold is in force and no paid runs have happened
Long-horizon, chaos and memory suites	[new] —

3.12 Known open items carried into this roadmap

From `docs/LEAVING-OFF.md`, the PR Deferred sections and the fix reports:

- Memory v3 H2 (history search, applied learned skills) and H3 (dream pass, mounts, sync).
- Live computer-use validation on the owner's Mac.
- A live run on a priced model: the spend gate and review-share cap have only been exercised at \$0.
- Orchestration wave B: task DAG, fan-out, shared scratchpad, writer retry.
- The eval hold.
- Two `fix2-agent` reviewer tests that stay ignored, with replacement tests through the real paths.
- `release.yml`'s `plan` job fails on every PR; `pr-run-mode = "skip"` would stop it.
- Rotate the provider key pasted in the 2026-09-20 session.
- Move the repository out of iCloud-synced Desktop.
- Bash background processes are now killed when the command finishes; only `setsid` survives (behaviour change from `fix2-tools`).
- Subagent worktrees can pile up now that gitignored output counts as change.

4. The Competitive Bar

“Crush every other harness” is only meaningful against a concrete bar. This chapter sets one per capability, using the harness teardowns in `docs/agent-harness-playbook.pdf` (Chapter 2 of the playbook) and the 2026-09-29 pillar refresh. Competitor details are tagged **[src]**: they come from those earlier research notes, not from fresh measurement, and products move quickly. Workstream E8 re-measures the bar on the eval rig before any public claim.

4.1 The best idea from each harness, and our answer

Harness	Best idea worth taking [src]	Overseer today	How we go past it
Claude Code	ToolSearch deferral; auto-memory index cap (200 lines / 25 KB); fork subagents that inherit the cache; hooks; skills	Deferral via <code>tools</code> [have] ; INDEX cap [have] ; skills and hooks [have]	Cache-inheriting subagent forks (O16); hooks v2 with typed events (P10); resident footprint about a tenth of theirs
Codex	App-server JSON-RPC; per-model tuned prompts (~1.5K)	Wire protocol crate exists, thin	ACP adapter plus app-server parity (X3); per-model prompt variants through the policy compiler (P2)
Factory Droid	Missions: a feature graph plus validator droids; light/medium/heavy model slots	Tiers [have] ; no graph	Missions on the task DAG with executable validators per node (O15)
Devin	Fusion lead and sidekick; message-forest revert; cache keepalive; cloud handoff	Single lead; fork and rewind [have]	Persistent sidekick agents (O7, O8); keepalive (K3); local-first with optional cloud executor (R18)
Amp	Handoff instead of compaction; oracle consult	Consult [have] ; deterministic compaction [have]	Handoff as a measured alternative to compaction (C9)
OpenCode	One background server, every UI a client; assets embedded	Gateway daemon [have] ; embedded web assets [have]	The supervisor makes the daemon the owner of run liveness (R4); every frontend is a client of it
omp	Hashline edits (vendor claims –61% output tokens); stream rules (TTSR); role fallback chains; CDP relay into the real browser	Plain edits	Hashline edits behind a measured gate (T1); stream rules (P8); failover ladders (L4)
Hermes	cua-driver computer use; bounded memory files; skill self-improvement; FTS session search; cron plus messaging	Computer use [have] ; learning review [have]	History search (M1), applied skills (M2), dream pass (M3), durable workflows (A)
DeepSeek harness	Everything is a plugin; append-only session log with fork and replay; code mode; peer-runtime subagents over ACP	Log, fork, replay [have] ; code mode [have]	Plugin model with trust tiers (Y1); peer agents over ACP (Y4)
pi	Smallest resident prompt, four core tools	~1.76K tokens resident	Stay within 2K resident while carrying far more capability (Z1, K)

Harness	Best idea worth taking [src]	Overseer today	How we go past it
mini-SWE-agent	A ~100-line bash-only loop scores above 74% on SWE-bench Verified: the capability floor	Frozen control scaffold in the rig [have]	Beat the floor per component, measured and published (E1, E4)
Synara	Patched cua-driver; foreground versus background consent; screen-lock revocation; progress guard; hard denylist; 25-step validated batches	Lean driver backend [fix2]	Consent modes (U2), progress guard (U3), lock revocation (U7); denylist left to owner-configured permissions by decision
OpenHands, Goose, Aider, Gemini CLI	Event-stream architecture (OpenHands); MCP-first extensibility (Goose); architect/editor split and repo map (Aider); large context and free tier (Gemini CLI)	Event log, MCP consult, repo map [have]	Beat them on long-horizon survival and cost per solved task, the two axes none of them targets

4.2 The capability bar, axis by axis

For each axis: the best known level, where we are, and the target that counts as crushing it.

Axis	Best known [src/infer]	Overseer today	Target
Resident startup tokens	pi ~1.1K; Codex ~8.5K; OpenCode ~6.9K; Claude Code 27–33K	~1.76K	≤ 2.0K with every pillar in place
In-session cache hit rate	Reported ops target 90–95%	Telemetry exists; live rate not yet measured on priced traffic	≥ 92% on Anthropic, ≥ 85% on OpenAI-compatible, measured on the long-horizon suite
Unattended run length	Hours, with manual resume in most CLIs [infer]	Until the first transient error	≥ 100 hours with zero human intervention on the long-horizon suite
Survival of provider outages	Simple retry in some harnesses [infer]	None	Backoff plus failover; no run ends on a transient
Crash and reboot survival	Manual resume [infer]	Manual resume	Automatic resume within 60 s of the supervisor restarting
Parallel work	Background agents, fork subagents [src]	4 background readers, isolated writers	DAG with merge queue, persistent agents, depth and fan-out caps
Automation	Cron plus messaging (Hermes), scheduled agents [src]	Single-shot triggers	Durable, versioned workflows with compensation
Memory	Index files plus topic files, skill self-improvement [src]	v3 learning loop	Outcome memory and measured uplift from a warm store
Verification	Tests when present [infer]	Verify mode and gate	Mandatory executable or fresh-context verification for every “done”
Security	Permission prompts, some sandboxes [src]	Rule-of-Two latches plus sandbox	Plus egress proxy, scoped grants, full-auto audit trail, signed builds
Explainability	Logs [infer]	Event log	overseer why on any decision (P11), traces (B1)
Leanness	Rust rewrites around 20 MB RSS best case; Codex ~80 MB binary [src]	~5.5 MB binary	≤ 15 MB binary with every feature, budgets enforced in the build

4.3 What “crush” means as a claim

A claim of superiority is made only when all of these hold, and Chapter 25 holds the scoreboard:

1. **Same model, same tasks, paired statistics.** Pass rate with a 95% confidence interval from the seeded cluster bootstrap the rig already implements, against each competitor run on the identical model and task set.
2. **Cost per solved task**, not cost per task: total spend divided by tasks solved, with the cache breakdown.
3. **Long-horizon survival:** the fraction of long-horizon suite runs that complete without human intervention, under injected faults (E3).
4. **No regression in leanness:** resident tokens, binary size, RSS and startup within budget.
5. **Published method:** the report card format from the rig, with harness commit, model, seeds and environment digest.

WHAT COULD BREAK

Benchmarks lie quietly. Vendor-reported numbers fall sharply in independent reruns (the memory benchmark record in the pillar refresh shows LoCoMo claims of 92–96% dropping to 65–85%). Our own claims are only as good as the rig’s contamination hygiene and the fairness of competitor configuration. Mitigation: E8 runs competitors with their recommended settings, records those settings in the report, and invites reproduction; E9 handles contamination.

WHAT COULD BREAK

The bar moves while we build. Every competitor ships monthly. Mitigation: the bar is re-measured at the close of every wave (E5), and targets in this chapter are revised in the roadmap’s next edition rather than silently.



Part II — The Pillars

5. Reliability and Long-Horizon Execution

“tasks can go on for 100+ hours no issue and uninterrupted if needed” (owner, 2026-10-07)

Reliability is the spine of this roadmap. Orchestration, workflows, policy and memory all assume that the unit they build on, a run, survives. Today it does not survive much: the first rate limit, network drop or crash ends it. This chapter turns “resumable” into “self-healing”.

5.1 Why a hundred hours changes the architecture

A hundred hours is not a short run made longer. It crosses thresholds where failures that are rare per call become certain per run.

Assume a long run makes one provider call per minute on average. Over 100 hours that is about 6,000 calls. If one call in two hundred fails transiently (rate limit, overload, network blip, a provider’s bad minute), the expected number of transient failures in the run is about 30, and the probability of seeing none is about 0.995^{6000} , which is effectively zero **[infer]**. The same arithmetic applies to every other component: a hundred hours on a laptop will include sleep, network changes, probably an OS update, and possibly a reboot. A run that dies on the first of these cannot reach hour 100.

So the requirement is not “fewer failures”. It is that **no transient or infrastructure failure ends a run**: runs end only for terminal conditions that are recorded and owner-controlled (a total budget reached, a fatal invariant violation, the owner abandoning the run). That needs five things the engine does not have: a classification of failures, retry policies per class, a process that brings dead runs back, a record of intent that survives everything, and a record of side effects that makes coming back safe.

5.2 Where we stand

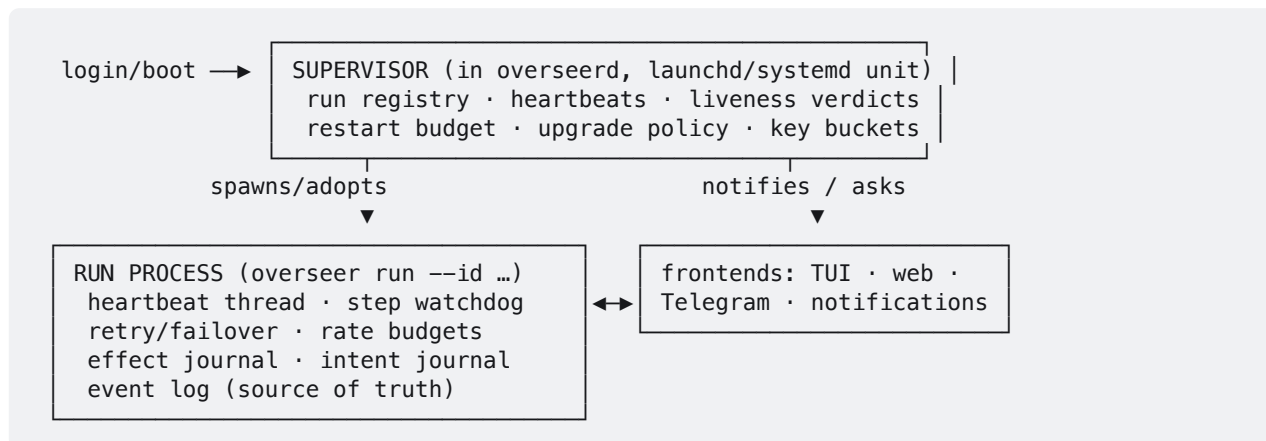
The substrate is right, and that is most of the battle:

- The event log is immutable, hash-chained and torn-tail tolerant, so a run’s history survives a crash **[have]**.
- `Agent::resume` rebuilds a run from its log **[have]**, and compaction is a view, so resuming never depends on a lost in-memory summary.
- Subagent liveness is tracked with per-process nonces and dead tasks are reaped **[have]**.
- Most tool calls have timeouts, and **[fix2]** closes the worst hang (bash grandchildren holding the pipe).

What is missing, from the audit in Chapter 3:

- Every provider error except one malformed-response retry ends the run.
- Nothing restarts a dead run; resume is manual.
- No wall-clock budget exists; cost and step budgets are tuned for minutes.
- The ask dialog waits forever; gateway-spawned runs are killed after 600 s.
- Intent lives in the context window and the compaction summary.
- `bash` side effects are not recorded, so a resumed run cannot know whether its last command ran.
- Logs, spill files and worktrees grow without bound.

5.3 The target in one picture



The run process does the work and keeps its own durable records. The supervisor watches, restarts, enforces global limits and talks to the owner. Frontends are clients that attach to runs; closing a frontend never ends a run.

5.4 Workstreams

R1 — Failure taxonomy

What. A closed classification of every failure the engine can see, used by every retry, failover, pause and end decision.

Design. A `FailureClass` enum in core, assigned at the point of failure, logged on the event that records the failure:

Class	Examples	Default handling
Transient	Connection reset, DNS failure, TLS handshake failure, 502/503/504, stream cut mid-response, a memory store lock reporting BUSY	Retry with backoff (R2)
Throttled	429, provider “rate limit” errors, opencode pool saturation	Honour <code>retry-after</code> ; slow the key’s bucket (R10)
Overloaded	Anthropic 529, “overloaded” messages	Backoff; after N attempts consider failover (L4)
Degraded	Repeated malformed or truncated responses, empty responses	Retry once; then failover or effort change
ContextLimit	Context-window exceeded	Compact (existing behaviour)
Budget	Cost, step, wall-clock or rate budget exhausted	Pause (rate) or end (total), per R10
Auth	401/403, revoked or expired key	Park with a clear owner action; never retry blindly
Policy	Gate refusal, hard deny	Return to the model as a tool result (existing)
Environment	Disk full, workspace missing, TCC revoked, MCP server gone	Pause or park with a named cause (R11, R16)
Fatal	Internal invariant violation, corrupt log line before the tail	End with a recorded reason and a preserved log
Refused	A successful response whose <code>stop_reason</code> or finish reason is a refusal or content filter, or an adapter-detected refusal in the body	Recorded with the raw reason (invariant 7); one retry with the refusal noted; repeated refusals on the same step park the run with the refusal text attached, instead of burning steps until the stuck detector trips

Class	Examples	Default handling
Unknown	Anything unclassified	Treated as Degraded for one retry, then park; every Unknown is a bug to classify

Interfaces. `ProviderError` gains a `class()` method; tool errors and environment errors map through one function. The raw provider `stop_reason` and error body stay preserved (invariant 7).

Edge cases. Providers misuse status codes (a 400 that means overload, a 500 that means bad request). Per-provider mapping tables live in the adapters, with fixtures captured from real errors. A class can be upgraded by repetition: three `Transient` in a row on one endpoint inside a minute becomes `Overloaded`.

GATE

Every `ProviderError` variant and every tool error path maps to a class, proven by an exhaustive match with no wildcard arm. Fixture tests per provider adapter for at least 429, 500, 502, 503, 529, 401, connection reset and mid-stream cut. `RunEnd` carries a reason from the closed set (invariant 10).

R2 — Provider retry, backoff and resumable streams

What. A run never ends on a `Transient`, `Throttled` or `Overloaded` failure while retry budget remains.

Design.

- **Backoff.** Decorrelated-jitter exponential backoff: base 2 s, cap 5 minutes, attempt delay `min(cap, random(base, previous × 3))`. When the response carries `retry-after` or a provider-specific reset header, wait at least that long, capped by a configurable ceiling (default 30 minutes).
- **Budgets on retrying itself.** Per class, a maximum consecutive attempt count (default 12 for `Transient`, unlimited-but-paced for `Throttled`), and a global retry budget per hour per run. Exhausting it moves the run to `waiting{backoff}` with a growing interval rather than ending it. Only `Auth` and `Fatal` end or park immediately.
- **Ledger.** Each attempt is a call, so each gets its ledger row (the existing one-row-per-call rule). Retries are visible in cost and in `overseer stats` as their own purpose.
- **Spend gate.** Each attempt is re-priced against the remaining budget, so a retry storm cannot overspend.
- **Partial streams.** A response cut mid-stream is discarded whole; nothing partial is appended to the event log. The attempt is retried from the same request. The log only ever holds complete model responses, which keeps tool pairing and invariant 6 safe.
- **Idempotency.** Provider calls have no side effects beyond billing, so they are always safe to retry. Tool calls are not; they are covered by R7.

Edge cases.

- A provider returns 200 with a body that is an error. Adapters classify by body as well as status.
- `retry-after` values in the hours. Above the ceiling the run moves to `waiting{throttled}` and notifies the owner instead of sleeping silently.
- Retries during an owner interrupt. The interrupt wins; backoff sleeps are interruptible (the same condition-variable pattern `wait_for` uses).
- Clock jumps during a sleep. Backoff uses monotonic time (R12).

GATE

Fault-injection tests (R15) at 1%, 10% and 50% transient rates over 1,000 simulated calls show zero run ends, correct ledger rows per attempt, and backoff timing within bounds. A `retry-after` of 90 s is honoured to within 1 s. An interrupt during backoff stops the run within 200 ms.

R3 — Close every timeout hole

What. Invariant 14: every wait has a deadline and an escalation. The audit found these holes, each with its fix:

Hole	Fix
No wall-clock run budget	<code>max_wall_hours</code> on <code>AgentConfig</code> , counted in monotonic time while awake (R12); default off for interactive runs, set by the long-run profile (§5.6)
Provider HTTP is one 600 s global timeout	Split into connect (10 s), time-to-first-byte (120 s, longer for high-effort reasoning profiles) and stream idle (90 s between chunks). Hypothesis to verify: the pinned <code>ureq</code> version exposes separate connect and receive timeouts; if not, wrap the stream reader with an idle watchdog
Ask dialog waits forever	Away policy (R8)
Desktop notifier has no timeout	Spawn with a 5 s deadline and kill; move off the daemon tick to a notifier thread
Telegram calls run inside the daemon tick	Move channel polling to a dedicated thread with a bounded queue into the tick
Gateway spawn watchdog kills at 600 s	Replace with supervisor-owned runs (R4) that carry their own wall and rate budgets; the watchdog becomes a heartbeat check
Foreground subagents inherit an unbounded clock	Per-task <code>max_wall_minutes</code> (tier defaults), counted against the parent's wall budget
A single step can run indefinitely (long tool + long provider call)	Step watchdog: a step exceeding <code>step_max_minutes</code> (default 30) raises a hung verdict for the supervisor

GATE

A test per hole proves the deadline fires and the escalation runs. A static check in CI lists every `Command::status()`, `Command::output()`, `recv()` and blocking read in the workspace and fails on any not wrapped in a deadline or explicitly allow-listed with a comment.

R4 — The supervisor

What. A component that owns run liveness: knows every run, detects death and hangs, restarts runs safely, and survives reboots.

Where it lives. In the gateway daemon (`overseerd`), which is already always-on, single-threaded and Unix-only. It is started at login by a `launchd` agent on macOS and a `systemd` user unit on Linux, both installed by a new `overseer daemon install` command. The supervisor adds a module to the tick loop; it does not change the daemon's thin, credential-free character (playbook Alfred architecture). Runs stay separate processes.

The run registry. `~/.overseer/runs/<run-id>.json`, one sidecar per supervised run, modelled on the existing `task.json`:

Field	Meaning
<code>id</code> , <code>session_dir</code> , <code>workspace</code>	Identity and location
<code>binary</code> , <code>binary_version</code> , <code>schema</code>	What started it (R13)
<code>state</code> , <code>state_reason</code> , <code>since</code>	Lifecycle (R14)

Field	Meaning
pid, nonce	The current owning process; the nonce is per process, as in sidecars
restarts, restart_window	Crash-loop accounting
profile	Interactive, long, or workflow-step (§5.6)
budgets	Wall, cost and rate limits, plus spend so far

Heartbeats. The run process runs a heartbeat thread that rewrites `<session>/heartbeat` every 10 s with the pid, nonce, a monotonic counter, the current step number and the time of the last step transition. Two signals, not one: the thread proves the process is alive; the step transition proves the loop is moving.

Liveness verdicts.

Verdict	Condition	Action
alive	Heartbeat fresh, steps moving within <code>step_max_minutes</code>	None
hung	Heartbeat fresh, no step transition beyond the step watchdog, and not in a legitimate wait (backoff, human, budget)	Interrupt; after a grace period, kill the process group; then resume
dead	Heartbeat stale beyond 60 s and the pid is gone, or the nonce does not match the live lock holder	Resume
zombie	Pid alive but heartbeat stale and the live lock not held	Kill; resume

The kernel `flock` behind `live.lock` is released when a process dies, so lock state is a reliable death signal that cannot go stale (it is never stolen, per the existing design).

Restarting. Resume means: `spawn overseer run --resume <session>` with the run's profile, which reacquires `live.lock`, replays the log, runs the side-effect reconciliation (R7), injects a resume briefing (R5) and continues. A restart budget (default 5 restarts per rolling hour, with exponential spacing) prevents crash loops; exceeding it parks the run with the last crash's evidence attached.

Reboots. On daemon start, the supervisor scans the registry. Every run in `running` or `waiting` whose process is gone is resumed (subject to R13 version policy). Runs in `paused` or `parked` stay put.

Detached runs and frontends. Today the TUI owns its agent on a worker thread, so closing the TUI ends the run. Under the supervisor, a run can be started detached (`overseer run --detach`, and workflows always detach). The TUI and web UI attach to a detached run over the daemon's control socket, rendering its events and sending input and interrupts. Interactive runs stay in-process by default, and can be detached on demand (`/detach`), which hands the run to the supervisor at a step boundary. Interactive runs still register in the run registry with the `interactive` profile; if their frontend dies, the supervisor does not resume them silently but notifies the owner and offers a one-step resume, because an interactive run may have been mid-conversation.

Edge cases.

- **Two daemons.** A pidfile plus `flock` on `~/overseer/daemon.lock`; the second exits with a clear message.
- **The daemon itself crashes.** `launchd/systemd` restart it; runs keep running (they are separate processes); the restarted supervisor re-reads the registry and reattaches by pid and nonce.
- **The workspace is deleted or moved.** The run parks with `Environment` class and a named cause.
- **The owner resumes the run manually at the same time.** The live lock decides; the loser reports who holds it.

- **The run is open in a TUI when it dies.** The TUI shows the death and the supervisor's restart, then reattaches.
- **Laptop on battery at 5%.** Supervisor policy can pause long runs below a battery threshold (owner setting, R12).

GATE

Chaos tests (R15, E3): `kill -9` of the run process at 200 random points across a scripted long run; the run completes with an event log identical in content (modulo restart events) to an uninterrupted control. Daemon `kill -9` with five runs in flight: all five survive. Simulated reboot (daemon and runs killed, daemon restarted): all running runs resume within 60 s. Crash loop: a run that dies on startup parks after the restart budget with evidence attached.

R5 — Durable intent: the run journal

What. Invariant 12. The run's goal, plan, progress and next step live in a journal outside the context window, so a resume after any amount of compaction knows exactly where it stands.

Why compaction is not enough. The compaction summary is mechanical and lossy by design, and it must never be re-summarized (invariant 8). After the third or tenth compaction of a long run, the window holds a summary of the last stretch and a two-turn tail. The goal stated on hour 1 and the decisions taken on hour 20 are gone from view unless something durable carries them.

Design. The journal is part of the event log, so state remains a view (invariant 1). New event kinds:

Event	Payload	Who writes it
JournalGoal	The goal statement and acceptance criteria, as the owner gave them	Engine, at run start; only the owner can amend
JournalPlan	The structured plan: items with id, title, state (todo, doing, done, blocked, dropped), dependencies, evidence links	Agent through the <code>plan</code> tool (promoted from deferred to resident in long runs)
JournalNote	A decision, finding or assumption, with its kind	Agent, through <code>plan op=note</code>
JournalMilestone	A plan item reached <code>done</code> , with evidence (verify result, test counts, commit)	Agent; the engine verifies evidence references exist
JournalNext	What the agent intends to do next, one paragraph	Agent, at each milestone and before any pause

Materialized views (`<session>/journal/goal.md`, `plan.json`, `notes.md`) are written for humans and frontends; they are caches, rebuilt from events.

Resume briefing. After any resume, compaction or failover, the engine injects a deterministic briefing at the recency edge (below the cache boundary, provenance-wrapped as `journal`), built only from journal events:

```
Goal: <goal> · Acceptance: <criteria>
Plan: 14 items — 9 done, 1 doing (#10 migrate auth module), 2 todo, 1 blocked (#12: needs owner)
Last milestone (2h ago): #9 billing module migrated; 412/412 tests pass (verify e8812)
Decisions: 6 (latest: keep the legacy adapter until #13)
Next: finish #10, then run the integration suite
In-doubt effects: none
```

Agent contract. The long-run prompt segment tells the agent to keep the plan current, and the engine enforces freshness: if no journal event lands in N steps (default 25) while the run is busy, a nudge asks for an update (this is a deterministic check like the stuck detector, not a model call).

Edge cases. An agent writes a plan that is wildly wrong; the goal audit (O9) catches drift. A plan item marked done without evidence is flagged in the briefing as `unverified`. Journal size is bounded: notes beyond 200 roll into an archived view, but goal, plan and the last 20 notes are always in the briefing.

GATE

A scripted 40-compaction run resumes after each compaction and each injected crash with a briefing that names the correct goal, plan state and next step (asserted against the journal events). The briefing never exceeds 1,500 tokens. Journal events replay on old binaries as skipped unknown kinds without error (invariant 21).

R6 — Crash-consistency audit of every durable write

What. Prove that every durable store survives a crash at any instruction.

Inventory. Event log, ledger, run registry, task sidecars, heartbeat, journal views, checkpoints and manifests, spill files, memory stores and their indexes, pending queue, skill ledger, rules file, inbox and outbox, worktrees, the web token.

Policy per store. For each: the write pattern (append, temp-file plus rename, in-place), `fsync` placement (file and, for renames, the directory), what a torn write looks like, and how readers recover. The rules:

- Appends that matter for correctness (event log, ledger, effect intents) are `fsynced` at turn boundaries and before every classified side effect (R7).
- Replacements use temp-file, `fsync`, rename, directory `fsync`.
- Readers tolerate exactly the torn states the writer can produce, and nothing else.

GATE

A table in the decision record covering every store, and a crash-injection test per store that kills the writer at each write step and asserts recovery. A power-loss simulation on a disk image (Linux CI-equivalent run locally) for the event log and ledger.

R7 — The side-effect journal and idempotency

What. Invariant 13. Coming back after a crash is only safe if the run knows which side effects already happened.

Design. For every tool call whose classification is above Read:

1. Before dispatch, append and `fsync` an `EffectIntent { call_id, tool, class, target, idempotency_key, fingerprint }`.
2. After completion, append an `EffectOutcome { call_id, status, evidence }`.
3. On resume, reconciliation finds intents with no outcome: the **in-doubt set**.

Resolution per tool family:

Family	How in-doubt is resolved
<code>write, edit</code>	Compare the file's current hash with the intended result (the pre-image is in the checkpoint, the intended content is in the call). Matches → outcome <code>applied</code> ; matches the pre-image → <code>not_applied</code> ; neither → <code>conflict</code> , surfaced
<code>bash</code>	Cannot be known in general. The model receives an explicit in-doubt notice ("this command may or may not have run; verify before repeating") and the command's text. Repeating an in-doubt command requires a verification step first, enforced as a gate rule. Model-chosen verification is best effort, not proof; commands where best effort is not good enough are declared never auto-repeatable (below) and take the unverifiable path
git operations through <code>bash</code>	Recognized patterns (commit, push, merge, tag) are resolved by inspecting refs and reflog
memory writes	Idempotent by content hash; replays are harmless and deduplicated
<code>task</code> spawns	Sidecar state resolves them (existing reaping)
External comms (gateway sends, MCP calls on ask lanes, navigate)	Never auto-replayed. Outbox items carry stable ids so a resend deduplicates where the channel supports it; otherwise the run parks the decision for the owner
Money and identity	Never auto-replayed; always parked

In-doubt policy is declared at intent time, not decided at resume. Each effect family (and, through the policy compiler, each command pattern) carries one of four policies, recorded in the `EffectIntent` itself so resume never has to improvise:

Policy	Meaning	Examples
<code>verify</code>	A read-back exists and the engine resolves it	<code>write, edit</code> , git ref operations, memory writes
<code>replay_safe</code>	Re-executing is harmless or deduplicated by an idempotency key the far side honours	<code>cargo test</code> , outbox sends on channels with id deduplication
<code>model_verify</code>	No engine read-back; the model must verify before any repeat (best effort)	Ordinary <code>bash</code> commands
<code>unverifiable</code>	No read-back and no safe replay	POSTs without idempotency keys, messages on channels without receipts, never-auto-repeat commands, ambiguous GUI actions, money and identity

The unverifiable path does not wedge the run. An `unverifiable` in-doubt effect blocks only the plan item (or graph node) that depends on it: that item becomes `blocked{in_doubt}`, an approval-style request goes to the owner through the away policy (R8) asking "did this happen?", and the run continues with independent work. If nothing independent remains, the run parks with the question as its reason. The journal's role for these effects is detection and gating, never recovery; the document says so plainly because no engine can reconcile an effect that leaves no trace.

Idempotency keys. Generated as a hash of the session id and the call id only, never the attempt number, so every retry and every resumed attempt of the same effect carries the same key; passed to every integration that accepts one (outbox, webhooks, future connectors), so a retried send is recognized by the far side.

Edge cases. A command that is idempotent by nature (`cargo test`) is still in doubt, but its verification is free: run it again. The policy compiler (P2) lets the owner, and the project's `AGENTS.md`, mark command patterns as safe to replay or as **never auto-repeatable** (database migrations, deploy scripts); for the latter an in-doubt instance always parks for the owner. The in-doubt notice names the command's risk class so the model chooses a verification proportionate to it (premortem one, Chapter 22). A crash between intent and dispatch produces a false in-doubt; harmless, because resolution checks reality.

GATE

For each family, a crash test between intent and dispatch, during execution, and between completion and outcome, asserting correct resolution. Zero double-sends of outbox items across 1,000 injected crashes.

R8 — Away policy: asking when nobody is there

What. Invariant 14 applied to humans. A long run must keep moving when an approval is needed and the owner is away.

Design.

- **Non-blocking asks.** An Ask verdict no longer blocks the agent thread. The gate records an `ApprovalRequested { id, call, class, preview, deadline }`, returns a tool result telling the model the call is pending with an id, and the agent may continue with independent work (other plan items, other DAG nodes once O2 lands). When the decision arrives, an `ApprovalResolved` notice is delivered at the loop top like `SubagentDone`, and the agent can retry the call, which then passes the gate on the recorded grant.
- **Interactive runs keep the dialog.** When a frontend is attached and focused, the existing dialog appears as today. Non-blocking mode applies to detached runs and to interactive runs whose frontend has been unfocused beyond a threshold.
- **Deadlines and escalation.** Per run (from its profile or the policy compiler): an escalation chain (attached frontend → desktop notification → Telegram or another channel) with a delay per step, and an on-timeout action:

On timeout	Behaviour
<code>deny_once</code> (default)	The call is denied with a reason; the run continues and records the gap in the journal
<code>park</code>	The run parks until the owner answers
<code>wait</code>	Wait indefinitely (explicit owner choice only)

- **Late answers.** An approval that arrives after timeout is recorded and applies to the next identical request, never retroactively.
- **Full automation.** Under the owner's full-auto consent mode (P5), Asks do not arise for the domains it covers, so away policy is moot there; everything is still journaled (S4).

GATE

A detached run hits an Ask with no frontend attached: the notification chain fires at the configured delays (tested with a fake channel), the run continues independent work meanwhile, `deny_once` applies at the deadline, and a later approval enables the next identical call. With a focused TUI attached, behaviour is unchanged from today.

R9 — Progress ledger and long-window stall detection

What. The stuck detector catches local loops over a handful of steps. A long run can fail differently: busy for hours and getting nowhere.

Design. A progress ledger derived from events: journal milestones, plan items completed, verify outcomes, test pass counts when a verify command reports them, files changed, commits made. Windowed signals

over hours: no milestone within the expected window (sized as below), monotone growth of failed verify attempts, oscillating diffs (the same files edited back and forth), rising cost per milestone.

Window sizing. Windows scale with the current plan item's own history (time already spent on it against the median time per completed item), not with the plan's total size, so a long plan cannot hide a stuck item for most of a day (premortem one, Chapter 22).

Actions, escalating. A nudge with the evidence → a fresh-context goal audit (O9) that reads the journal and recent diffs and returns a structured verdict → park with a report for the owner. Each step is journaled.

GATE

Scripted scenarios for each stall signal trigger the right escalation step; a healthy long run with steady milestones never triggers.

R10 — Pacing: rate budgets and shared provider limits

What. Invariant 15. A hundred-hour run cannot have a \$5 total cap, and an uncapped one is reckless. Budgets need a rate form.

Design.

- **Rate budgets.** Per run: dollars per hour, dollars per day, and tokens per hour, alongside totals. Breaching a rate moves the run to `waiting{rate_budget}` until the window has room; breaching a total ends the run (as today) unless the profile says park.
- **Global budgets.** The supervisor enforces daily and monthly limits across all runs from the global ledger rollup (B2).
- **Alerts.** Notifications at 50%, 80% and 100% of any total or daily budget.
- **Shared provider limits.** Runs that share a key share its rate limit. The supervisor keeps a token bucket per key and model, adapted by additive-increase, multiplicative-decrease on 429s. Run processes do not ask permission per call (the daemon's tick loop is single-threaded, and a round trip per call would add latency to every request); instead each run holds a short lease of call capacity, renewed in the background, and reports 429s so the bucket shrinks for everyone. When the daemon is unreachable, a run falls back to local backoff alone. This stops five parallel runs from collectively hammering a limit and all failing.
- **Subscription quota windows.** Several providers bill through plans with rolling usage windows (the opencode Go subscription the eval rig uses is one; Claude and ChatGPT plans have the same shape). Overseer Life's usage readers already know each account's usage and reset time (Phase 0). Pacing consumes them: slow down before a window is exhausted, wait for the reset rather than failing on it, or route to another configured account (L4) when the owner allows.
- **Paused is not finished.** A run paused by a rate budget sends its own notification, distinct from completion, stating when it will resume on its own (premortem one, Chapter 22).
- **Forecasting.** The run's cost rate and remaining plan items project an estimated total, shown in the cockpit (X1) and used to warn early (B4).

GATE

A simulated run under a \$2/hour rate pauses and resumes at window boundaries without losing work. Five simulated runs sharing a fake key with a hard limit converge to the limit without any run ending on a 429.

R11 — Resource guards: disk, logs, spills, worktrees

What. Invariant 16. A week of activity must not fill the disk or slow replay to a crawl.

Design.

- **Disk preflight and thresholds.** Before each step, check free space on the session and workspace volumes. Below a warning threshold, notify; below a hard threshold, pause the run with `Environment` class.
- **Replay snapshots.** Replaying a 100-hour log on every resume gets slow. At compaction boundaries the engine writes a snapshot event holding the derived state needed for replay (the compaction view, ledger totals, latches, journal state, read-dedup table), plus the hash of the event it covers. Resume starts from the latest valid snapshot and replays forward. The snapshot is a cache: if it fails verification, replay from the start (invariant 1 and the “everything else is a cache” principle).
- **Log segments.** `events.jsonl` rotates into numbered segments past a size threshold, with the hash chain continuing across segments and a manifest listing them. Old segments compress. Nothing is ever deleted by the engine; archival to cold storage is an owner command.
- **Spill collection.** Spill files no longer referenced by the current view, the journal or an artifact are compressed after a day and moved to an archive directory after a week.
- **Worktree collection.** Merged or discarded writer worktrees are removed (the branch stays until merged or explicitly pruned); the `[fix2]` change that keeps worktrees with gitignored output gets a size check so `target/` directories do not accumulate. `overseer gc` reports and reclaims.
- **Memory stores.** Already capped by INDEX limits and the dream pass (M3).

GATE

A synthetic 100-hour log (generated at realistic event rates) resumes in under 2 s from its latest snapshot on the owner’s M1. Disk-full injection pauses, and freeing space resumes. `overseer gc` on a session with 50 worktrees reclaims all merged ones.

R12 — Time, sleep, power and networks

What. Laptops sleep, clocks jump, networks change. None of it should look like a failure.

Design.

- **Monotonic time for every timeout and budget,** wall time only for schedules and display. Rust’s `Instant` on macOS does not advance during sleep **[infer: platform behaviour to confirm per OS in the decision record]**, which is the behaviour wanted for step watchdogs; the wall-clock budget (R3) is defined as awake time, and the decision record states this explicitly.
- **Sleep detection.** A wall-time jump that monotonic time did not see means the machine slept. On wake: refresh HTTP agents and MCP sessions, re-verify the driver for computer-use runs, and record a `Resumed{after_sleep}` event.
- **Keeping awake.** Optional and owner-controlled: while a long run is active and the machine is on mains power, hold a power assertion (macOS `caffeinate -i -w <pid>` as a supervised child, so it dies with the run). Never by default on battery.
- **Battery policy.** Pause long runs below a configurable charge level when unplugged.
- **Network changes.** DNS and connection failures after a network change are `Transient` and retried (R2); the HTTP agent is rebuilt after a detected change to drop dead pooled connections.

GATE

Simulated sleep (clock-offset injection) does not trigger hung verdicts or budget expiry; real lid-close and reopen during a long run on the owner's Mac resumes without a restart or an error event.

R13 — Upgrade-safe runs and schema evolution

What. Invariant 21, extended to live runs. The owner will upgrade Overseer while runs are in flight.

Design.

- **Versions recorded.** `SessionStart` records the binary version and an event schema version; the run registry records the binary path.
- **Compatibility rule.** A newer binary can resume a run if it can read every event kind in the log (old kinds never removed, new fields always defaulted). An older binary resumes a newer log only if every unknown kind is marked skippable; otherwise it refuses with a clear message.
- **Upgrade policy in the supervisor.** When a new binary is installed: runs at a safe boundary (between steps) are handed to the new binary on their next restart; the owner can choose `drain` (finish current runs on the old binary, start new runs on the new one) by keeping the previous binary installed side by side, which the installer supports (Z3).
- **Golden log corpus.** Every release adds representative logs to a test corpus; every later build must replay them all.

GATE

The golden corpus replays on every build in the gate. An in-flight run survives a binary swap at a step boundary in a local test.

R14 — Run lifecycle and control surface

What. One state machine for every run, and commands to see and steer them.

States. `created` → `running` ⇌ `waiting{kind}` (backoff, throttled, human, rate_budget, resource, dependency) → `paused` (owner) / `parked` (needs owner, with a reason) / `suspended` (process gone, resumable by the supervisor) → `completed` / `failed{reason}` / `abandoned` (owner). Every transition is a `RunState` event with a reason.

Commands.

Command	Effect
<code>overseer runs</code>	List runs with state, age, spend, rate, progress and next step
<code>overseer attach <id></code>	Open the TUI on a detached run
<code>overseer pause/resume <id></code>	Pause at the next step boundary; resume
<code>overseer park <id> --reason</code>	Park deliberately
<code>overseer abandon <id></code>	End with reason <code>abandoned</code> ; worktrees kept for inspection
<code>overseer report <id></code>	Generate the run report (goal, plan, milestones, decisions, effects, spend, verification)
<code>overseer logs <id> [--follow]</code>	Human-readable event stream

The same operations exist in the web UI and over Telegram for approvals and status (X4).

GATE

Property test over random sequences of commands and injected failures: the state machine never reaches an undefined state, and every terminal state has a reason.

R15 — Fault injection built into the engine

What. Reliability is proven by breaking things on purpose, repeatedly and reproducibly.

Design. A fault layer compiled into test and debug builds only (a cargo feature, off in release), configured by a seeded spec: provider errors by class and rate, mid-stream cuts, tool timeouts, disk-full, `kill -9` at event N or at random with a seed, clock jumps, slow I/O, MCP server death, notifier hangs. The eval rig's chaos suite (E3) and every reliability gate in this chapter use it.

GATE

Every fault kind is exercised by at least one test, and the fault layer is provably absent from release builds (symbol check in the build gate).

R16 — Environment drift

What. Over days, the world under the run changes.

Drift	Detection	Response
The owner edits files or moves HEAD in the run's workspace	Workspace fingerprint (HEAD, index, dirty set excluding the agent's own recorded effects) checked each step	Pause and ask, or adapt if the profile allows; never overwrite owner changes
Dependencies change (lockfile edits, toolchain updates)	Fingerprint of lockfiles and toolchain versions	Note in the journal; rerun the verify baseline
Provider key revoked or expired	<code>Auth</code> class	Park with the exact owner action
macOS revokes Accessibility or Screen Recording	Driver refusal codes	Park computer-use steps; continue the rest
An MCP server disappears or changes its tool list	Spawn failure or schema change	Mark tools unavailable with a hint; re-probe periodically (T7)
The repository is rebased upstream under a writer worktree	Merge-base check in the merge queue	Rebase-and-verify (O6) or park
Two top-level runs target the same working tree	A workspace lease (a lock in the repository's git directory, held by the writing run, released on death like <code>live.lock</code>)	The second run is offered a worktree of its own or waits; two runs never write the same tree (invariant 20)

R17 — Process hygiene for very long lives

What. A process that runs for a hundred hours accumulates leaks that short tests never see.

Design.

- **Soak instrumentation.** RSS, thread count, open file descriptors and child processes sampled into the heartbeat; trends checked by the supervisor.
- **Rejuvenation.** Because a run's state is a view over its log, restarting the process is cheap and safe. The supervisor restarts a long run's process at a safe boundary every N hours (default 12) or when soak metrics cross a threshold. This turns slow leaks from a correctness risk into a non-event.

- **Child reaping.** Every spawned child is in a known process group and is reaped; the soak test asserts zero zombies.

GATE

A 24-hour soak on the owner's Mac (local, no cloud) with a scripted workload shows flat RSS after rejuvenation, zero zombies, and stable descriptor counts. A 100-hour soak runs before the W3 wave closes.

R18 — Optional remote executor

What. Sometimes the laptop has to close for a long time. A run should be able to continue elsewhere.

Design. Export a run bundle (session directory, journal, worktree state as a git bundle, run registry entry), start the same binary on a remote host, stream events back to the local log mirror, and re-import on completion. Credentials are never exported; the remote host uses its own. This is local-first by construction: the local log stays the source of truth once re-imported. One cloud session at a time, and only with the owner's go-ahead (standing rule).

NOTE

R18 is deliberately late. Everything before it makes local runs survive; R18 only matters once they do.

5.5 Configuration surface

All reliability behaviour is configurable through the policy compiler (P2), with defaults chosen so interactive use is unchanged.

Key	Interactive default	Long-run default
<code>retry.transient.max_consecutive</code>	6	12
<code>retry.retry_after_ceiling</code>	5 min	30 min
<code>budget.max_wall_hours</code>	off	120
<code>budget.cost_per_hour</code>	off	owner-set (required)
<code>budget.cost_per_day</code>	off	owner-set (required)
<code>budget.on_total</code>	end	park
<code>step.max_minutes</code>	30	30
<code>away.ask_deadline</code>	n/a (dialog)	30 min
<code>away.on_timeout</code>	n/a	<code>deny_once</code>
<code>away.chain</code>	—	frontend → desktop → Telegram
<code>journal.freshness_steps</code>	off	25
<code>supervisor.restart_budget</code>	—	5 per hour
<code>supervisor.rejuvenate_hours</code>	—	12
<code>power.keep_awake_on_ac</code>	false	owner choice
<code>power.pause_below_battery</code>	—	20%

5.6 The long-run profile

`overseer run --profile long "..."` (and every workflow step) applies the long-run defaults above, detaches under the supervisor, promotes `plan` to a resident tool, turns on the journal contract and non-

blocking asks, and requires the owner to set rate budgets explicitly the first time. That last point is deliberate: a hundred-hour run should never start with an implicit spending policy.

5.7 What could break

WHAT COULD BREAK

Auto-resume repeats a side effect. The most damaging failure in this chapter. Mitigated by R7's intent-before-act records and per-family resolution; external, money and identity effects are never auto-replayed. Residual risk: a `bash` command with external effects (for example `curl -X POST`) that the classifier treats as internal. Mitigation: the sandbox denies network by default, so such a command fails unless the owner has opened egress; once the egress proxy lands (S1), egress-capable commands are classified ExternalComms.

WHAT COULD BREAK

Retry storms make things worse. Many runs retrying in lockstep against an overloaded provider. Mitigated by decorrelated jitter, per-key buckets with multiplicative decrease, and global retry budgets.

WHAT COULD BREAK

The supervisor becomes a single point of failure. If it wedges, nothing restarts. Mitigated by `launchd`/`systemd` restarting it, by runs being independent processes that keep working without it, and by a supervisor self-heartbeat checked by the run processes, which notify the owner if the supervisor has been silent too long.

WHAT COULD BREAK

Snapshots disagree with the log. A bug in snapshot derivation would make resumed runs subtly wrong. Mitigated by the snapshot storing the hash of the covered event, by periodic full-replay verification in tests, and by falling back to full replay on any mismatch.

WHAT COULD BREAK

The journal becomes theatre. The agent keeps a beautiful plan while doing something else. Mitigated by evidence-checked milestones, the progress ledger (R9) comparing claims against diffs and verify results, and fresh-context goal audits (O9).

WHAT COULD BREAK

Non-blocking asks confuse the model. A pending approval may lead the model to assume success. Mitigated by an explicit pending result wording tested on the rig, and by the gate refusing any dependent action that would assume the pending call ran.

WHAT COULD BREAK

Keeping the laptop awake has costs. Heat, battery, and a machine that never sleeps. Mitigated by making it opt-in, mains-only, and visible in the cockpit.

5.8 Metrics

Metric	Target
Runs ended by a transient-class failure	0
Long-horizon suite unattended completion (E2)	$\geq 95\%$ of tasks pre-declared solvable (E2)
Mean time to automatic resume after a crash	≤ 60 s
Resume-from-snapshot time for a 100-hour log	≤ 2 s
Double-executed external effects across chaos runs	0
Owner interventions per 100 run-hours (excluding genuine approvals)	≤ 1

6. Orchestration and Subagents

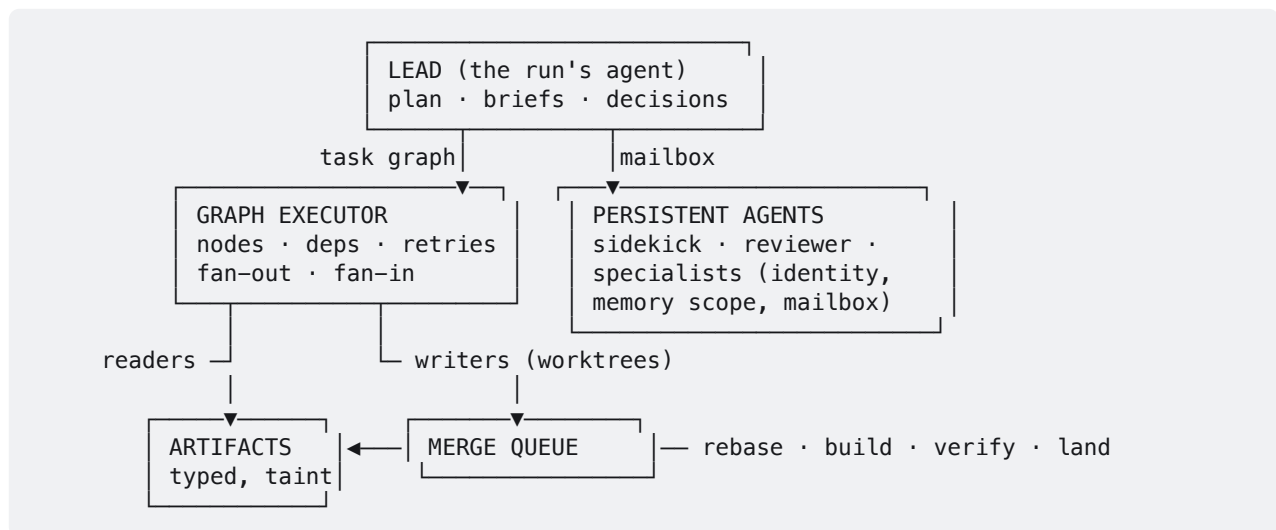
6.1 The bar

Multi-agent work pays when the work is parallel or exceeds one context, and it costs a lot of tokens: Anthropic’s research system beat a single agent by about 90% on its internal evaluation while using roughly 15 times the tokens of a chat [src]. The failure science is just as clear. MAST’s study of more than 1,600 traces attributes about 44% of multi-agent failures to specification, 32% to inter-agent misalignment and 24% to verification [src]. So the best orchestration layer is not the one with the most agents. It is the one that writes good briefs, shares exactly the right context, verifies independently, serializes writes, and spends tokens only where parallelism or context size pays for them.

6.2 Where we stand

The primitives are strong (Chapter 3.6): four modes, three tiers, worktree isolation, composing budgets, a verifier with tamper evidence, escalation, cancellation through a control tree, background digests, sidecar-based liveness. What is missing is structure above the single call: dependencies, shared artifacts, persistent agents, a merge path for parallel writers, limits on depth, and any way to coordinate more than “spawn and wait for a digest”.

6.3 Target architecture



6.4 Workstreams

O1 — Depth and fan-out limits

What. Bound the shape of the agent tree, not just its cost.

Design. `max_depth` (default 2: lead → subagent → one level of grandchildren for writers only) and `max_total_agents` per run (default 16 live, 64 lifetime per hour), enforced in `task spawn`. Readers, verifiers and consults never get `task`. The supervisor (R4) holds a global cap across runs so a workflow fan-out cannot exhaust the machine.

GATE

Spawning beyond depth or total limits returns a policy refusal naming the limit; a property test over random spawn trees never exceeds either.

O2 — The task graph

What. Dependencies between subagent tasks, so the lead can express “B after A” and “C after all of A1... A8” in one plan instead of babysitting each step.

Design. `task` gains a graph form:

```
{ "op": "graph",
  "nodes": [
    { "id": "inv", "mode": "read", "prompt": "Inventory every use of the old API", "outputs": ["inventory"] },
    { "id": "mig-a", "mode": "write", "after": ["inv"], "inputs": ["inventory"], "prompt": "...module A...", "verify": true },
    { "id": "mig-b", "mode": "write", "after": ["inv"], "inputs": ["inventory"], "prompt": "...module B...", "verify": true },
    { "id": "integ", "mode": "verify", "after": ["mig-a", "mig-b"], "prompt": "Full suite on the merged tree" }
  ],
  "on_failure": "continue_independent" }
```

- **Semantics.** A node starts when all its `after` nodes have succeeded and its inputs exist. `on_failure` is `fail_fast` (cancel the rest), `continue_independent` (default: nodes not downstream of the failure continue), or `ask_lead` (deliver a notice and wait for the lead’s decision).
- **Per-node retries.** Readers escalate as today; writers retry once with the failure’s evidence in the brief only if the node says `retry: 1`.
- **Execution.** The graph executor lives in the parent run process and writes node states as events (`GraphNode{id, state, task_id}`) through the run’s single log writer, so the event log stays totally ordered with one writer (no second thread appends to it directly). It is resumable: after a crash, completed nodes stay completed (their sidecars and artifacts prove it) and running nodes go through normal subagent reaping.
- **Budget.** The graph gets one cap carved from the parent; nodes carve from the graph. Unspent reservations return as nodes finish.
- **Validation.** Cycles, unknown ids, missing inputs and over-limit fan-out are rejected before any node starts (the same “validate the whole batch first” rule computer use now follows).

GATE

Graph tests: diamond, fan-out of 8, fan-in, failure under each policy, crash mid-graph with resume, cycle rejection. The lead’s context receives one notice per completed node plus one graph summary, never per-step chatter.

O3 — Artifacts

What. A typed, durable way for agents to hand each other results larger and more structured than an 8,000-character digest.

Design. `<session>/artifacts/<name>@<version>` with a manifest: name, version, media type (markdown, JSON, patch, file list, table), producer task, content hash, size, taint (invariant 19), and a short description. Agents write artifacts through `task`’s output contract (`outputs: [...]`) or a small `artifact` op; consumers receive them as inputs, either inline when small (under 4,000 characters) or as a reference with a summary and a `read` path when large.

Taint. An artifact produced by a tainted agent is tainted; consuming it arms the consumer’s untrusted latch, exactly like reading a web page.

Caps. Per artifact 1 MB, per run 256 MB, collected with the run (R11).

GATE

An artifact round-trips between a reader and a writer; a tainted producer taints its consumer; size caps refuse cleanly; artifacts survive a crash and resume.

O4 — The blackboard

What. A cheap shared scratchpad for a run’s agents: discovered facts, open questions, claimed work, so parallel readers stop rediscovering the same things.

Design. An append-only, per-run list of short entries (≤ 400 characters) with author, kind (fact, question, claim, warning) and provenance, exposed as `task op=board` read and post. Entries are visible to agents started after them, injected as a compact section in their brief (capped at 1,500 characters, newest and highest-ranked first). Claims let a node announce “I am handling X” to avoid duplicate work.

WHAT COULD BREAK

A blackboard is a prompt-injection relay: a tainted agent can post text that steers clean ones. Mitigation: entries carry taint; tainted entries are shown provenance-wrapped and arm the reader’s untrusted latch; claims from tainted agents are advisory only.

O5 — Detach and attach

What. A foreground `task` blocks the parent for its whole run. The lead should be able to send a running foreground task to the background when it turns out to be long, and to wait on a background task when it becomes urgent.

Design. `task action=detach id=...` from a steer or the frontend converts a foreground task to background at its next step boundary; `task action=wait id=... timeout=...` blocks on a background task with a deadline. Both are events.

O6 — The merge queue

What. Invariant 20. Parallel writers each produce a branch in their worktree; integration must be serialized and verified.

Design.

1. A writer finishing successfully enqueues its branch.
2. The queue takes one entry at a time: rebase onto the current integration base (the parent’s branch), run the project verify command (and the writer’s own checks), and fast-forward the base only if both pass.
3. On a rebase conflict, a resolution task is spawned (write mode, with both sides as context, verify required), or the entry parks for the lead under `ask_lead`.
4. On verify failure after a clean rebase, the entry goes back to its writer (if `retry` allows) with the failure, or parks.
5. Every step is an event; the queue is resumable after a crash.

Edge cases. The base moves because the owner commits (R16 detects it); the queue rebases subsequent entries onto the new base. Two writers touch the same file without conflict but break each other semantically; the verify step after each landing catches it, which is why every landing verifies.

GATE

Eight parallel writers on overlapping files produce a linear history where every intermediate commit passes verify; an induced conflict produces a resolution task; a crash mid-queue resumes without losing or double-applying any entry.

O7 — Persistent agents

What. Agents that outlive one task: a sidekick that keeps its context between handoffs, a standing reviewer, and the specialist agents Overseer Life needs (accountant, security officer, social manager).

Design.

- **Identity.** `~/ .overseer/agents/<name>/agent.toml`: role prompt, tier, tool set, permission profile, memory scope, budgets, and schedule if any.
- **State.** Each persistent agent owns a session log like any run, resumed on each handoff; compaction and the journal keep it bounded.
- **Memory scope.** A persistent agent can have its own memory store (`agent:<name>`), mounted alongside user and project stores with its own write bars.
- **Mailbox.** Messages from the lead or workflows arrive as `MailboxMessage` events, processed at the loop top. Replies go back as artifacts or digests.
- **Lifecycle.** Persistent agents run under the supervisor (R4) and inherit every reliability property.

DECISION

Persistent agents never write to shared state directly. A persistent writer still works in a worktree and lands through the merge queue (O6). This keeps the refusal on parallel writers intact.

O8 — Teams and roles: the lead and sidekick protocol

What. Make the lead and sidekick pattern (briefs and reports, never transcripts) a first-class protocol, because it is the cheapest form of multi-agent work that consistently pays **[src: Fusion, vendor claim of 35–60% lower cost]**.

Design. A `brief` schema (goal, context, constraints, done criteria, verification commands, runtime state notes) and a `report` schema (what was done, evidence paths, deviations, open questions). The lead's prompt contract and the sidekick's role prompt are tested on the rig. Roles are configuration: `lead`, `sidekick`, `reviewer`, `researcher`, `specialist:<name>`.

O9 — Goal audits

What. A periodic fresh-context check that the run is still doing what the owner asked.

Design. Triggered by the progress ledger (R9), by plan milestones, or every N hours on long runs. A verify-mode agent with read access gets the goal, the journal, the last diffs and recent verify outputs, and returns `{on_track | drifting | blocked, evidence[], recommendation}`. "Drifting" with evidence produces a notice to the lead and an entry in the journal; repeated drifting parks the run.

O10 — Best-of-N with executable selection

What. When a deterministic check exists, sampling several attempts and selecting by the check beats smarter control flow (Agentless and CodeMonkeys evidence) **[src]**.

Design. `task op=best_of n=3 mode=write verify=...`: N writers in parallel worktrees, each verified; the first that passes (or the best by a scored check) enters the merge queue; the others are discarded. Budget is N times one writer, so the policy compiler can restrict it to tasks marked as high value.

O11 — Cross-provider verification

What. LLM judges prefer their own outputs [src]. A verifier on a different model family is a stronger independent check.

Design. When credentials for a second family are present, verify mode prefers a model from another family at the same tier. Deferred today on multi-provider credentials in one session; unblocked by the model layer's multi-key support (L4).

O12 — Handoff as an alternative to compaction

See C9. For orchestration, handoff means a run can end its own context deliberately and continue as a fresh agent seeded with the journal and a brief, which is often cleaner than a tenth compaction.

O13 — Brief quality

What. Specification failures are the largest MAST class. Most bad subagent results start with a bad brief.

Design. A deterministic brief linter at spawn time: refuses briefs under a minimum length for write mode; warns when no done criteria or verification are given; warns when the brief references “the above” or “as discussed” (context the subagent does not have). The lead sees warnings as tool results. The linter's thresholds are tuned on the rig.

O14 — Seeing the tree

What. The owner must be able to see what every agent is doing.

Design. A graph view in the TUI and web UI (X1): nodes with state, tier, spend, step, last action; drill into any node's transcript; artifacts listed with producers. The data is already in events and sidecars.

O15 — Missions

What. Factory's missions idea: a feature graph where each node has a validator. On Overseer it is a graph (O2) whose nodes all require executable verification, plus a mission-level acceptance suite, plus a journal (R5) that tracks the graph.

Design. A `mission` is a saved graph template with acceptance criteria, startable as a long run. It is mostly composition of O2, O6, O9 and R5; the new part is the template format and its validation.

O16 — Cache-inheriting forks

What. A subagent started from a fork of the parent's conversation can reuse the parent's cached prefix, which makes “look at this from another angle with everything I know” nearly free in input cost [src: Claude Code fork subagents].

Design. `task mode=fork`: the subagent's first request is the parent's exact prefix (system, tools, messages up to the fork point) plus the brief as a new user turn, so the provider's cache hits. Fork subagents are read-only by default and inherit the parent's taint. The tool set must be byte-identical for the cache to hit, so forks use the parent's registry with a policy filter applied at the gate rather than a different spec array.

GATE

On Anthropic, a fork subagent's first call shows cache read tokens covering at least 90% of the parent's prefix.

O17 — Messaging with deadlines

What. Request and response between agents that are both alive (lead and persistent agents), with deadlines, so coordination does not degrade into polling.

Design. `MailboxMessage { from, to, kind, body, reply_to, deadline }`; replies correlate by id; a missed deadline produces a timeout notice (invariant 14). Messages are bounded in size and count per hour.

O18 — Scheduling and machine resources

What. Agents compete for the machine as well as for budget: eight writers all running `cargo build` will thrash a laptop.

Design. A resource-aware scheduler in the graph executor and the supervisor: a cap on concurrent heavy commands (builds and test suites, recognized by pattern or declared in `AGENTS.md`), queued fairly; node priorities; and a CPU load check that delays heavy starts when the machine is saturated.

6.5 What could break

WHAT COULD BREAK

Token blow-up. Multi-agent runs can cost an order of magnitude more. Mitigation: graph budgets carved from the parent, tier defaults that keep readers light, fork subagents that reuse cache, and E7 measuring cost per solved task with and without orchestration so the lead’s prompt is tuned to orchestrate only when it pays.

WHAT COULD BREAK

Merge-queue livelock. Writers repeatedly conflicting with each other. Mitigation: the queue serializes and verifies after every landing; repeated conflicts on the same files trigger a re-plan notice to the lead instead of endless resolution tasks.

WHAT COULD BREAK

Persistent agents accumulate stale context and stale memory. Mitigation: compaction, journal, and the dream pass apply to agent memory scopes; an agent’s context is reset to its journal on a configurable cadence.

WHAT COULD BREAK

Orchestration hides failures. A node fails quietly and the summary says “done”. Mitigation: every node’s verdict is in the graph summary, and the run cannot complete while any node is failed or unknown unless the lead explicitly drops it with a reason recorded in the journal.

6.6 Metrics

Metric	Target
Cost per solved task with orchestration ÷ without, on tasks marked parallel	≤ 1.5 with a pass-rate gain, or orchestration is not used for that class
Merge-queue landings that later fail verify	0
Fork subagent cache read share of prefix	≥ 90%

Metric	Target
Graph runs resumed after crash without lost or duplicated nodes	100%

7. Memory

7.1 The bar

Production harnesses converged on memory as a filesystem the agent edits: an index of pointers, topic files, progressive disclosure, git versioning, and a periodic cheap consolidation pass [src: [playbook thesis 5](#)]. Procedural memory (ACE-style append-only playbooks) shows the highest upside [src: [playbook thesis 5](#)]. Memory writes are side effects and must be gated, because poisoning attacks reach very high write success in research (MINJA about 70%, AgentPoison at least 80%) [src: [playbook thesis 5](#)]. Multi-hop conflict resolution remains unsolved for every system measured (MemoryAgentBench at or below 7%) [src: [playbook thesis 5](#)].

Overseer's memory already meets most of the converged bar and exceeds it on safety. The roadmap's job is to finish v3, then go where nobody else is: memory of outcomes, run journals feeding long-term learning, and measured uplift.

7.2 Where we stand

See Chapter 3.7. Memory v2 and v3 H1 are complete and red-teamed, with [fix2] closing every review finding except fix2-memory finding M8, which W0 resolves with the owner's option (a). Open: H2 (history search, applied learned skills) and H3 (dream pass, mounted repos, sync), plus the every: reminder form and the SKILL and Merge/Contradict operations that parse but do not apply.

7.3 Workstreams

M1 — History search (memory op=history)

What. v3 §4: search raw past sessions with Overseer's own BM25F engine, no model call.

Design. As specified in the memory v3 record: one document per user turn, fields weighted (user text 3, first prompt 2, assistant text 1), tool result bodies never indexed, RRF over BM25F, recency and same-project, four shapes (query, around, session, recent), output $\leq 8,000$ characters, redacted, provenance-wrapped as history, threat-flagged. Built lazily, refreshed by modification time.

Long-run addition. Journal events (R5) are indexed too, so “what did I decide about the auth adapter last week” finds the decision note directly.

GATE

The v3 record's verification list for §4, plus a test that tool result bodies never appear in any history output, plus a latency bound (≤ 50 ms over 400 sessions on the owner's M1, release build).

M2 — Applied learned skills

What. v3 §5: the review's SKILL and PATCH-SKILL operations actually create and patch skills, with probation, ledger and rollback.

Design. As in the record: learned skills under `~/.overseer/skills/<slug>/` with `created_by: overseer-learn`, status lifecycle, content-addressed ledger and blobs, read-before-write patching, usage tracking. User-authored skills are never touched.

GATE

The v3 record's §5 verification list; a red-team test that a tainted window can never create or patch a skill; rollback restores byte-identical content.

M3 — The dream pass

What. v3 §6: consolidation, exact-duplicate supersession, near-duplicate and contradiction judging with source checking, fading, skill lifecycle, `MEMORY.md` regeneration.

Scheduling. The deterministic subset at session start when stale (as specified), and the full pass as a daily workflow (A) on the gateway rather than a bare cron trigger, so it inherits retries and journaling.

GATE

The v3 record's §6 verification list; unattended runs stage every model-judged change; a dry run produces an exact diff of what would change.

M4 — Mounted memory repositories and sync

What. v3 §9.4–9.5: AMR-compatible mounted repos with ownership and trust, and `ff-only` sync that never forces or merges on its own.

M5 — Outcome memory

What. Remember what worked and what failed, per project and per task class, and use it when choosing an approach. No harness surveyed does this systematically **[infer]**.

Design.

- **Capture.** At run end (and at each journal milestone in long runs), the engine records an `Outcome` note candidate deterministically: task class (from the plan or the first prompt, classified by a small fixed taxonomy), approach markers (tools used, files touched, verify commands), result (verified pass, fail, abandoned), cost, duration, and the journal's decisions. No model call is needed for capture.
- **Distillation.** The learning review (and the dream pass) may turn repeated outcomes into procedural notes: “in this repo, migrations need the schema check before the test suite; three runs failed without it”.
- **Use.** Recall gains an outcome lane: when a new task's class and project match past outcomes, the top relevant lesson (success pattern or failure warning) is offered in the memory notice, within the existing three-note cap.
- **Safety.** Outcomes from tainted runs are quarantined like any other tainted write.

GATE

On a repeated task family in the rig, a warm store with outcome memory shows a measured pass-rate or cost improvement over a cold store, with paired statistics (E6). If it does not, outcome recall stays off by default.

M6 — Journals into memory

What. The run journal (R5) is the richest record of a long run. At run end its decisions and milestones feed the episodic note and the review digest, so a week-long run leaves durable lessons instead of a one-paragraph episode.

Design. The episode note for a journaled run is derived from the journal (goal, plan outcome, decisions, verify evidence) rather than the first prompt alone. The review digest includes journal decisions as a section with its own cap.

M7 — Cross-project promotion

What. A lesson learned in three projects is probably a user-level lesson.

Design. The dream pass looks for semantically equivalent procedural notes across project stores (term overlap plus one judge call, staged unattended) and proposes a user-level note that supersedes the copies' recall weight without deleting them.

M8 — Recurring reminders

What. The deferred `every`: prospective form (`every: weekday 09:00, every: 7d`), firing once per period with the existing claim-file mechanism. Long schedules belong to workflows (A5); `every`: covers personal reminders inside sessions.

M9 — Memory evaluation

What. Measure memory instead of trusting it.

Design. A LongMemEval-S adapter in the rig (planned since memory v2), plus Overseer-specific suites: repeat-question rate across sessions, recall precision on seeded facts, correction persistence (a correction given once is honoured in the next N sessions), poisoning resistance (seeded tainted content must not reach a live note), and the warm-versus-cold uplift test from M5.

M10 — Poisoning defences, extended

What. Every new path into memory (journals, outcomes, artifacts, persistent agents' stores, mounted repos) inherits the threat scan and taint rules.

Design. A single write chokepoint audit: every store write goes through one function that applies redaction, the strict threat scan, the taint check and the write bars. A test enumerates every caller.

M11 — Memory you can see and edit

What. Memory is only trustworthy if the owner can inspect and correct it easily.

Design. A memory panel in the TUI and web UI (X7): browse layers, see provenance and confidence, approve or reject pending items, edit or forget notes, see what was recalled in each turn and why (the ranking factors). The CLI already covers `pending`, `approve`, `reject`, `stats`, `log`, `restore`.

M12 — Shared and team memory

What. Several people or several machines sharing a project's memory.

Design. Mounted repos (M4) with a private remote are the mechanism; ownership and trust per mount decide who writes. Conflict handling stays git-native with `ff-only`, never automatic merges. Deliberately late: the owner's posture is "me first".

7.4 What could break

WHAT COULD BREAK

Memory makes the agent confidently wrong. A stale or wrong note recalled with authority. Mitigation: confidence and validity on every note, FEEDBACK lowering confidence on wrong notes, fading, and recall shown provenance-wrapped so the model treats it as evidence, not instruction.

WHAT COULD BREAK

Outcome memory learns superstition. Correlation from a handful of runs becomes a rule. Mitigation: distillation requires repeated outcomes with consistent evidence, new rules start at low confidence on probation, and M5's gate requires measured uplift before outcome recall is on by default.

WHAT COULD BREAK

The cache breaks. Memory content above the cache boundary changing mid-session would rewrite the prefix. Mitigation: the existing rule holds (resident memory assembled once per agent); all new recall and outcome lanes inject below the boundary.

7.5 Metrics

Metric	Target
Repeat-question rate across sessions on seeded facts	$\leq 5\%$
Correction persistence over the next 5 sessions	$\geq 95\%$
Poisoned content reaching a live note in the red-team suite	0
Warm-store uplift on repeated task families	Positive with 95% confidence, or the feature stays off
History search latency over 400 sessions	≤ 50 ms

8. Context Management

Context management decides **what is in the window**. Token efficiency (Chapter 10) decides **what it costs**. They overlap, but the questions differ: a cheap window full of the wrong things still fails the task.

8.1 The bar

Advertised context is not usable context: effective context runs at a half to a quarter of the sticker on long-context benchmarks, and every tested model degrades as input grows [src: **RULER, NoLiMa, Context Rot**]. Plain observation masking halves cost against a raw agent and matches LLM summarization on SWE-bench Verified [src: **Lindenbauer et al. 2025**]. Critical state belongs at the edges of the prompt. Summaries must never be re-summarized.

Overseer already does the hard part (compaction as a deterministic view, masking of stale results). The gaps are in precision: eviction counts results instead of measuring them, repeated content re-enters verbatim, notices replay forever, and nothing keeps the run's intent at the edge of the window.

8.2 Where we stand

See Chapter 3.4: deterministic compaction at 70–83% of context with a two-turn tail; keep-last-5 tool results with pinned `tools` search results; last 2 images; read dedup; spill to file over 30,000 characters.

8.3 Target: a context planner

Today, each mechanism acts on its own rule. The target is one deterministic **context planner** that builds each request's view from the log under an explicit token budget, with every inclusion and exclusion explainable. The planner is a pure function of (events, profile, budget) so it stays deterministic and testable, and it keeps the prefix stable so caching survives (Chapter 10).

Priority order inside the budget, highest first:

1. System prefix and tool specs (stable, cached).
2. The resume briefing and pinned journal state (R5), at the recency edge.
3. The recency tail (complete turns, on model-response boundaries).
4. Pinned items: loaded tool schemas, the active plan, files the agent is currently editing.
5. Recent tool results, newest first, until the result budget is spent.
6. Older material, represented by the compaction summary.

8.4 Workstreams

C1 — Token-aware eviction

What. Replace “keep the last 5 tool results” with “keep the newest results that fit a token budget”, because five results can be 150 KB or 5 KB.

Design. A result budget per profile (default 25% of usable context), filled newest first using the token estimator (K4), with pinned items charged first. Evicted results become the existing stale stub. Results the agent referenced in its latest reasoning or that belong to a file under active edit are pinned for the next turn.

Cache interaction. Eviction changes message content, which moves the cache point. To keep it rare, eviction runs in batches (when the budget is exceeded by a margin, evict down to a lower watermark), not one result per turn, so the prefix changes in occasional steps rather than every request.

GATE

On the rig, token-aware eviction matches or beats keep-last-5 on pass rate with lower input tokens per solved task, paired, on the same model. Cache hit rate does not drop by more than 2 points.

C2 — Cross-turn result deduplication

What. An identical `grep`, `glob` or `bash` output re-entering the window verbatim is waste. `read` already dedups; generalize it.

Design. Each tool result gets a content hash. When a new result's hash matches a result still in view, the new one is replaced by a stub naming the earlier call ("identical to the result of call c41"). When the earlier one has been evicted, the new one is shown in full. Deduplication is per-view, never altering events.

C3 — Notice and digest aging

What. `MemoryNotice` recalls, `SubagentDone` digests and reminders replay verbatim forever. After a few turns most of them are dead weight.

Design. Notices age like tool results: after K turns (default 6) a notice becomes a one-line stub with its id, unless pinned by the planner (for example, a digest whose artifact the current plan item consumes). Graph summaries supersede per-node digests.

C4 — Shedding old reasoning

What. Invariant 6 says reasoning blocks are opaque and round-trip verbatim. It does not say they must stay in the window forever.

Design. Per provider, following each provider's rules: where the API permits dropping reasoning from turns before the latest tool-use exchange (for example, providers that only require the most recent reasoning for continuity), old reasoning blocks are dropped whole from the view. Never edited, never summarized. Where a provider requires full round-trip (signed reasoning tied to tool pairing), the planner keeps them. Each adapter declares its rule with a fixture-tested reference to the provider documentation.

WHAT COULD BREAK

Getting a provider's reasoning rule wrong breaks requests or degrades quality silently. Mitigation: per-adapter declarations with documentation references, fixture tests, and an eval-rig A/B before enabling by default on any provider.

C5 — Edge placement of critical state

What. Put what matters most where models attend best: the start and the end.

Design. The stable start is the cached system prefix (unchanged). The end gets the resume briefing (R5), pending approvals (R8), in-doubt effects (R7) and the current plan item, in one compact block, rebuilt each turn from events. The block is small (budgeted at 600 tokens outside resume briefings) so it costs little uncached.

C6 — Retrieval-scoped reading

What. The model often reads whole files to find one function. Give it precise slices.

Design. Extend `read` with symbol and range targets backed by the repo map and tree-sitter where available (`read path=... symbol=AuthAdapter::refresh`), returning the symbol with a few lines of context and a line-range header. `grep` results include enclosing symbol names. `symbol` and `repo_map` stay deferred tools; the slice capability lives in `read` because it is the hot path.

GATE

On the rig, symbol reads reduce input tokens per solved task without lowering pass rate.

C7 — The context planner itself

What. The unifying piece described in §8.3, replacing the separate eviction, capping and aging rules with one budgeted planner.

Design. A `plan_view(events, profile, budget) -> View` function with an explanation trace (`ViewTrace`: what was included, what was stubbed, why) recorded on demand for overseer `why` (P11). The existing rules become planner policies with identical default behaviour, so the switch is behaviour-preserving before any policy changes.

GATE

Golden tests: for a corpus of logs, the planner with legacy policies produces byte-identical views to today's code. Only then are new policies enabled one at a time, each with its own rig A/B.

C8 — Optional semantic compaction

What. Mechanical summaries lose nuance on long refactors. An LLM condensation pass might retain more, at a cost.

Design. An opt-in compaction policy that adds a small-model condensation of the evicted span to the mechanical summary, never replacing it and never re-summarizing an earlier condensation (invariant 8 holds: each condensation covers raw events only). Ledgered, budgeted, and measured; it stays off by default unless the rig shows gains on long tasks.

C9 — Handoff instead of compaction

What. Amp's idea: instead of compacting for the tenth time, start a fresh context seeded deliberately.

Design. A `handoff` action (by the agent, or by the planner when compaction count passes a threshold): the run writes a `JournalNext` and a handoff brief, then the engine starts a fresh context window inside the same run and session, whose first user turn is the resume briefing plus the brief. The event log continues; the view resets. This is cleaner than repeated compaction because the new window holds a deliberate brief instead of a mechanical summary of a summary's neighbourhood.

GATE

On the long-horizon suite, handoff-after-N-compactions versus compaction-only: equal or better completion and lower cost. The threshold is tuned from the data.

C10 — Context health telemetry

What. Make context visible: per request, the token split between prefix, briefing, tail, results, notices and summary; the number of compactions; eviction counts; dedup hits.

Design. Recorded on each `ModelRequest` event as a compact breakdown; shown in the cockpit (X1) and `overseer stats`.

8.5 What could break**WHAT COULD BREAK**

Over-eager eviction hides the evidence the model needs. Mitigation: pinning rules for active files and referenced results; the stale stub always says how to get the content back (re-run or `read` the spill); rig A/B before any default change.

WHAT COULD BREAK

View changes destroy cache hit rate. Every eviction or aging edit changes message bytes. Mitigation: batched watermark eviction, aging at fixed turn counts, and the cache metrics in K10 to catch regressions.

8.6 Metrics

Metric	Target
Input tokens per solved task	Down $\geq$ 25% from the W0 baseline on the rig
Pass rate	No regression (non-inferiority at 3 points)
Share of window used by stale material at compaction time	$\leq$ 15%

9. Tool Efficiency

9.1 The bar

Tools are where an agent spends most of its turns and most of its tokens. The evidence on what works:

- Deferred tool loading cut tokens by 85% on Anthropic's example workload and raised MCP accuracy substantially [**vendor**]; top-k tool retrieval halved prompt tokens and tripled selection accuracy in RAG-MCP [**src**]; accuracy falls past roughly 30–40 tools [**src**].
- Code-as-action beats JSON tool calls by up to 20% success with about 30% fewer turns across 17 models [**src**: **CodeAct**], and programmatic tool calling reports large input-token cuts on heavy catalogs [**vendor**].
- Anchored (hashline) edits claim large output-token cuts [**vendor**: **omp**, **−61%**], unverified independently.

Overseer is already near the top here: deferral through `tools`, capability-free QuickJS `run_code`, byte-stable specs, one gated pipeline for every call. The remaining work is reach, precision, speed and proof.

9.2 Where we stand

See Chapter 3.5.

9.3 Workstreams

T1 — Hashline edits, behind a measured gate

What. Address edit targets by line plus a short content hash instead of reproducing old text, rejecting stale anchors before applying.

Design. `read` optionally returns lines prefixed with a short hash (`17:a3f|` `let x = ...`); `edit` accepts anchor: `"17:a3f"` ranges as an alternative to `old_string`. The harness checks the anchor's hash against the file's current line; a mismatch refuses with the current content of that region (read-before-edit, invariant 5, enforced at line granularity). Prototype behind an `EditFormat` switch, as the pillar refresh planned.

GATE

Adopt as default only if the rig shows lower output tokens per solved task with no pass-rate loss, paired, on at least two model families. Otherwise it stays an option.

T2 — `run_code` reach

What. `run_code` cannot call `computer` or `memory` today (deferred on live computer-use verification and memory v2 adoption).

Design. Allow `computer` sub-calls once U1 (live validation) passes, so GUI batches run as one script with intermediate observations never entering context. Allow `memory` search and get (read operations) immediately; memory writes stay native so the per-turn write cap and write bars apply at the top level.

T3 — A persistent QuickJS context

What. Let a script keep state across `run_code` calls within one run (parsed data, helper functions), deferred on demonstrated need.

Design. Opt-in `session: true` keeps one context alive per run with the same caps, reset on resume and on any error. Gate: the rig shows multi-call pipelines that benefit.

T4 — Native deferred loading on Anthropic

What. Use Anthropic's native tool-search and deferred loading for Anthropic sessions, so deferred tools get native calls instead of going through `tools op=call`.

Design. Behind a profile flag; the advertised spec array stays byte-stable because deferred definitions expand in the conversation, not the cached prefix `[doc]`. Gate: measured accuracy parity or better on the rig (as already recorded in the tool economy deferred list).

T5 — Project-level MCP configuration

What. `<repo>/.overseer/mcp.json` alongside the user file, so a project can declare its servers.

Design. Project servers default to `trust: ask` regardless of what the file says until the owner approves the file once (by content hash), because a cloned repository is untrusted input. Approval is stored per hash; any change re-asks.

T6 — Diagnostics beyond cargo

What. `diagnostics` is cargo-only. Make it language-aware.

Design. A checker table detected per project: `cargo check`, `tsc --noEmit`, `pyright/mypy`, `go vet`, `eslint`, plus an optional LSP client mode for languages where a server is installed, all in the sandbox with the bash environment allowlist (the existing rule). Output normalized to `file:line:col severity message` and budgeted like any result.

T7 — Availability that follows reality

What. Optional tools are detected once per registry. Installing `ast-grep` mid-session does nothing until a new session.

Design. Re-probe on explicit request (`tools op=refresh`) and at resume. A re-probe that changes availability is applied at the next resume or handoff only, never mid-window, so the spec array stays byte-stable within a context.

T8 — Parallel read-only tool calls

What. Models often emit several independent read-only calls in one turn. Run them concurrently.

Design. Calls classified `Read` with no ordering dependency (different paths, pure searches) dispatch on a small thread pool (default 4), results returned in call order. Writes, bash and anything above `Read` stay sequential. The control boundary semantics (interrupts and steering at launch boundaries) are preserved by treating the parallel batch as one launch boundary.

GATE

Wall-time reduction on turns with multiple reads, zero ordering-dependent test failures, identical event content to sequential execution apart from timestamps.

T9 — Shaped results

What. Tool output designed for the model, not for a terminal.

Design. Per tool: compact headers (path, line range, match counts), consistent truncation markers that say how to get more, TOON for uniform arrays (have), and for `bash`, structured exit status and a separated

stderr tail. Each change is A/B tested on the rig, because “obviously better” output formats sometimes are not.

T10 — Web fetch and search

What. Overseer has no web tool; bash network is denied. Research journeys (Chapter 1, journey B) need the web.

Design. A deferred web tool with `fetch` (URL to readable text, size-capped) and `search` (through a configured provider), routed through the egress proxy (S1) with domain policy, classified ExternalComms for anything but GET, and always latching the untrusted taint. Depends on S1.

T11 — Tool latency budgets

What. Measure and bound how long tools take, because over a long run tool time dominates wall time.

Design. Per-call durations already exist in events; add per-tool percentile telemetry (B2), slow-call warnings, and budget checks in the rig (p95 per tool must not regress between waves).

T12 — Tool descriptions as tested artifacts

What. Tool names and descriptions have non-trivial effects on accuracy [doc: Anthropic, “Writing effective tools for agents”].

Design. Every spec change runs a rig A/B; the startup-token test (have) pins size; a description changelog records why each change was made.

9.4 What could break

WHAT COULD BREAK

Parallel reads race with writes from the same turn. Mitigation: only pure reads run in parallel, and a turn containing any write dispatches everything sequentially.

WHAT COULD BREAK

Project MCP files become a supply-chain attack. A malicious repository declares a server that runs arbitrary code. Mitigation: hash-pinned approval before any project server spawns, servers run with the environment allowlist, and `trust: read` is never honoured from a project file without explicit owner approval.

9.5 Metrics

Metric	Target
Resident tool spec size	Target $\leq$ 6,000 characters (the test currently pins 6,136 plus 5% headroom), with every new tool deferred unless it is hot
Output tokens per solved task	Down $\geq$ 15% if T1 is adopted
Wall time per turn with multiple reads	Down $\geq$ 30% with T8

10. Token and Cache Efficiency

10.1 The bar

Agentic coding is an input-token business: sessions run about 25:1 input to output typically and up to about 150:1, and 60–66% of session cost is cache reads of re-sent context [src: **playbook thesis 3**]. Cache hit rate is the single most important infrastructure metric, with 90–95% in-session as the target. The economics are dominated by stable-prefix discipline, append-only history, deterministic serialization and tool-result budgets, not by cleverer prompts.

Provider mechanics that this chapter depends on, from the 2026-09-29 refresh [doc]:

- **Anthropic:** up to 4 breakpoints and a 20-block lookback; a minimum cacheable prefix of 1,024 to 4,096 tokens depending on the model, below which caching silently does not happen; a 5-minute TTL refreshed free on a hit; writes at $1.25\times$ and reads at $0.1\times$ of the input price; a 1-hour TTL at $2\times$ write; a top-level automatic caching option.
- **OpenAI:** automatic caching above 1,024 tokens, hits rounded to 128; `prompt_cache_key` matters for routing on models before GPT-5.6 and only separates accounting from 5.6; explicit breakpoints from 5.6.
- **DeepSeek:** on-disk caching by default, best effort.
- **opencode Zen:** best effort, pool-routed, `x-opencode-session` as the sticky key.

These details change. Every workstream below that depends on one re-verifies it against current documentation in its decision record.

10.2 Where we stand

See Chapter 3.4. The cache discipline is the strongest part of the engine: frozen segment order, prefix built once per agent, three Anthropic breakpoints, byte-stable specs, telemetry with an alert below 90%. The spend gate is careful. What has never happened is a live run at real prices: the gate and the review-share cap have only seen a \$0 model.

10.3 Workstreams

K1 — The fourth Anthropic breakpoint

What. Use the fourth breakpoint to protect the stable middle of the conversation.

Design. After a compaction, the compaction summary and the turns before the recency tail are stable until the next compaction. A breakpoint at the end of that stable region means that view edits in the recent part (eviction stubs, notice aging) only invalidate the part after it. Placement: tools tail, system tail, end of the stable region (new), rolling last block (existing). When there is no stable region yet (before the first compaction), the fourth breakpoint moves to the last block before the newest tool exchange, giving a two-step rolling pattern that tolerates one evicted result without a full miss.

GATE

On recorded long sessions replayed against the provider's usage accounting (live, budgeted, with owner approval for the spend), the fourth breakpoint raises cache read share or holds it equal while view edits are active. Never lowers it.

K2 — Time-to-live policy

What. Long runs wait: backoff, approvals, long builds. A 5-minute cache dies during a 10-minute build.

Design. A deterministic TTL choice per breakpoint. The expected gap before the next request is estimated from what the run is about to do (a long-running tool, a backoff, a pending approval); when the expected gap exceeds 5 minutes and the prefix is large, the stable breakpoints use the 1-hour TTL. The cost model is explicit: 1-hour write costs $2\times$ versus $1.25\times$, so it pays when it avoids at least one rewrite. The provider's ordering constraints on mixed TTLs are honoured and fixture-tested.

K3 — Cache keepalive

What. Refresh a large cached prefix during a wait instead of paying to rebuild it [src: Devin cache keepalive].

Design. While a run is in `waiting` and its cached prefix is larger than a threshold, issue a minimal request (smallest allowed output) just before the TTL would expire. A keepalive costs about $0.1\times$ of the prefix; a rebuild costs $1.25\times$ (or $2\times$ for 1-hour), so keepalives pay for about a dozen 5-minute refreshes before letting it expire is cheaper. The engine computes the break-even per wait and stops refreshing beyond it. Keepalives are ledgered with their own purpose. They are out-of-band: the response is discarded and never enters the event log or the model view. No keepalive is sent while the run is in `waiting{throttled}` or `waiting{rate_budget}`, because it would spend the very capacity the run is waiting for.

GATE

Simulated waits of 3, 10, 30 and 120 minutes choose keep, keep, keep-or-1h, expire respectively under the cost model; live spot check on one priced session with owner approval.

K4 — Accurate token counts without a heavy tokenizer

What. `tokens.rs` estimates with characters per token per family. Compaction timing, eviction budgets and the spend gate all depend on it.

Design. Self-calibration instead of shipping tokenizers: after every call, the provider reports actual input tokens; the engine compares with its estimate for the same request and updates a per-model correction factor (exponentially weighted, clamped). Within a few calls the estimate tracks the real tokenizer closely without adding a dependency. For OpenAI models where a tokenizer crate is small and exact, it can be added behind a feature if Z1's budget allows.

GATE

After 10 calls, estimate error below 3% on recorded sessions for every supported family.

K5 — Adaptive compaction threshold

What. `compact_at` is a fixed fraction per profile. The best moment depends on cache state and cost.

Design. Compaction resets the cache, so it is cheapest right after a cache expiry (a wait just ended) and most expensive right after a cache write. The planner compacts early when a cache miss is happening anyway and context is above a lower threshold, and otherwise waits until the hard threshold. Deterministic, logged with its reason.

K6 — Cost-aware routing inside a run

What. Use cheaper models where they do just as well, deterministically.

Design. Rules, not a learned router (refusal in Chapter 2): aux calls on the small model (have); subagent tiers by mode (have); plus per-task-class tier hints recorded in outcome memory (M5) when the rig or the owner's history shows a lighter tier succeeds for that class. The lead can always override.

K7 — Effort that comes back down

What. Each stuck trip raises effort one notch and it never decays.

Design. Decay one notch after N successful steps without a stuck signal (default 10), never below the profile default. Logged.

K8 — Output discipline

What. Output tokens are the most expensive per token. Agents narrate.

Design. A concise-output contract in the static prompt (measured on the rig), `max_tokens` sized per call type (tool-heavy turns need less than final reports), and the existing reflection caps.

K9 — One view of spend and cache across everything

What. `overseer stats` reads one session. The owner needs the whole picture.

Design. A global rollup (B2) over every session's ledger: spend by day, model, project, purpose (main, review, retry, keepalive, subagent tier); cache hit rate by model and profile; cost per run and per milestone. Feeds the supervisor's global budgets (R10) and Overseer Life's AI accounts module (F5).

K10 — Cache-miss forensics

What. When cache hit rate drops, find out why automatically.

Design. Each request records a prefix fingerprint per segment (system segments, tools, stable region, breakpoint positions). When cache read tokens fall unexpectedly between consecutive requests, the engine diffs fingerprints and records which segment changed, as a `CacheMiss{cause}` diagnostic. The existing hygiene lint catches volatility in tests; this catches it in production.

K11 — Provider-native context features

What. Providers now offer server-side context editing and compaction features. They might beat ours, or break our invariants.

Design. Evaluate each against invariants 1, 6 and 8 and on the rig. Adopt only those that keep the event log authoritative and the view deterministic (for example, a server-side tool-result clearing that mirrors our own eviction and saves bytes on the wire). The compaction seam removed as dead code stays removed until one passes.

K12 — The first priced live run

What. The spend gate, review-share cap and cost accounting have never run against real prices.

Design. One deliberate, owner-approved, budget-capped run on a priced model (Anthropic, since its caching is explicit), on a fixed task set, verifying ledger rows against the provider's own usage dashboard and the gate's clamping behaviour at the cap. Single session, local, with the owner's spend approval (standing rule).

GATE

Ledger totals match the provider's reported usage to within rounding; the gate refuses at the cap with the expected reason; the review stays under its share.

10.4 What could break

WHAT COULD BREAK

Price and policy changes at providers. A TTL price change or a breakpoint rule change silently makes our policies wrong. Mitigation: provider mechanics live in profile data with a verified-on date, and K10's forensics plus the cache alert catch regressions quickly.

WHAT COULD BREAK

Self-calibration drifts on unusual content. Code-heavy and prose-heavy requests tokenize differently. Mitigation: separate correction factors per content mix bucket, clamped, with the raw heuristic as the floor.

WHAT COULD BREAK

Keepalive spends money for nothing. Mitigation: the explicit break-even model, a per-run keepalive budget, and ledger visibility.

10.5 Metrics

Metric	Target
In-session cache hit rate, Anthropic	$\geq 92\%$
In-session cache hit rate, OpenAI-compatible	$\geq 85\%$
Resident startup tokens	$\leq 2,000$
Cost per solved task	Down $\geq 30\%$ from the W0 baseline at equal pass rate
Token estimate error after calibration	$< 3\%$

11. Policy: Principles, Rules, AGENTS.md and the Instruction Compiler

The owner named principles, rules and AGENTS.md as sub-pillars. This chapter treats them as one system, because that is what they are: every source of instruction and permission the harness obeys. Today they are scattered. The target is a compiler.

11.1 The bar

Production system prompts are assembled from section builders with a static/dynamic cache boundary and per-model variant flags; instructions are a compiler problem [src: **playbook thesis 8**]. There is no cryptographic separation between instructions and data, so the hierarchy is defence in depth: provenance-marked tool results, probabilistic detectors, deterministic tool-call gating, and human approval for irreversible actions. Prose rules shape attempts; only harness enforcement decides outcomes.

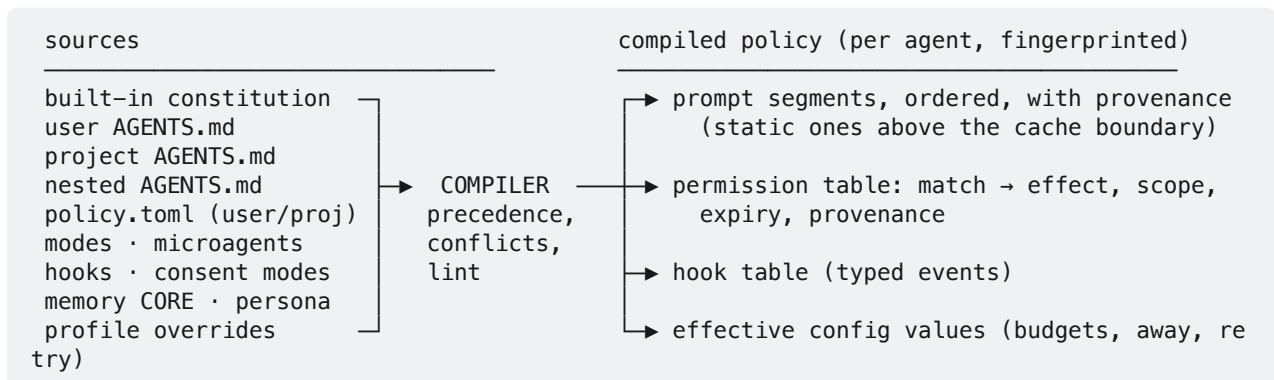
Every serious harness reads a project instruction file (AGENTS.md has become the cross-tool convention; Claude Code reads CLAUDE.md). Overseer, which keeps one of the most detailed AGENTS.md files of any project, does not read it at all.

11.2 Where we stand

Source	How it reaches the model or the gate today
Built-in contract and safety segments	Static prompt segments (<code>prompt.rs</code>)
Memory CORE and INDEX	Resident memory segment, assembled once per agent
Skills	Metadata index segment; bodies on demand
Persona	Approved bodies only, as a segment
Modes	Six built-ins: tool removal, edit globs, a prompt fragment as a nudge
Microagents	Substring triggers, per-turn nudges, capped
Hooks	<code>hooks.json</code> data rules at three points
Persisted rules	<code>~/.overseer/rules</code> , exact match, append-only, no deny
Autonomy ladder and presets	Flags and defaults in <code>perm.rs</code>
AGENTS.md	Not read
Consent for computer use	Per-call Ask today; owner decisions recorded, not implemented

Each source is consulted where it is used, with its own matching rules and no shared notion of precedence, scope, provenance or explanation.

11.3 Target: one compiler, two outputs



The compiler runs once per agent build (start, resume, model switch, handoff), like prompt assembly today, so the static prefix stays byte-stable within a context. Its output is fingerprinted; a change between builds is logged with a diff summary.

11.4 Workstreams

P1 — The AGENTS.md loader

What. Read project instructions, in the cross-tool format, with the right trust and placement.

Discovery.

Level	Location	Placement
User	<code>~/.overseer/AGENTS.md</code>	Static prefix (stable per session)
Project root	<code><git toplevel>/AGENTS.md</code>	Static prefix
Compatibility imports	<code>CLAUDE.md</code> , <code>.github/copilot-instructions.md</code> , <code>.cursorrules</code> at the root, read only if <code>AGENTS.md</code> is absent or imports them	Static prefix
Nested	<code>AGENTS.md</code> in any directory under the root	Injected below the cache boundary the first time the agent reads or edits a path under that directory, provenance-wrapped as <code>agents-md: <path></code> , once per context
Imports	<code>@path</code> lines (the convention some tools use) resolving inside the repository only	Inlined at the importing file's position, depth-limited

Size. The root file's resident share is capped (default 8,000 characters). Longer files are split by heading: the first section and a heading index stay resident, and sections load on demand through a `policy op=section` read, the way skills load bodies.

Trust. A repository's `AGENTS.md` is written by whoever wrote the repository. For the owner's own repositories that is trusted; for a freshly cloned third-party repository it is untrusted input that sits in the system prompt. Rule: the first time a project's `AGENTS.md` (by content hash) is seen, interactive sessions ask once to trust it; headless and detached runs treat an untrusted file as provenance-wrapped context below the boundary instead of a system segment, and arm nothing else. Trust is stored per hash, so any change re-asks. Because the owner's own repositories change their `AGENTS.md` often (Overseer's changes with most PRs), the owner can instead trust a repository by its path and remote, which covers every version of its files; that grant is listed and revocable like any other (P4). Path and remote are values a repository can change, so the grant is bound to the repository's identity as well (its canonical git directory and the device and inode of that directory), and a fresh clone or a remote change at the same path invalidates it.

Format. Plain Markdown. No special syntax is required; optional frontmatter may declare machine-readable parts (verify commands, heavy commands for the scheduler O18, protected paths), which the compiler lifts into config with provenance.

GATE

Fixture repositories for each discovery case; the root file appears in the static prefix and the prefix stays byte-stable across turns; nested files inject once on first touch; an untrusted file never lands above the cache boundary; Overseer's own `AGENTS.md` is read correctly and its section index fits the cap.

P2 — The instruction compiler

What. Invariant 17. One pipeline that turns every source into ordered prompt segments, a permission table, a hook table and effective config.

Precedence. From strongest to weakest:

1. **Engine invariants and hard denials.** Not overridable by any file (containment, secret handling, budgets exist). The owner's explicit consent configuration can relax gates where the owner has decided so (P5), recorded as such.
2. **Owner configuration** (user `policy.toml`, user AGENTS.md, consent modes).
3. **Project configuration** (project `policy.toml`, project AGENTS.md, nested AGENTS.md), within what the owner allows.
4. **Modes and microagents.**
5. **Built-in defaults.**

Within a level: deny beats ask beats allow, and a more specific match beats a general one. A project can tighten anything; it can loosen only what the owner's configuration delegates (`allow_project_relax = [...]`).

Conflicts. Contradictory instructions (one file says “always run tests with `--release`”, another “never”) are detected for the machine-readable parts and reported by `overseer policy lint`; prose conflicts cannot be detected reliably, so the compiled prompt orders sources by precedence and states that order to the model.

Per-model variants. The compiler can select segment variants per model family (the Codex practice of per-model tuned prompts), each variant tested on the rig.

Commands. `overseer policy show` (the compiled result with provenance per line), `overseer policy explain <tool> <resource>` (which rule decides and why), `overseer policy lint`, `overseer policy diff` (between two builds).

GATE

Every current source is migrated into the compiler with behaviour unchanged (golden tests on gate verdicts and prompt bytes for a corpus of configurations); afterwards no module reads a raw rules source directly (enforced by a lint).

P3 — Rules engine v2

What. Replace the bash-only, exact-match, append-only rules file with real rules.

Design. `policy.toml` at user and project level:

```
[[rule]]
match = { tool = "bash", command = "cargo test*" }
effect = "allow"
scope = "always"
reason = "tests are safe here"

[[rule]]
match = { tool = "write", path = "migrations/**" }
effect = "ask"

[[rule]]
```

```
match = { class = "ExternalComms", origin = "untrusted" }
effect = "deny"
```

Matchers: tool, glob or regex on the resource (command, path, URL, MCP tool name), irreversibility class, autonomy domain, taint state, origin (interactive, workflow, gateway channel), agent role. Effects: allow, ask, deny. Scopes and expiry from P4. `~/.overseer/rules` lines migrate automatically into equivalent rules, and the old file becomes read-only. The **[fix2]** grant keys for non-bash tools are the starting point.

P4 — Scoped, expiring, revocable grants

What. Approval fatigue makes security unusable; permanent grants make it meaningless. Grants need scopes.

Scopes. `once` (this call), `task` (this subagent or plan item), `run` (this run, including after resume), `session`, `origin+action` (for example, this website and this kind of action), `always`. Each grant carries an expiry (time, turn count, or run end), a reason, and its provenance (who granted it, from which frontend). `overseer grants` lists and revokes; the web UI and Overseer Life's Permissions page show the same table (F4).

DECISION

Money, security-setting and account-deletion actions always ask, in every mode except full automation where the owner has configured otherwise (Life security design approval trust model, 2026-10-05, and the consent decision of 2026-10-07).

P5 — Consent modes

What. Implement the owner's computer-use consent decisions (2026-10-07) as general modes the compiler understands.

Mode	Behaviour
Per-action (default)	The first screen read and the first mutating action in a task each ask; approval grants <code>task</code> scope for that domain; later calls in the same task do not ask
Full automation	Never asks, including the Rule-of-Two floor, per the owner's decision; secrets and sensitive surfaces follow the owner's dedicated permissions (P6)

Under full automation, nothing is silent in the record: every call that would have asked is journaled with the verdict that would have applied (S4), the kill switch remains (invariant 24), and the mode is shown in every frontend's status line.

GATE

The consent-modes branch (planned after integration) lands with tests for both modes across computer, bash, file and MCP domains; the per-task grant expires at task end; full automation produces the shadow-verdict journal.

P6 — Dedicated permissions for secrets and sensitive surfaces

What. The owner's decision: in full automation, secrets and similar surfaces are controlled by dedicated, user-configured permissions rather than a hard-coded denylist.

Design. Named categories, each with allow, ask or deny, configured per mode:

Category	Examples
secrets	Typing a brokered secret; reading secret-bearing files
password_managers	1Password, Keychain Access, Passwords, Bitwarden windows
system_security	System Settings privacy panes, SecurityAgent dialogs
payments	Checkout pages, payment apps, the Money class
identity	Account settings, 2FA setup, recovery codes, the Identity class
messaging_send	Sending email, posts, messages

Onboarding (X8) asks the owner to set these when full automation is first enabled; no default is imposed silently. Categories are matched by bundle identifier, application name normalization, URL patterns and tool classes, so renaming an application cannot launder it (the Synara technique, reused as a matcher rather than an unconditional denylist).

P7 — User-defined modes

What. Modes as files: `~/overseer/modes/<name>.toml` and project equivalents, declaring tool set, edit globs, prompt fragment, default tier and policy overlays. The six built-ins become files shipped with the binary.

P8 — Stream rules

What. omp’s TTSR idea [**src**]: a regex or syntax match on the model’s output stream aborts the generation, injects a rule, and retries from the same point. Rules cost no context until they fire.

Design. Rules declared in policy (`[[stream_rule]] pattern = "..." message = "..."`). On a match during streaming, the partial response is discarded (never logged, consistent with R2’s partial-stream rule), the rule text is appended as a provenance-wrapped nudge, and the request is retried. A per-run cap prevents loops. Measured on the rig before shipping any default rules.

P9 — Contextual rules (microagents, version 2)

What. Microagents trigger on crude substrings. Make triggers precise.

Design. Triggers by path glob (the agent touched `*/*.sql`), file type, tool use, word-bounded keywords, plan item tags, and `always`. Bodies inject below the boundary once per context, capped as today. Existing microagent files keep working.

P10 — Hooks, version 2

What. Typed hook events with an optional script form.

Events. Before and after a tool call, before and after a model call, run start and end, compaction, approval requested and resolved, effect intent (R7), journal milestone.

Forms. Data rules (as today), and script hooks that receive JSON on stdin and return JSON, run in the sandbox with the bash environment allowlist and a hard timeout, failing open or closed as each hook declares. Script hooks from project configuration need the same hash-pinned trust as project MCP (T5).

P11 — Explain any decision

What. Invariant 18. `overseer why <event-id>` (and a “why” affordance on every UI element) explains: for a permission verdict, the rule and its provenance; for an eviction or compaction, the planner trace (C7); for a retry or failover, the failure class and policy; for a routing choice, the rule that picked the tier.

P12 — Managing permissions

What. A permissions page in the web UI and TUI, and the `overseer grants` and `overseer policy` commands: every rule and grant with scope, expiry, provenance and last use; revoke in one action. The same model backs Overseer Life's Permissions page (F4).

P13 — Principles enforced by the build

What. Turn the constitution into checks where possible.

Check	Enforces
Prefix hygiene lint (have)	Invariant 2
Every DEFERRED (marker has an owner and a gate	Deferral principle
Every blocking call wrapped in a deadline or allow-listed (R3)	Invariant 14
Every new event kind declares skippability; golden logs replay (R13)	Invariant 21
Every store write goes through the chokepoint (M10)	Invariants 19 and 25
No module reads raw policy sources (P2)	Invariant 17
Size, RSS and startup budgets (Z1)	Invariant 22

P14 — The instruction hierarchy as defence in depth

What. Make the hierarchy explicit to the model and enforced by the engine: system segments are instructions; everything inside a provenance wrapper is data. The compiled prompt states this once, briefly. The gate never trusts the model's claim that an instruction came from the owner. Injection research is tracked and the corpus extended (S6).

P15 — Git safety defaults

What. Long unattended runs touch git constantly. Some git operations destroy work or publish it, and they need defaults that hold in every mode unless the owner configures otherwise.

Defaults compiled into every policy.

Operation	Default
Force push, history rewrite of published branches, branch deletion on a remote	Deny
Push to <code>main</code> , <code>dev</code> or the default branch	Deny (the branch ladder in <code>AGENTS.md</code>)
Push to any other remote branch	Ask (ExternalComms)
<code>git reset --hard</code> , <code>git clean -fdx</code> , checkout over uncommitted changes in the owner's tree	Ask; allowed silently inside the agent's own worktrees
Commits	Allowed in the agent's worktrees, with an agent trailer so authorship is clear; commit signing follows the owner's git config
Pull requests	Ask; repository conventions such as Overseer's <code>[skip actions]</code> rule come from its <code>AGENTS.md</code> frontmatter and are applied automatically

11.5 What could break

WHAT COULD BREAK

A malicious repository steers the agent through its `AGENTS.md`. Mitigation: hash-pinned trust before the file enters the system prompt; untrusted files are data, below the boundary; engine invariants are never overridable by project files.

WHAT COULD BREAK

The compiler becomes a bottleneck of complexity. Mitigation: it is a pure function with golden tests; every source migrates behaviour-preserving first; new semantics land one at a time.

WHAT COULD BREAK

Full automation removes the safety floor by design. That is the owner's decision and this roadmap honours it. Mitigation within that decision: the shadow-verdict journal, dedicated sensitive-surface permissions set at onboarding, the always-visible mode indicator, and the kill switch.

11.6 Metrics

Metric	Target
Approval prompts per 100 tool calls in per-action mode on the rig workload	≤ 2
Decisions with a complete <i>why</i> explanation	100%
Policy sources read outside the compiler	0


```
[[step]]
id   = "quota"
kind = "tool"
tool = "life.connector"
args = { sources = ["claude", "codex", "cursor", "devin", "opencode"] }

[[step]]
id   = "ci"
kind = "agent"
brief = "Check CI status on my open PRs in sirrayi/overseer; one line each."
tier  = "light"
mode  = "read"

[[step]]
id   = "digest"
kind = "agent"
after = ["quota", "ci"]
brief = "Write the morning digest from the inputs. Name any source that failed."
inputs = ["quota", "ci"]

[[step]]
id   = "send"
kind = "notify"
after = ["digest"]
channel = "telegram"
body   = "${digest.output}"
```

Interpolation is a tiny, deterministic expression language (field access, defaults, string formatting), never a model call and never code execution.

GATE

A parser with precise errors (line, step, field), a validator that rejects cycles, unknown references, missing budgets and unknown step kinds before any instance starts, and a schema document generated from the code.

A2 — The durable executor

What. Execute workflow instances so that any crash, reboot or upgrade resumes them exactly where they were.

Design.

- **Event-sourced instances.** `~/.overseer/workflows/instances/<id>/events.jsonl` with the same hash-chained log machinery as sessions: `InstanceStarted{definition, version, trigger_payload}`, `StepScheduled`, `StepStarted{attempt}`, `StepCompleted{output_ref}`, `StepFailed{class}`, `StepCompensated`, `InstanceCompleted{reason}`.
- **Execution semantics.** Steps run at least once; effects inside them are exactly-once through R7's intent records and idempotency keys (derived from instance id and step id, never the attempt, so retries share a key). A step whose outcome is unknown after a crash is reconciled before being retried, with the same per-family rules as R7.
- **Outputs.** Step outputs are artifacts (O3), referenced by id, carrying taint.
- **The daemon holds no credentials.** Tool steps that need credentials (a Life connector, an authenticated MCP server) run in a spawned step process that uses the credential broker, exactly as agent runs do; the executor only schedules and records. This keeps the gateway's thin, credential-free design (playbook Alfred architecture).

- **Agent steps** run as supervised long runs (R4) with the step's brief, tier, budgets and the workflow's policy overlay; their completion is an event delivered to the executor. Unlike today's gateway spawns (`overseer exec --bare`, whose session is throwaway in the temp directory), agent steps keep a durable session under the instance directory so their journal, effects and logs survive for resume and audit; whether they read memory is part of the workflow's policy.
- **Concurrency.** A global cap on concurrent instances and agent steps (extending today's `max_concurrent`), queued fairly, with per-workflow limits ("never two morning digests at once").

GATE

Crash injection at every step transition across a 10-step workflow with agent, tool, wait and approval steps: every instance completes with each effect exactly once. Daemon restart mid-instance resumes it. 1,000 instances of a trivial workflow complete with bounded memory.

A3 — Step kinds

Kind	Behaviour
agent	Supervised run with brief, mode, tier, budgets, inputs; output is its final report or declared artifacts
tool	One deterministic tool or connector call (a sandboxed command, a Life connector fetch, an MCP tool on a read lane)
wait	Duration, until a time, or until an event (with a deadline)
approval	An inbox item with a preview and a deadline; outcome <code>approved</code> , <code>rejected</code> or <code>expired</code>
branch	Choose the next steps by a deterministic condition over earlier outputs
map	Fan out a step over a list (with a concurrency cap) and fan in
workflow	Call another workflow as a sub-instance
notify	Send through a channel via the outbox, idempotent
compensate	Declared undo for an earlier step, run on failure (A4)

A4 — Retries, timeouts and compensation

What. Failure handling per step, and sagas for multi-step changes.

Design. Each step declares `retry = { max, backoff, on = ["Transient", "Throttled"] }` (classes from R1) and `timeout`. When a step fails terminally, the instance runs the `compensate` steps of completed steps in reverse order (for example, close the PR that an earlier step opened; delete the draft), then ends `failed` with a report. Compensation steps are themselves retried and journaled. Steps with no possible compensation (a sent message) are marked `irreversible` in the definition, and the validator warns when an irreversible step precedes a fallible one without an approval between them.

A5 — Schedules and time

What. Schedules that are right in every time zone, across daylight saving changes, after sleep, and after missed windows.

Design.

- **Real time zones.** The gateway's attention gate uses a fixed offset (`OVERSEER_TZ_OFFSET_MIN`), which is wrong for half the year in any zone with daylight saving. Schedules carry an IANA zone name, resolved against the system's zoneinfo database. Ambiguous and skipped local times at transitions have defined behaviour (run once at the first occurrence; run at the next valid minute).
- **Missed runs.** Per trigger: `skip`, `run_once` (one catch-up run on wake), or `run_all` (bounded).
- **Calendars.** Business days, quiet hours, and owner-defined blackout dates.

- **Jitter.** Optional, to avoid many workflows firing on the same second against the same provider.

GATE

Table tests across spring-forward and fall-back transitions in at least three zones; a simulated eight-hour sleep across a scheduled time produces exactly the configured missed-run behaviour.

A6 — Taint through workflows

What. Invariant 19. A workflow that reads an email in step 1 and sends a message in step 5 is the lethal trifecta spread across steps.

Design. Each step output carries the taint of its inputs and of anything the step read. A step consuming tainted input runs with its untrusted latch armed from the start; the policy overlay can require approval for external effects downstream of tainted steps. The gateway's existing untrusted floor (Act downgraded to draft for untrusted origins) generalizes to: no step downstream of untrusted input performs an external effect without approval, unless the owner's consent configuration for that workflow says otherwise, recorded explicitly.

A7 — Inbox, version 2

What. The inbox is the proactive frontend: triage → item → approve, reject or snooze → receipt [**src: playbook, Agent Inbox**].

Design. Items with typed actions (approve, reject, edit then approve, snooze until, open run), grouping into digests, deadlines with the away policy (R8), receipts after execution, and availability in every frontend (TUI, web, Telegram, later the Life app). Items never expire silently: expiry is an outcome recorded in the instance.

A8 — The workflow library

What. Built-in templates that make the engine useful on day one, each a tested definition:

Template	What it does
morning-digest	Quota, CI, calendar-free status digest (the example above)
memory-dream	Nightly dream pass (M3)
ci-watch	Watch open PRs; on failure, diagnose with a read agent and post a summary to the inbox
dependency-review	Weekly: list outdated dependencies, run <code>cargo deny</code> , draft an update PR in a worktree for approval
quota-guard	Hourly: warn when an AI account crosses a usage threshold; suggest routing changes
cert-expiry	Daily: check certificate expiry on owner domains
session-gc	Weekly: <code>overseer gc</code> report and approved cleanup (R11)
long-run-babysitter	For any long run: hourly health summary to the owner, only if something changed

A9 — Versioning and migration

What. Changing a workflow definition must not corrupt running instances.

Design. Instances pin the definition version they started with. A new version applies to new instances. Running instances finish on their version by default; the owner can migrate an instance at a step boundary when the new version declares a mapping of step ids. Old definitions are retained while any instance uses them.

Two further rules, because workflows run for months and the engine will be upgraded under them:

- **Mapping is not enough when meaning changes.** Migration is refused when a mapped step keeps its id but changes its kind, effect class, irreversibility or compensation, unless the owner confirms that specific change.
- **Engine compatibility is declared and checked.** Each definition declares the engine range it was written for; the instance log records the engine version at every step; a binary outside the range refuses to resume the instance and the supervisor keeps the previous binary for it (the R13 drain policy, applied to workflows).

A10 — Testing workflows

What. A workflow is code; it needs tests.

Design. `overseer workflow test <name>` runs a definition against fakes (recorded tool outputs, stub agents returning fixtures, a fake clock), asserting the path taken and the outputs. `overseer workflow replay <instance> --definition <file>` replays a past instance's trigger and recorded step outputs against a new definition to show what would change. `overseer workflow dry-run` validates and prints the plan without effects.

A11 — Triggers, version 2

What. Better triggers than polling.

Trigger	Design
File events	Native notifications (FSEvents on macOS, inotify on Linux) behind a feature gate, with the existing mtime polling as fallback
Webhooks	HMAC signature verification per source, replay protection (have, improved by [fix2]), schema-checked payloads
Channels	Telegram (have); more channels as the outbox gains them
Connector events	Overseer Life connectors emit change events (follower delta, quota threshold, renewal soon)
Run and workflow events	A run completing, a workflow failing, a budget threshold: workflows can trigger on other workflows
Manual	<code>overseer workflow run <name> [--input ...]</code> and the inbox

A12 — Seeing automations

What. `overseer workflows` (definitions, schedules, last and next runs, failure counts) and `overseer instances` (state, current step, history), with the same views in the web UI and a daily health line in the digest.

Failure paths that cannot fail with what they report on (premortem two, Chapter 22):

- A failure of the digest workflow itself alerts through a channel independent of the digest: the desktop notifier and a cockpit badge.
- Every scheduled workflow has a dead-man's switch ("no successful completion within its expected window") with a primary and a fallback channel.
- Channel credentials (bot tokens, webhook secrets) get their own daily health check, which reports through the other channel when one fails.

12.5 What could break

WHAT COULD BREAK

Automations that nobody watches fail silently for weeks. Mitigation: every instance ends with a recorded reason; failures produce inbox items; the digest reports failed and skipped instances; a dead-man's switch alerts when a scheduled workflow has not completed within its expected window.

WHAT COULD BREAK

A buggy workflow spams the owner or spends money in a loop. Mitigation: per-workflow budgets and rate limits, per-channel send caps in the outbox, the kill switch, and the validator's warnings.

WHAT COULD BREAK

Daylight saving and sleep cause double or missed runs. Mitigation: A5's real time zones, missed-run policies and tests at transitions.

WHAT COULD BREAK

A trigger becomes a remote-control channel for an attacker. A webhook or Telegram message that starts a workflow is untrusted input. Mitigation: signature verification, the untrusted floor, taint propagation (A6), and approval before external effects downstream.

12.6 Metrics

Metric	Target
Instances resumed correctly after daemon crash or reboot	100%
Duplicate effects across chaos tests	0
Scheduled workflows completing within their window over 30 days	$\geq 99\%$ (excluding owner pauses)
Silent failures (failure without an inbox item or digest line)	0



Part III — Supporting Systems

13. Security and Trust

13.1 The bar

The “lethal trifecta” (private data, untrusted content, an exfiltration channel) is a session property, and a taint bit is the highest-leverage security feature [src: **playbook thesis 8**]. Agentic browsers were broken repeatedly in 2026 through prompt injection (Trail of Bits on Comet, Zenity’s zero-click hijacks, the University of Washington same-origin study) [src: **Life plan §1.5**]. Credentials must never enter context; consequential actions need approval unless the owner deliberately decides otherwise.

Overseer’s security posture is already unusual for a harness: Rule-of-Two latches, sandboxed bash with network off, credential broker sentinels, a hardened localhost UI, an injection corpus in the tests, and taint that persists across resume. Long runs and automation widen the attack surface in time and in reach, so the posture must widen with them.

13.2 Workstreams

S1 — The egress proxy

What. The open item since v1: a loopback proxy with domain allowlists, so commands and tools can reach the network selectively instead of not at all.

Design. A small proxy thread in the run process (no new runtime) bound to loopback with a per-run token; the sandbox allows connections only to it; it enforces per-run domain policy compiled by P2 (allow, ask, deny per domain and method), logs every request as an event (host, method, bytes, verdict), and blocks known exfiltration patterns (large uploads to unknown hosts under the untrusted latch). Commands needing the network (package installs, `git fetch`) get it through policy rather than `--no-sandbox`. The web tool (T10) and future connectors route through the same proxy.

Platforms differ, so S1 lands in two parts.

- **S1a, macOS (W4).** Seatbelt profiles can allow outbound connections to one loopback port while denying everything else, so “only the proxy” is directly expressible.
- **S1b, Linux (W10, with Z5).** `bwrap` network isolation is all or nothing: `--unshare-net` gives the sandbox an empty network namespace with no route to a proxy outside it. Reaching the proxy needs extra machinery, chosen in S1b’s decision record from: a user-mode network stack for the namespace (`pasta` or `slirp4netns`) restricted to the proxy, or a Unix socket bind-mounted into the sandbox with a minimal relay inside it. Until S1b lands, Linux keeps today’s deny-all egress; nothing silently opens.

GATE

On macOS (S1a): sandboxed `curl` to an allowed domain succeeds and is logged; to a denied domain fails with a clear message; under an armed untrusted latch, a POST to a new domain asks; DNS rebinding and direct-IP bypass attempts fail.

S2 — Finish the credential broker

What. Consent grants are recorded but rate and window enforcement is not wired (`cred.rs`).

Design. Enforce per-credential rate limits and time windows at the broker; scoped capability tokens per run; brokered secrets never in child environments (invariant 25); a leak test that scans every event, spill, artifact, journal and snapshot for sentinel-resolved values.

S3 — A threat model for long runs and automation

What. Extend the Life security design's threat model (`docs/research/2026-10-05-life-security.md`) to the harness's new shape.

New threat	Mitigation
Injection read at hour 60 steers the rest of a long run	Latches persist (have) and propagate through artifacts, blackboard and workflow steps (invariant 19); goal audits (O9)
A tainted subagent poisons shared artifacts	Artifact taint (O3); consumers' latches arm
A webhook or channel message triggers harmful automation	Signatures, untrusted floor, approval before external effects downstream (A6)
Auto-resume replays a harmful or duplicate effect	Effect journal (R7)
A malicious repository's AGENTS.md, MCP file, hooks or skills	Hash-pinned trust (P1, T5, P10)
The supervisor or daemon is abused as a confused deputy	The daemon stays thin and credential-free; its control socket is owner-only with request deadlines (have); every spawn carries the originating trust level
Full automation removes approvals	Owner's decision; shadow-verdict journal (S4), sensitive-surface permissions (P6), kill switch
Persistent agents accumulate excess privilege	Per-agent permission profiles, grant expiry, periodic review prompts in the digest

S4 — The full-automation audit trail

What. The owner chose a full-automation mode that never asks. The record must make that safe to live with.

Design. Every call that would have asked under the per-action mode is recorded with a `ShadowVerdict { would_have: ask, rule, class, taint }`; a daily digest section summarizes them by class; `overseer audit --mode full-auto --since ...` lists them; any external, money or identity effect performed under full automation is highlighted. Nothing is blocked by this; it only makes the decisions visible after the fact.

S5 — Signed builds and provenance

What. Signed, notarized macOS builds; reproducible release builds; an SBOM; release provenance attestation. Waits on the Apple Developer account (owner action) for signing; reproducibility and the SBOM can start now.

S6 — The injection corpus, extended

What. `tests/injection_asr.rs` gates the latches today. Extend it with content types the new pillars introduce: AGENTS.md payloads, artifact and blackboard payloads, workflow trigger payloads, MCP tool descriptions, memory recall content, web pages through the `web` tool, screen text from computer use. Track attack success rate per category; any regression fails the gate.

S7 — Secrets hygiene

What. Scan session logs, spills and archives for secrets that slipped in through owner input (the 2026-09-20 pasted key is the motivating case), redact on request, and remind the owner to rotate. The provider key rotation itself is an owner action already on the to-do list.

S8 — Sandbox hardening

What. Linux: add Landlock and seccomp filters alongside `bwrap` where available; gVisor as a pinned runtime option (exists as a runtime name). macOS: keep `sandbox-exec` profiles minimal and tested; track

Apple's direction on the deprecated profile language and plan for an Endpoint Security or container-based alternative if it is removed.

S9 — A standing red-team programme

What. The fix2 cycle proved the value: adversarial review found real bugs in every area. Make it periodic: at every wave close, an independent adversarial review of the wave's delta (locally, honouring the cloud posture), findings tracked to closure with the same rigour as fix2.

S10 — The kill switch everywhere

What. Invariant 24. A single owner action stops every run and workflow: TUI key, web button, `overseer stop --all`, a Telegram command, and a `STOP` file the daemon watches (exists for the daemon). Latency target: every agent at its next boundary within 2 seconds; every child process killed within 5.

GATE

From each frontend, the kill switch stops five concurrent runs and two workflow instances within the target latency, verified by a test harness.

S11 — Data at rest, retention and backup

What. Session logs, journals, artifacts, spills and memory stores hold the owner's code, conversations and sometimes personal data. Today they are plaintext under owner-only permissions in `~/overseer`. A week-long run multiplies how much accumulates.

Design.

- **Retention.** Owner-configured retention per data class (sessions, spills, artifacts, workflow instances), applied by `overseer gc` (R11): compress, archive, then delete only on the owner's explicit command or retention rule. Memory stores are excluded; they have their own lifecycle (M3).
- **Encryption at rest (optional).** Archived segments and backups encrypted with a key in the macOS Keychain (Linux: the secret service), matching the Life security design's approach. Live logs stay plaintext for performance and debuggability unless the owner opts in.
- **Backup and restore.** `overseer backup` writes an encrypted bundle of memory stores, policy, agents, workflows and selected sessions; `overseer restore` verifies hash chains before importing.
- **Export and erase.** Export everything about a project; erase a project's sessions and memory in one audited command.
- **Secrets.** The leak scan (S7) runs over backups before they are written.

GATE

Retention rules applied to a synthetic year of sessions leave exactly the expected set; a backup restores to a fresh home directory with every hash chain verifying; an encrypted archive is unreadable without the key.

13.3 What could break

WHAT COULD BREAK

Security friction drives the owner to full automation everywhere. Then the per-action protections never run. Mitigation: per-action mode tuned so prompts are rare (≤ 2 per 100 calls, P-metrics), scoped grants, and good defaults.

WHAT COULD BREAK

The egress proxy becomes a bypass. Mitigation: the sandbox profile only allows the proxy's loopback port; the proxy authenticates the run token; tests cover bypass attempts.

14. Computer Use and the Browser

14.1 The bar

OSWorld-Verified frontier results sit around 83–86%, mostly vendor-reported [src]. An accessibility snapshot costs a few hundred tokens against thousands for a screenshot [src]. Synara, built on the same cua-driver, adds foreground versus background consent, screen-lock revocation, a progress guard, settle-based waits, approval queue limits and a hard denylist [code: synara probe, 2026-10-07].

14.2 Where we stand

After the lean pass on `fix2-computer` [fix2]: cua-driver is the only backend; every argument Overseer sends exists in the installed driver's schemas (v0.26.1) and in Synara's patched build; observations are lean (roles without the AX prefix, noise actions dropped, visible-depth indentation); model-visible results carry no audit fields; screenshots are sized to what the active model sees without provider downscaling, with one refit retry and refusal beyond; post-action reads only happen for observed windows; batches validate fully before dispatch; marks bind by ancestry path; modal dialogs gate actions. Nothing has ever run against a live desktop.

14.3 Workstreams

U1 — Live validation on the owner's Mac

What. Run the ignored `computer_live` test and a scripted live checklist after granting CuaDriver Accessibility and Screen Recording.

Checklist. Marks persist or churn as expected on real AX trees; `wait_for` latency and verify formats; modal roles in real apps; schema names; secret typing through the broker; screenshot sizes and image budgets at each model tier; zoom coordinates mapping back correctly; refusal codes (`target_not_on_active_space`, `auth_sheet_focused`, `stale_element_token` and others) surfaced usefully.

GATE

Every checklist item passes or produces a filed finding; findings fixed before U2 merges.

U2 — Consent modes for computer use

What. P5 applied: per-action mode asks once per task for screen reads and once for mutating actions; full automation never asks, with P6's sensitive-surface permissions and S4's shadow-verdict journal.

U3 — Progress guard

What. Synara's idea: count repeated refusals and unverified acts on the same target; after three, block that target for the task and tell the model why. Feeds the stuck detector as a computer-specific signal.

U4 — Settle waits

What. `wait_for` polls a predicate. Add `settle`: wait until the window's AX tree stops changing for a quiet interval (bounded), using the driver's settle support where the patched build provides it and AX-hash polling otherwise.

U5 — Clipboard

What. Read, write and paste as explicit actions, each its own permission category (clipboard contents are often secrets), with reads latching the untrusted taint.

U6 — Overseer’s own computer helper

What. Today the owner installs CuaDriver.app separately. The target is an Overseer-managed helper app built from the MIT cua-driver crates at a pinned revision (patched like Synara’s where useful), installed on first use. macOS attaches Accessibility and Screen Recording grants to a signed app bundle, so this needs stable signing: it waits on the Apple Developer account (owner action). Until then the external driver remains supported.

DECISION

The helper must stay a separate signed app; the harness binary itself must never hold Accessibility or Screen Recording grants, because macOS would attribute them to the terminal, extending screen and input access to every process run in it.

U7 — Revocation on lock, sleep and session switch

What. Any standing computer-use grant dies when the screen locks, the machine sleeps, or the session switches, exactly as Synara does. Detected through the supervisor (R12) and session notifications.

U8 — The browser

What. Chromium-family browsers through CDP refs (have); others through the AX tree (have). Next: page snapshots as structured, token-lean text; navigation through the egress proxy (S1) for domain policy; per-origin grants (P4). The in-app WKWebView browser is Overseer Life’s (Phase 1 of the Life plan); the harness provides the browser-agent protocol (element refs, actions, proofs) it implements.

U9 — Computer-use evaluation

What. A local OSWorld-style subset of tasks on the owner’s Mac apps, run with owner presence (live desktop), measuring success, steps, tokens and image tokens per task, before and after each computer-use change.

U10 — GUI scripts through `run_code`

What. T2’s computer reach: a script that observes and acts in a loop without each observation entering context. Waits on U1.

14.4 What could break

WHAT COULD BREAK

Real AX trees break assumptions made against the fake driver. Mitigation: U1 before anything else ships; fixtures captured from real apps replace hand-written ones.

WHAT COULD BREAK

Computer use in full automation reaches a password manager or payment page. Mitigation within the owner’s decision: P6 categories configured at onboarding, matched by bundle id and normalized name, and S4’s audit.

15. The Model Layer

15.1 Where we stand

Adapters for Anthropic, OpenAI-compatible endpoints, the Responses API and Gemini, plus opencode routing (Chapter 3). A profile table (`profile.rs`) holds per-model context sizes, prices, effort maps and compaction fractions; rows today include `claude-fable-5`, `claude-opus-4-8`, `claude-sonnet-5`, `claude-sonnet-4-5`, `claude-haiku-4-5`, `deepseek-v4.1-flash` and `muse-spark-1.3-contributor`, with an `unknown` fallback. Tiers derive light and heavy models from the family. A small-model aux tier escalates to main. Errors other than one malformed retry end the run.

15.2 Workstreams

L1 — Error classification in every adapter

What. R1's taxonomy, implemented per adapter with fixtures captured from real error responses (status, headers, body). Shared with R1; listed here because each adapter owns its mapping table.

L2 — A living profile registry

What. Profiles drift: prices change, models launch and retire, context limits grow.

Design. Profile data moves to a data file embedded at build time with a `verified_on` date per row and per field group (prices, limits, cache rules); a CLI command shows stale rows; an owner-approved refresh updates them from provider documentation (manually researched, not scraped at runtime). Unknown models fall back to conservative defaults with a visible warning.

L3 — Capability probes

What. Know what a model can do before relying on it.

Design. A cheap probe per model on first use (cached per binary version): tool calling, parallel tool calls, image input and limits (feeding computer use's image sizing), reasoning output and its round-trip rule (C4), cache reporting fields, maximum output. Results recorded in a local capability cache and in the session's `SessionStart`.

L4 — Failover ladders

What. When the primary model is unavailable, continue on an equivalent rather than stopping.

Design. Per profile, an ordered ladder (`opus` → `sonnet` → `another family's flagship`), with rules: failover only on `Overloaded`, sustained `Throttled` or `Degraded` (R1); read-only subtasks fail over freely; the lead's own turns fail over only to models of equal or higher tier unless the owner allows a downgrade; every failover is an event and a journal note; the run returns to the primary after a recovery probe succeeds. Cross-family failover requires reasoning blocks from the other family to be dropped from the view (they cannot round-trip across providers); the planner handles it at a handoff boundary (C9), never mid-exchange. Multi-key support in one session (for example, two Anthropic accounts) is part of this, which also unblocks cross-provider verification (O11).

GATE

Fault injection marks the primary overloaded for 30 simulated minutes: the run continues on the ladder, returns to the primary afterwards, and the journal records both transitions. No reasoning block ever crosses providers.

L5 — Local and open-weight models

What. A first-class local path (an OpenAI-compatible local server) for privacy-sensitive work and offline operation, with profiles for common local models, conservative tool-calling assumptions verified by L3, and zero-cost ledger rows.

L6 — Native provider features

What. Adopt provider features that fit the invariants: Anthropic's tool search (T4), extended cache TTL (K2), server-side tool-result clearing if it matches our eviction (K11), structured outputs for verifiers and the review protocol where supported. Each behind a profile flag with a rig A/B.

L7 — Routing by task class

What. Deterministic routing hints by task class (K6), recorded per decision so `overseer why` can explain the tier choice. No learned router.

L8 — Deprecation watch

What. A model scheduled for retirement mid-run is a reliability problem. Profiles carry known retirement dates when providers publish them; the supervisor warns before starting a long run on a model retiring within its expected duration, and the failover ladder handles the switch if it happens.

16. Observability

16.1 Where we stand

Every event is in the log; every provider call has a ledger row with cache fields; `RunEnd` carries a cache delta; `overseer stats` summarizes one session; the TUI shows a quiet run line (`19 steps · 1m 12s · $0.042 · 87% cached`) and a stats panel. There is no cross-session view, no trace structure, no timeline, and no alerting beyond the cache alert.

16.2 Workstreams

B1 — Traces from events

What. Turn the flat event stream into a tree of spans: `run` → `step` → `model call / tool call` → sub-calls (`run_code`, `tools op=call`) → subagents, with durations, tokens, cost and outcomes. Derived from events (no new instrumentation in the hot path beyond timestamps already recorded), so it works on old logs.

B2 — Global rollups

What. One local index over all sessions, workflow instances and ledgers: spend by day, model, project, purpose and tier; cache hit rate by model; tool latency percentiles; failure classes over time; run outcomes. Rebuildable from the logs (a cache, per the principles). Feeds R10's global budgets, K9's dashboard and F5.

B3 — The run timeline

What. A visual timeline per run in the web UI (and a compact form in the TUI): steps, waits (by kind), retries, failovers, compactions, approvals, milestones, restarts. The single most useful view for understanding a 100-hour run.

B4 — Forecasting

What. For a running long run: projected total cost and remaining time from plan progress and recent rates, with confidence bands; warnings when the forecast exceeds budgets (R10).

B5 — Alerts

What. Owner-configured alert rules over B2's metrics (cache hit rate below threshold, failure class spikes, budget forecasts, stalled runs, failed workflows), delivered through the notifier and channels with deduplication, and through the digest.

B6 — Export

What. Optional export of traces and metrics in OpenTelemetry format to a local collector, for owners who want their own dashboards. Off by default; never includes tool result bodies or secrets.

B7 — Replay debugger

What. Step through any session's events and see the exact view the model received at each request (from the context planner's deterministic reconstruction, C7), the verdicts, and the outputs. Because views are deterministic functions of the log, this is exact, not approximate.

B8 — Long-run health panel

What. For each active run: heartbeat age, state, step rate, spend rate against budget, cache hit rate, last milestone age, in-doubt effects, pending approvals, soak metrics (RSS, descriptors). The cockpit's core (X1).

17. Evaluation and Proof

17.1 Why this chapter decides everything

The playbook’s governing rule: build the evaluation rig before the agent, and beat the minimal loop measurably, per component, without regressing leanness [src: **playbook §12.1**]. The harness is a first-class variable worth roughly a model generation (2–9 point swings on Terminal-Bench 2.1 between harnesses on the same model, and far larger in some sweeps) [src: **playbook thesis 1**]. Overseer built the rig early; it has never been pointed at live models because of the eval hold. Every “better” claim in this roadmap waits on that.

17.2 Where we stand

The rig (Chapter 3.11): task spec v2 with mandatory oracles, deterministic graders, a k-seed scheduler, an immutable store, paired bootstrap and McNemar statistics, report cards, a frozen bash-only control scaffold; 40 public and 5 canary held-out tasks; adapters for SWE-bench Verified, Terminal-Bench 2.x, SWE-rebench, τ^2 and LiveCodeBench. Zero spend so far.

17.3 Workstreams

E1 — Lift the hold: the first measured baseline

What. With the owner’s go-ahead and a spend cap, run Overseer and the control scaffold on the local corpus and a small SWE-bench Verified slice, on one priced model and the free opencode model, k seeds each. This produces the W0 baseline every later wave is measured against.

DECISION

The eval hold stays until the owner lifts it. Everything in this chapter that spends money is gated on that decision (Chapter 26).

E2 — The long-horizon suite

What. Tasks that take hours to days: multi-module migrations on pinned open-source repositories, multi-step research with verifiable citations, and the five journeys of Chapter 1 as scripted scenarios with oracles. Measured: completion, unattended completion, cost, wall time, owner interventions, effects executed exactly once.

Solvability is decided before scoring, never after. Each task declares, before any scored run, the evidence that it is solvable: an oracle that passes its grader (the rig’s existing rule for every task), or a reference run that completed within twice the scored budget. A task without that evidence is excluded from the denominator up front. A scored failure cannot reclassify a task as unsolvable; disputes are settled by a second seeded reference run, and the outcome is recorded with the task’s version.

Cost control. Long tasks are expensive. The suite runs on the cheapest capable model by default (the free opencode model where possible), with a small priced subset; runs are local, one at a time.

E3 — The chaos suite

What. The long-horizon suite again, under R15’s fault injection: provider errors at several rates, crashes at random points, disk-full, clock jumps, simulated reboots, MCP deaths. Pass criterion: same outcomes as the fault-free run, no duplicate effects, bounded extra cost.

E4 — Cost per solved task, published

What. Every report card carries cost per solved task with the cache breakdown, against the control and against competitors on the same model.

E5 — Regression gates per wave

What. Each wave closes only when the rig shows no regression on pass rate (non-inferiority margin 3 points), cost per solved task, resident tokens, binary size and startup, against the previous wave's baseline, with paired statistics.

E6 — Memory evaluation

What. M9's suites: LongMemEval-S adapter, repeat-question rate, correction persistence, poisoning resistance, and the warm-versus-cold uplift test for outcome memory (M5).

E7 — Orchestration evaluation

What. Tasks labelled parallelizable and not; measure pass rate and cost per solved task with and without the task graph, merge queue and best-of-N, so the lead's prompt orchestrates only where it pays.

E8 — Competitors on the same model

What. Run competitor harnesses (with their recommended settings, recorded) on the same tasks and model through the rig's adapters (Harbor already hosts several agents), so "crush" is a measured, reproducible claim rather than a slogan.

Equivalence policy, published before any run. Same model and version wherever the competitor allows model selection. Where it does not (subscription-only products, fixed routing), the closest equivalent is used and the mapping is published next to the result. A harness that cannot be run fairly is listed as "not comparable" with the reason, never dropped silently. Settings, versions and dates are part of the report card.

E9 — Contamination hygiene

What. The held-out suite is repo-visible and therefore a canary check, not a true holdout (the rig's README says so). Add a private holdout kept outside the repository, rotate tasks, and keep the canary audit on every stored run. The holdout is authored early (W2), long before W9 consumes it, and each task records its authoring date: a holdout written after its material was public would be suspect.

17.4 What could break**WHAT COULD BREAK**

Evaluation costs more than the owner can spend. Mitigation: free models where possible, small priced subsets, local runs one at a time, and the spend gate on every run.

WHAT COULD BREAK

Overfitting to our own suite. Mitigation: external benchmarks alongside the local corpus, the private holdout, and competitors on the same tasks.

18. Frontends and Experience

18.1 Where we stand

A full-screen TUI with transcript scrolling, an inline mode, a plain-text REPL, and a hardened localhost web UI that streams the same surface; typed permission previews; rewind, diff with hunk-level rejection, fork tree, session picker, search; OSC notifications, prompt marks and file links; themes and reduced motion **[have]**. All of it assumes a run lives inside the frontend that started it.

18.2 Principles from the playbook

Cell-diffed rendering, synchronized output probed rather than guessed, a frame budget under 16.7 ms, O(1) per-token append, scrollbar preservation, accurate interrupt affordance, contextual permission prompts, checkpoints and rewind, queued and editable messages, and a mode badge **[src: [playbook thesis 9](#)]**. Overseer already follows these. The new work is about long runs, many runs, and being away.

18.3 Workstreams

X1 — The long-run cockpit

What. One screen (TUI and web) for everything running: runs and workflow instances with state, health (B8), spend against budget, forecast, last milestone, next step, pending approvals and in-doubt effects; drill into a run's timeline (B3), journal, agent graph (O14) and transcript. Attaching to a detached run (R4) opens it here.

X2 — The inbox frontend

What. A7's inbox in the TUI and web UI: items grouped by urgency and source, with typed actions, previews, snooze, and receipts. The same items appear on Telegram and later in the Life app.

X3 — ACP and the editor bridge

What. ACP has become the editor protocol (about 35 agents, Zed and JetBrains as clients) **[src]**. A thin ACP adapter crate over `overseer-proto`, sync threads, no tokio, so Overseer runs inside editors; plus app-server parity for other clients.

X4 — Approvals and status from the phone

What. Telegram first (the channel exists): approve or reject with the preview, see run status, pause, resume, kill. Messages carry signed, expiring action tokens so a forwarded message cannot approve anything. Later the Life iPhone app takes over (F).

X5 — Notifications that respect attention

What. The EV gate (have) decides whether to interrupt; notifications are batched, deduplicated, quiet-hours aware (with real time zones, A5), and every notification links to the item it is about.

X6 — Steering long runs

What. Steering today is a queue checked at tool-launch boundaries. For long runs: steer messages land in the journal as owner notes (R5) so they survive compaction and resume; "change of plan" steers trigger a plan revision; steers to a specific graph node or persistent agent go to its mailbox (O17).

X7 — Web UI parity

What. Every TUI capability in the web UI, plus the views that need a canvas: timeline, graph, memory panel (M11), permissions page (P12), workflow editor with validation (A1).

X8 — Onboarding

What. The playbook’s OpenCrane-style onboarding: a short interview → a draft persona and policy (consent mode, sensitive-surface permissions P6, budgets) → owner approval → each insight traceable to the answer that produced it.

X9 — Accessibility

What. Screen-reader-friendly output in the plain REPL (exists), high-contrast and mono themes (exist), keyboard-complete web UI, and ARIA roles in the web renderer.

X10 — Documentation as part of the product

What. Every workstream that changes behaviour updates the user-facing documentation in the same PR: the README, `overseer --help` and per-command help, a reference generated from the code for configuration keys, event kinds and policy syntax, and a short guide per major capability (long runs, workflows, policy, orchestration). `AGENTS.md` stays the contributor reference; decision records stay the “why”.

GATE

A doc check in the local gate: every configuration key, event kind and CLI command present in the code appears in the generated reference.

X11 — Importing from other harnesses

What. Switching costs keep people on incumbents. `overseer import` brings readable state across: Claude Code and OpenCode session histories as read-only, replayable sessions (indexed by history search, M1), their memory and instruction files (`CLAUDE.md` is already covered by P1’s compatibility imports), and their permission rules translated into rules v2 (P3) for review before activation. Imports are untrusted content until the owner approves them, and nothing is ever written back to the other tool’s files (the standing Life rule).

NOTE

Voice and other frontends named in the playbook’s Alfred diagram are not planned in the harness. They belong to Overseer Life’s later phases, which consume the harness through the SDK (Y4).

19. Extensibility and Ecosystem

19.1 Where we stand

Skills (metadata resident, bodies on demand), MCP servers with trust lanes, `hooks.json` data rules, microagents, six built-in modes [have]. No plugin model, no versioning or trust tiers for extensions, no project MCP configuration, no SDK.

19.2 Workstreams

Y1 — A plugin model

What. One packaging format that bundles skills, modes, contextual rules, hooks, MCP server declarations, workflow templates and persistent agent definitions, so a capability ships as one unit.

Design. A directory with a manifest (`plugin.toml`: name, version, author, declared capabilities and the permissions they need), installed under `~/.overseer/plugins/`. Plugins contribute through the policy compiler (P2), so precedence, provenance and explanations apply; a plugin cannot loosen owner policy. No plugin executes code in the harness process: code runs only as sandboxed hook scripts or MCP servers.

Y2 — Versioning and trust tiers

What. Every extension (skill, plugin, MCP server, hook) has a version and a trust tier: `builtin`, `owner` (authored or explicitly trusted by the owner), `project` (hash-pinned trust per repository), `untrusted` (content only, never instructions). Learned skills (M2) have their own lifecycle and ledger.

Y3 — Project-level MCP

What. T5's project `mcp.json` with hash-pinned approval.

Y4 — An embedding SDK and peer agents

What. A stable library surface for running Overseer's engine inside other programs (Overseer Life is the first consumer), with the wire protocol for out-of-process clients; peer-runtime subagents over ACP so Overseer can delegate to, or be delegated by, other harnesses [src: **DeepSeek harness**].

Y5 — The hooks protocol

What. P10's typed events and JSON protocol, documented and versioned so third-party hooks keep working across releases.

Y6 — Sharing

What. Publishing and installing plugins from git URLs with hash pinning and a review screen showing every capability and permission requested. No central marketplace until the owner decides the product phase has begun.

20. Runtime, Performance and Distribution

20.1 Where we stand

Core has no tokio and no reqwest; it uses `ureq` with `rustls` and `regex-lite`; the binary is about 5.5 MB; QuickJS added 1.26 MB under a 1.5 MB gate [[src: pillar refresh, tool economy record](#)]. `cargo-dist` is configured but installers are off; the `release.yml` plan job fails on every PR. `overseer-core --no-default-features` and `overseer-life` compile for the iOS simulator; QuickJS is the only blocker there [[src: Life Phase 0 results](#)].

20.2 Workstreams

Z1 — Budgets enforced by the build

What. Invariant 22. A local gate script (no Actions spend) that fails when any budget is exceeded:

Budget	Limit
Release binary size (all default features)	≤ 15 MB
Cold start to first prompt render (TUI, M1)	≤ 50 ms
Idle RSS of the TUI with no session	≤ 30 MB
Run process RSS after rejuvenation (R17)	≤ 150 MB at steady state
Resident startup tokens	≤ 2,000
Static prompt base	≤ 1,000 characters (existing pin)
Dependency count growth per wave	Reported; any new dependency needs a decision record line

The startup, RSS and binary limits are targets until W0 measures today's values on the owner's M1. Where a measurement already exceeds a target, the W1 decision record sets the path to it rather than silently relaxing the number.

Z2 — Signed and notarized builds

What. Developer ID signing and notarization for macOS (owner's Apple Developer account), stable signing identity so privacy grants survive updates, hardened runtime.

Z3 — Installers and safe self-update

What. Shell and Homebrew installers from `cargo-dist`; `overseer` update that installs side by side, keeps the previous version for draining runs (R13), and never replaces the binary under a running process. Setting `pr-run-mode = "skip"` in `dist-workspace.toml` stops the failing PR job (owner decision pending, Chapter 26).

Z4 — iOS gating

What. A `host-tools` feature in core that removes process spawning, sandboxing and bash for iOS builds; `code-mode` off on iOS; Life's `security/sqlite3` paths gated; memory falls back to plain files without git. This is the engine half of the Life iPhone app (F).

Z5 — Linux parity

What. Every feature at parity on Linux: `systemd` user unit for the supervisor, inotify triggers, Landlock and `seccomp` (S8), notification backends, power and sleep handling where meaningful.

Z6 — Windows plan

What. Deferred by the owner's ordering (Life decision: Windows later). The plan records the blockers so nothing built now makes Windows harder: Unix-only pieces (gateway control socket, flock, sandbox-exec/bwrap) sit behind traits; path handling avoids Unix assumptions in new code.

Z7 — Soak and leak testing

What. R17's soak runs as a release gate: 24 hours before every dev release, 100 hours before every main release, on the owner's Mac, local only.

Z8 — Move the repository out of iCloud

What. The checkout lives in iCloud-synced Desktop with an 8.8 GB target/; an earlier .git/index 3 conflict came from that. Moving it (owner action) removes a class of corruption risk that a 100-hour run writing logs and worktrees would make much worse.

Z9 — Licences and attribution

What. Overseer ports ideas and code from MIT-licensed projects (Synara's usage readers, Hermes' review prompts and threat patterns, cua-driver) with notices. Every workstream that ports code records the source, commit and licence in the file header and the NOTICE file; cargo deny checks dependency licences (have); a checklist item in every PR review confirms attribution for ported material.

21. Overseer Life: How the Harness Carries the Dashboard

Overseer Life (plan: docs/research/2026-10-05-digital-life.md) is the digital life dashboard on top of the harness. Its later phases depend directly on pillars in this roadmap; building them in the harness first means Life inherits them instead of reinventing them.

21.1 Dependency map

Life phase (from its plan)	Needs from this roadmap
Phase 1: foundation, browser v1, connector SDK	Embedding SDK (Y4), iOS gating (Z4), policy compiler for connector permissions (P2), egress proxy patterns (S1)
Phase 2: AI accounts hub	Global spend and cache rollups (K9, B2) feeding the module; multi-key model layer (L4) for “best account right now” routing
Phase 3: socials hub	Durable scheduled workflows for counts and follower scans with spend caps (A2, A5, R10)
Phase 4: browser v2 (agent sidebar)	Origin-scoped grants (P4), taint through browser content (S3), computer-use consent (U2)
Phase 5: subscriptions and money	Money-class policy (P4, P6), approval workflows with compensation (A4), effect journal (R7)
Phase 6: automations, agents, computer use	Workflow engine (A), persistent specialist agents (O7), supervisor (R4), computer-use live validation and consent (U1, U2)
Phase 7: iPhone	Mobile approvals (X4), inbox (A7), sync of the event log (Life’s own sync design)
Phase 8: identity and security module	Sensitive-surface permissions (P6), audit trail (S4), red-team programme (S9)

21.2 Workstreams

F1 — Engine APIs for Life

What. The SDK surface (Y4) that Life’s UniFFI layer binds to: start and attach runs, stream events, answer approvals, list workflows and instances, query rollups.

F2 — Specialist agents on persistent-agent infrastructure

What. Life’s accountant, security officer, social manager and subscription canceller are persistent agents (O7) with their own memory scopes and permission profiles, defined as plugins (Y1).

F3 — Life automations as workflows

What. Life’s “triggers → actions with approvals” (its `life-auto` crate in the plan) is the workflow engine (A), not a second engine.

F4 — One permissions model

What. Life’s Permissions page (scoped, expiring, revocable grants; money, security and deletion always ask outside full automation) is P4 and P12’s model rendered natively.

F5 — Telemetry into the AI accounts module

What. Overseer’s own usage (B2) appears in Life alongside the provider usage readers already built in Phase 0, so the owner sees harness spend and plan quotas in one place.

NOTE

Life's own open owner items (X live spike, bank and email providers, Apple Developer enrollment) are listed in Chapter 26; they gate Life phases, not harness waves.



Part IV — Execution

22. What Could Break: The Failure Atlas

The owner asked: “get into this zone of the future, what could potentially break?” This chapter answers systematically. It lists every failure mode found while writing this roadmap, grouped by where it originates, with its likelihood over a 100-hour run (or over a year of use, for slow risks), its impact, and the workstream that handles it. Then it runs three premortems: stories of how a 100-hour run, a standing automation, and this roadmap itself could fail, written as if they already had.

Likelihood: **H** (expected), **M** (plausible), **L** (rare but real). Impact: **H** (data loss, money, security, or a dead run), **M** (wasted time or money, recoverable), **L** (annoyance).

22.1 Providers and models

Failure	L	I	Handled by
Rate limits (429) during a long run	H	H→L	R1, R2, R10
Provider overload (529) for minutes to hours	H	H→L	R2, L4
Network blips, DNS failures, TLS resets	H	H→L	R2, R12
Stream cut mid-response	M	M→L	R2 (discard partial, retry)
Provider returns 200 with an error body	M	M	R1 (body classification)
Misused status codes (400 meaning overload)	M	M	R1 (per-adapter tables with fixtures)
Key revoked, expired or out of credit	M	H	R1 <code>Auth</code> → park with owner action
Model retired mid-run	L	H	L8, L4
Model behaviour changes silently under the same name	M	M	E5 regression gates; L3 probes per binary version
Price change makes budgets wrong	M	M	L2 <code>verified_on</code> dates; K12 ledger reconciliation
Cache rules change (TTL, minimum prefix, breakpoint semantics)	M	M	K10 forensics; cache alert; L2
Reasoning round-trip rules change	L	H	C4 per-adapter declarations with fixture tests; L3 probes
Provider-side context features conflict with invariants	L	M	K11 evaluation before adoption
A cheaper failover model silently lowers quality	M	M	L4 tier rules; journal notes; O9 goal audits
Token estimate drifts, causing late compaction and context errors	M	M	K4 self-calibration; existing <code>ContextWindowExceeded</code> path

22.2 Processes and the machine

Failure	L	I	Handled by
Run process crashes (panic, OOM, signal)	M	H→L	R4 auto-resume
Laptop reboots (OS update, power loss)	M	H→L	R4 re-adoption at login
Laptop sleeps, lid closes	H	M→L	R12
Battery runs flat during a run	M	M	R12 battery policy, R4 resume
Daemon crashes	M	M	launchd/systemd restart; runs continue independently

Failure	L	I	Handled by
Two daemons start	L	M	Daemon lock
A step hangs (tool or provider never returns)	M	H	R3 deadlines; R4 hung verdict
Grandchild processes hold pipes open	M	H	[fix2] process-group kill
Zombie and orphan processes accumulate	M	M	R17 reaping and soak checks
Memory leak over days	M	M	R17 rejuvenation
File descriptor leak	M	M	R17
CPU saturation from parallel builds	H	M	O18 scheduler
Disk fills with logs, spills and worktrees	M	H	R11
The repo sits in iCloud sync, which corrupts git state	M	H	Z8 (owner action)
An Overseer upgrade breaks in-flight runs	M	H	R13, Z3
The machine's clock jumps (NTP correction, manual change)	L	M	R12 monotonic timing

22.3 Storage and state

Failure	L	I	Handled by
Torn write at the end of the event log	M	L	Have (torn-tail repair)
Corruption earlier in the log	L	H	Have (error names the line); R6 audit; segment manifests (R11)
A snapshot disagrees with the log	L	H	R11 (snapshot covers a hash; fall back to full replay)
Store written without fsync, lost on power loss	M	H	R6
Concurrent writers to a memory store	L	M	Have (flock, drift check)
Memory store lock contention once many supervised runs share a store (10 s acquire, then BUSY)	M	M	R1 classifies BUSY as transient and retries; learning reviews of concurrent runs are serialized by the supervisor
Two top-level runs write the same working tree	M	H	R16 workspace lease
Sidecar says running for a dead task	M	M	Have (nonce reaping)
Worktree deleted under a writer	L	M	R16; merge queue parks
Old logs fail to replay after a schema change	M	H	R13 golden corpus
Workflow definition changed under running instances	M	M	A9 version pinning

22.4 Side effects

Failure	L	I	Handled by
Auto-resume repeats a <code>bash</code> command with effects	M	H	R7 in-doubt notice and verify-before-repeat
Auto-resume resends a message or webhook	M	H	R7 never auto-replays external; outbox ids
A retried workflow step double-spends money	L	H	R7, A2 idempotency keys; money always parked
Compensation fails after a partial workflow	M	M	A4 retried, journaled compensation; irreversible steps flagged
A writer's merge lands and breaks the build	M	M	O6 verify after every landing
Rewind restores files but not external effects	H	M	Invariant 9 blind spot documented; R7 records what it cannot undo

22.5 Model behaviour

Failure	L	I	Handled by
Wrong diagnosis before the first edit	H	M	Verification (O10, verify gate); playbook evidence says this dominates failures
Futile loops late in a run	H	M	Have (stuck detector); R9 long-window stall detection
Drift from the goal while looking busy	M	H	R5 journal; R9; O9 goal audits
Premature “done”	M	H	Mandatory verification; unknown is never pass
Claims milestones without evidence	M	M	R5 evidence-checked milestones
Ignores AGENTS.md conventions	M	M	P1 loader; contextual rules (P9); memory
Narrates verbosely, wasting output tokens	H	L	K8
Assumes a pending approval succeeded	M	M	R8 wording, gate on dependent actions
Misreads a resume briefing as new instructions	L	M	Provenance wrapping; briefing format tested on the rig

22.6 Orchestration

Failure	L	I	Handled by
Bad brief produces useless subagent output	H	M	O13 brief linter; O8 brief schema
Subagents duplicate each other’s work	M	L	O4 blackboard claims
Token blow-up from over-orchestration	M	M	Graph budgets; E7 tuning
Merge-queue conflicts loop	M	M	O6 re-plan notice
Unbounded agent tree depth	L	M	O1
A failed node hidden in a “done” summary	M	H	O-design: run cannot complete with failed nodes
Tainted subagent poisons artifacts or blackboard	M	H	O3, O4 taint

22.7 Memory

Failure	L	I	Handled by
Poisoned note from untrusted content	M	H	Have (quarantine, threat scan); M10 for new paths
Stale note recalled with authority	M	M	Confidence, validity, fading (M3), FEEDBACK
Outcome memory learns superstition	M	M	M5 gate on measured uplift
Memory growth slows search	L	L	Have (bounded index); M3
Memory content breaks the cache mid-session	L	M	Have (resident memory frozen per agent)

22.8 Policy and security

Failure	L	I	Handled by
Malicious repository AGENTS.md, MCP file, hooks or skills	M	H	P1, T5, P10 hash-pinned trust
Prompt injection read late in a long run	M	H	Latches persist; invariant 19; S6 corpus
Exfiltration through an allowed domain	L	H	S1 proxy policies under taint
Approval fatigue pushes the owner to full automation everywhere	M	M	P4 scoped grants; per-action prompt rate target
Full automation performs a harmful action	L	H	Owner’s decision; P6 permissions; S4 audit; S10 kill switch
Forwarded Telegram approval message replayed	L	H	X4 signed, expiring action tokens

Failure	L	I	Handled by
A secret pasted by the owner lands in logs	M	H	S7 scanning and redaction; key rotation
Confused deputy through the daemon	L	H	S3: daemon thin, credential-free, trust levels propagate
sandbox-exec deprecated by Apple	L	H	S8 contingency

22.9 Automation

Failure	L	I	Handled by
Scheduled workflow silently stops running	M	H	A12 dead-man's switch; digest health line
Daylight saving causes double or missed runs	M	M	A5
A workflow spams channels	L	M	Outbox caps; budgets; kill switch
Webhook flood	M	M	Have (rate limits, replay table)
Long sleep causes a backlog of catch-up runs	M	M	A5 missed-run policy

22.10 People and process

Failure	L	I	Handled by
The roadmap is too large to finish	H	M	Waves are independently valuable; W1–W3 remove most failure classes
Cloud quota exhausted by parallel sessions (it happened)	M	M	Standing rule: one session at a time, local first
GitHub Actions minutes spent by accident	M	L	[skip actions] rule; Z3 pr-run-mode
Review rigour drops under schedule pressure	M	H	Standing review rule; S9 wave-close red team
Key decisions wait on the owner and block waves	M	M	Chapter 26 lists every decision with its blocking wave
Knowledge lives only in chat sessions	M	M	Decision records, LEAVING-OFF notes, this roadmap

22.11 The longer future

Risks that grow over years rather than days.

Trend	Risk to Overseer	Posture
Context windows keep growing and get cheaper	Context management becomes less critical; complexity in C rots	Every C workstream is gated on measured benefit; the planner keeps policies swappable
Models get much better at long tasks	Scaffolding encodes outdated weaknesses (playbook thesis 2)	Re-measure every component against the minimal loop each wave (E5); remove what no longer pays
Providers ship their own agent runtimes and memory	Competition from the model vendors themselves	Local-first, multi-provider, owner-controlled policy and memory are the differentiators
Agent protocols consolidate (ACP, MCP successors)	Integrations go stale	Thin adapters (X3, Y4); protocol code isolated
OS vendors restrict accessibility and sandbox APIs	Computer use and sandboxing break	Separate signed helper (U6), sandbox contingency (S8)
Regulation of autonomous agents	Full automation may need auditability by law	S4's audit trail and the event log already provide it
Terms of service tighten on undocumented endpoints	Life usage readers break	Life plan's connector health and per-source opt-in

22.12 Premortem one: the 100-hour run that failed

Written as if it happened. The migration ran for 61 hours and then stopped making progress. Looking back:

1. On hour 14, a writer ran a database migration script against a local development database through `bash`. The process crashed during the command. On resume, the agent saw the in-doubt notice but the verification it chose (listing tables) looked fine, so it re-ran the script, which applied a second, partially conflicting migration. **Lesson:** R7's verify-before-repeat is only as good as the verification; the policy compiler should let AGENTS.md declare commands that are never auto-repeatable, and the in-doubt notice should name the risk class.
2. On hour 30, a provider failover to another family happened mid-exchange because the ladder did not wait for a handoff boundary. Reasoning blocks were dropped from the view, and the model lost the thread of a subtle refactor. **Lesson:** L4 requires cross-family failover only at a handoff boundary; this is now in L4's design.
3. From hour 40, the plan in the journal kept saying "doing #10" while diffs oscillated in the same three files. The stall detector's window was sized for the plan's total length, so it did not fire until hour 58. **Lesson:** R9 windows must scale with per-item history, not plan size.
4. At hour 61 the rate budget paused the run overnight, and the owner, seeing "paused", assumed it had finished. **Lesson:** paused states need a distinct notification and a forecast of when they resume.

Each lesson is folded into the named workstream.

22.13 Premortem two: the automation that died quietly

Written as if it happened. The morning digest ran perfectly for five weeks, then stopped. Nobody noticed for nine days.

1. A Life connector started returning an authorization error after the owner logged out of one tool. The step failed, retried, and the instance ended `failed`.
2. The failure produced an inbox item, but the inbox item was in a digest that the failed workflow itself was supposed to send.
3. The dead-man's switch was configured for "no successful run in 48 hours" but the alert channel was the same Telegram bot whose token had rotated.

Lessons: failures of the digest workflow must alert through a channel independent of the digest (the notifier plus the cockpit badge); the dead-man's switch must have a fallback channel; channel credentials need health checks of their own (A12).

22.14 Premortem three: the roadmap that failed

Written as if it happened. A year later, half the waves were started and none were finished.

1. Waves ran in parallel to "save time", so no wave closed its gates and nothing was ever measured against a stable baseline.
2. The eval hold was never lifted, so every "improvement" was a guess, and some made things worse.
3. Big-bang branches for the policy compiler and the workflow engine grew for months, became impossible to review line by line, and were abandoned.
4. Leanness budgets were relaxed "temporarily" and never restored.

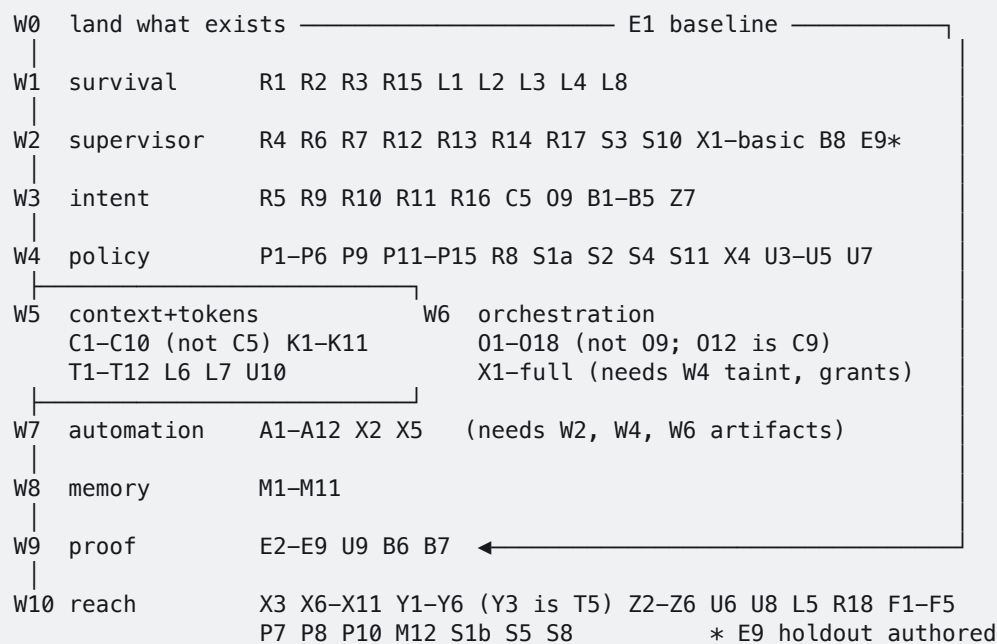
Lessons: one wave at a time with explicit gates (Chapter 23); E1 is an early owner decision (Chapter 26); every workstream lands in reviewable slices behind behaviour-preserving refactors first (the C7 and P2 golden-test pattern); Z1 budgets are part of every wave's exit gate.

23. Sequencing: Waves, Dependencies and Gates

23.1 Rules of sequencing

1. **One wave in flight at a time.** A wave closes when its exit gate passes. Parallel waves are how roadmaps die (premortem three).
2. **Inside a wave, workstreams land in reviewable slices.** A refactor that preserves behaviour (proven by golden tests) lands before any behaviour change, as in C7 and P2.
3. **Every wave's exit gate includes the standing gates** (and every wave carries X10 documentation and Z9 attribution for what it changes): formatting, clippy with warnings denied, the full test suites, the build budgets (Z1), the golden log corpus (R13), and an independent adversarial review of the wave's delta (S9). Where the eval hold is lifted, it also includes E5's regression check; where it is not, the free model on the local corpus stands in.
4. **Owner decisions that block a wave are asked before the wave starts** (Chapter 26).
5. **Local first.** Everything runs on the owner's Mac. One cloud session at a time, only deliberately.

23.2 Dependency graph



W5 and W6 both depend on W4 and not on each other. By rule 1 they still run one after the other; the order is the owner's choice (Chapter 26), with W5 first recommended because its savings compound across everything after it.

23.3 The waves in detail

W0 — Land what exists

Contents. Integrate `fix2-memory` into `feat/memory-v3` with the owner's option (a) for `fix2-memory` finding M8; integrate all `fix` branches plus the `--bare` skills `fix` into `feat/fortify`; full local gates; adversarial review of the `fix` delta; push both PR branches and update their Deferred sections; merge #54, retarget and merge #55. Then the consent-modes branch: P5 and its computer-use application U2, as a first cut on today's `perm.rs`, migrated into the compiler in W4. Owner actions in parallel: rotate the 2026-09-20 key (S7), decide on the eval hold (E1), live computer-use validation with owner presence

(U1), the first priced live run (K12), move the repository out of iCloud (Z8), the `pr-run-mode` decision (Z3). Hygiene: correct `eval/README.md`, which still says 35 public tasks after the corpus grew to 40.

Exit gate. #54 and #55 merged into `review` with green local gates and a clean adversarial review; the `consent-modes` branch merged with tests for both modes; owner decisions recorded.

W1 — Survival

Contents. R1 failure taxonomy; L1 adapter classification tables; R2 retry and backoff with discarded partial streams; R3 every timeout hole; R15 fault injection; L2 living profiles; L3 capability probes; L4 failover ladders with multi-key support; L8 deprecation watch; the disk preflight part of R11.

Exit gate. Zero run ends from transient classes under injected faults at 1%, 10% and 50% rates; the R3 blocking-call lint passes; failover round-trip test passes; no reasoning block crosses providers.

W2 — The supervisor

Contents. R7 effect journal (lands before auto-resume is enabled); R4 supervisor with registry, heartbeats, verdicts, restart budget, reboot re-adoption and detached runs; R14 lifecycle and commands; R6 crash-consistency audit; R12 time, sleep and power; R13 versioning and the golden log corpus; R17 process hygiene and rejuvenation; S10 kill switch everywhere; S3 threat model update; X1 basic cockpit (runs list, attach); B8 health panel; authoring of the private evaluation holdout (E9) so it seasons before W9 uses it.

Exit gate. The R4 chaos gates (200 random kills, daemon kill, simulated reboot, crash loop); zero duplicate effects across 1,000 injected crashes; kill switch latency targets from every frontend; a 24-hour soak (Z7).

W3 — Durable intent

Contents. R5 journal, plan tool promotion and resume briefings; R9 progress ledger and stall detection; R10 rate budgets, shared key buckets and forecasting; the rest of R11 (snapshots, log segments, spill and worktree collection); R16 environment drift; C5 edge placement; O9 goal audits; B1 traces, B2 rollups, B3 timeline, B4 forecasting, B5 alerts.

Exit gate. The 40-compaction briefing test; snapshot resume ≤ 2 s on a synthetic 100-hour log; rate budget pause and resume; the first 100-hour soak run on the owner's Mac completes unattended.

W4 — Policy

Contents. P1 AGENTS.md loader; P2 compiler with behaviour-preserving migration; P3 rules v2; P4 scoped grants; P5 consent modes migrated into the compiler; P6 sensitive-surface permissions; P9 contextual rules; P11 explanations; P12 permissions management; P13 build-enforced principles; P14 instruction hierarchy; P15 git safety defaults; R8 away policy with non-blocking asks; S1a egress proxy on macOS; S2 credential broker completion; S4 full-automation audit trail; S11 data at rest, retention and backup; X4 phone approvals; U3 progress guard, U4 settle waits, U5 clipboard, U7 revocation on lock.

Exit gate. Golden tests show no behaviour change from the compiler migration; no module reads raw policy sources; untrusted AGENTS.md never lands above the cache boundary; away-policy and egress gates pass.

W5 — Context and tokens

Contents. C1–C10 except C5, which landed in W3 (C7 planner first, behaviour-preserving); K1–K11; T1 hashline edits (measured); T2 `run_code` reach and U10 GUI scripts (after U1); T3 persistent QuickJS context (if the rig shows need); T4 native deferral (measured); T8 parallel reads; T9 shaped results; T10

web tool (after S1a; on Linux after S1b); T11 latency budgets; T12 description tests; T5 (which is also Y3), T6, T7; L6 native provider features and L7 routing by task class.

Exit gate. Cost per solved task down at least 30% from the W0 baseline with pass-rate non-inferiority; cache hit rate at or above target; resident tokens at or below 2,000.

W6 — Orchestration

Contents. O1–O18 except O9 (landed in W3) and O12 (the same work as C9, landed in W5), with O6 merge queue and O2 graph first; X1 full cockpit with the agent graph (O14).

Exit gate. The O2 and O6 gates; fork subagent cache share at least 90%; E7 shows orchestration pays where it is used.

W7 — Automation

Contents. A1–A12; X2 inbox frontend; X5 notifications.

Exit gate. The A2 crash gates; A5 time-zone tests; the workflow library templates pass their own tests; a 30-day run of the morning digest and memory dream workflows on the owner’s machine with zero silent failures.

W8 — Memory

Contents. M1–M11 (M12 waits for W10).

Exit gate. The memory v3 record’s verification lists for H2 and H3; M9’s suites; M5 shows measured uplift or stays off.

W9 — Proof

Contents. E2 long-horizon suite; E3 chaos suite; E4 published cost per solved task; E6, E7, E8 competitors on the same model; E9 private holdout; U9 computer-use evaluation; B6 export; B7 replay debugger.

Exit gate. A published report card with paired statistics against competitors on the same model, the long-horizon and chaos results, and every commitment in the executive summary either met or explicitly reported as not yet met.

W10 — Reach

Contents. X3 ACP adapter; X6–X11; Y1–Y6 except Y3 (landed as T5 in W5); Z2 signing and Z3 installers (Apple account); Z4 iOS gating; Z5 Linux parity; Z6 Windows plan; L5 local models; U6 Overseer’s computer helper (Apple account); U8 browser protocol; R18 remote executor; F1–F5 Life integration; P7 user modes, P8 stream rules, P10 hooks v2; M12 shared memory; S1b Linux egress transport; S5 signed provenance; S8 sandbox hardening.

Exit gate. Per workstream; W10 is a portfolio, and its items can close independently once W9 is done.

23.4 Coverage matrix: every gap has an owner

Every gap found in the baseline audit (Chapter 3) and every open item in `docs/LEAVING-OFF.md` maps to a workstream.

Gap (Chapter 3 reference)	Workstream	Wave
Provider errors end the run; <code>retry_after</code> unused (3.1)	R1, R2, L1	W1
No automatic resume or supervisor (3.2)	R4	W2
No wall-clock budget (3.3)	R3	W1
Ask dialog has no deadline (3.3)	R8	W4

Gap (Chapter 3 reference)	Workstream	Wave
Notifier and Telegram block the daemon tick (3.3)	R3	W1
Gateway runs killed at 600 s (3.3)	R3, R4	W1–W2
Provider HTTP single 600 s timeout (3.3)	R3	W1
Count-based eviction (3.4)	C1, C7	W5
No real tokenizer (3.4)	K4	W5
Only three breakpoints, 5-minute TTL only (3.4)	K1, K2, K3	W5
No web tool (3.5)	T10, S1a	W4–W5
run_code cannot reach computer or memory (3.5)	T2	W5
No project MCP, cargo-only diagnostics, one-time availability (3.5)	T5, T6, T7	W5
No task DAG, artifacts, depth cap, persistent agents, detach (3.6)	O1–O7	W6
Memory v3 H2 and H3 (3.7)	M1–M4	W8
Rules bash-only, exact-match, append-only, no deny (3.8)	P3, P4, P12	W4
AGENTS.md not read (3.8)	P1	W4
No instruction compiler (3.8)	P2	W4
Single-shot gateway, no workflows (3.9)	A1–A12	W7
Fixed time-zone offset in the gateway (3.9)	A5	W7
No egress proxy (3.10)	S1a (macOS), S1b (Linux)	W4, W10
Broker rate and window not enforced (3.10)	S2	W4
No signed builds (3.10)	S5, Z2	W10
No live evaluation results (3.11)	E1, K12	W0
No long-horizon, chaos or memory suites (3.11)	E2, E3, E6	W9
Live computer-use validation (3.12)	U1	W0
Live priced run (3.12)	K12	W0
Orchestration wave B (3.12)	O2, O3, O4, O6	W6
Eval hold (3.12)	E1 (owner decision)	W0
Ignored fix2-agent reviewer tests (3.12)	W0 integration review	W0
release.yml plan job failing (3.12)	Z3 (owner decision)	W0
Provider key rotation (3.12)	S7 (owner action)	W0
Repository in iCloud (3.12)	Z8 (owner action)	W0
Bash background processes killed after the command (3.12)	Documented behaviour; long-running services belong to workflows or supervised runs, not cmd &	W0 note
Subagent worktrees accumulate (3.12)	R11 worktree collection	W3
No cross-session observability (3.4, 16.1)	B1–B5	W3
No user-defined modes (3.1)	P7	W10
Microagent substring triggers (3.1)	P9	W4
Hooks are data-only (3.8)	P10	W10
Consent modes not implemented (3.8)	P5, U2	W0, W4

23.5 Relative size of each wave

Sizes are relative engineering effort, not calendar time. How long a wave takes depends on review capacity, owner decisions and what the evaluation shows, so this roadmap deliberately gives no dates.

Wave	Relative size	Why
W0	S	The fixes exist and are reviewed; the work is integration, gates and review
W1	M	Contained in the provider layer and the agent loop; fault injection is the larger half
W2	L	New process model (supervisor, detached runs) plus the effect journal; heavy crash testing
W3	L	Journal, snapshots and log segments touch the event layer that everything depends on
W4	XL	The compiler migrates every policy source behaviour-preserving, then adds new semantics; egress proxy
W5	L	Many independent, measured changes; the planner refactor is the critical piece
W6	XL	Graph executor, artifacts, merge queue and persistent agents are each substantial
W7	XL	A durable workflow engine is a product in its own right
W8	M	The memory v3 record already specifies H2 and H3 in detail
W9	L	Mostly building suites and running them; cost is money and machine time more than code
W10	XL	A portfolio of independent items that close one by one

23.6 How progress is reported

At each wave close: a short report in `docs/research/` (what landed, gate evidence, deferred items with owners and gates, metric deltas), an update to `docs/LEAVING-OFF.md`, an updated scorecard (the executive summary table) in the roadmap's next edition, and the PR and merge commit Deferred sections the standing rule requires.

24. The Verification System

Verification is the highest-leverage harness investment for agents [src: [playbook thesis 6](#)], and it is just as decisive for building the harness. This chapter defines how every workstream proves itself.

24.1 The layers

Layer	What it catches	Where it runs	When
Unit tests	Logic errors in one module	<code>cargo test</code>	Every change
Property tests	State-machine and parser edge cases (R14 lifecycle, A1 parser, O2 graph validation, P2 precedence)	<code>cargo test</code> with seeded generators	Every change
Golden tests	Behaviour drift in refactors (C7 views, P2 verdicts and prompt bytes, R13 log replay)	<code>cargo test</code> against committed corpora	Every change
Fixture tests	Provider and driver contract drift (R1 error tables, cua-driver schemas, reasoning rules)	<code>cargo test</code> against captured fixtures	Every change; fixtures refreshed deliberately
Fault-injection tests	Reliability claims (R15)	Test builds with the fault layer	Every change touching R, A, O
Crash-consistency tests	Durable stores (R6)	Kill-at-every-step harness	Every change touching a store
Injection corpus	Security latches (S6)	<code>tests/injection_asr.rs</code> and extensions	Every change
Adversarial review	Everything the tests did not think of	An independent reviewer, locally	Every wave close and every PR (standing rule)
Rig evaluations	Capability, cost and regressions (E)	The eval rig	Wave close, and per workstream where a gate demands it
Soak tests	Leaks and slow failures (R17, Z7)	The owner's Mac	24 hours per <code>dev</code> release, 100 hours per <code>main</code> release
Live checks	Reality versus fakes (U1, K12)	The owner's Mac, owner present	Before dependent work ships

24.2 Rules

1. **A bug is first a failing test.** Where tests exist, reproduce before fixing (the fix2 cycle did this and found real issues).
2. **Fakes are verified against reality.** Every fake (provider, driver, channel) has a fixture captured from the real thing, and the fixture's capture date is recorded.
3. **No test is weakened to pass.** Changing an assertion requires stating why the old one was wrong, in the commit.
4. **Ignored tests carry a reason and a gate,** like deferred code.
5. **Evidence is reported, not asserted.** A gate is passed when its evidence (test output, report card, soak log) is attached to the wave report.
6. **The ETXTBSY lesson generalizes.** Any test that writes and executes a file, or depends on timing, uses the retry and deadline helpers; flakes are bugs.

24.3 The local gate script

Because GitHub Actions minutes are off the table, the gate is a local script that the reviewer runs on the owner's Mac before any push to a PR branch:

Step	Command
Format	<code>cargo fmt --all -- --check</code>
Lint	<code>cargo clippy --workspace --all-targets -- -D warnings</code>
Tests	<code>cargo test --workspace --no-fail-fast</code> (twice, to catch flakes)
Supply chain	<code>cargo deny check</code>
Budgets	Z1 size, startup, RSS and resident-token checks
Golden corpus	R13 replay of every committed log
Static lints	P13 checks (deferral markers, blocking calls, event skippability, store chokepoint, policy reads)
Red team	The release-mode red-team suite at <code>REDTEAM_ITERS=20000</code>
Eval (when the hold is lifted)	E5 regression check against the previous wave baseline

25. The Scoreboard

“Crush every other harness” needs numbers. These are the metrics that decide it, their targets, and how each is measured. Baselines are taken in W0 (E1); targets are checked at each wave close and published at W9.

25.1 North-star metrics

Metric	Definition	Target	Measured by
Unattended long-horizon completion	Long-horizon suite runs that complete without owner intervention, over tasks pre-declared solvable (E2’s rule)	$\geq 95\%$	E2
Chaos parity	Long-horizon outcomes under injected faults equal fault-free outcomes	100% of tasks pre-declared solvable	E3
Cost per solved task	Total spend $\div$ tasks solved, same model, same tasks	Lowest among measured harnesses; $\geq 30\%$ below Overseer’s W0 baseline	E4, E8
Pass rate	Paired, same model, 95% CI	Higher than every measured competitor, or non-inferior at lower cost	E1, E8
In-session cache hit rate	Cache-read input tokens $\div$ total input tokens, per session	$\geq 92\%$ Anthropic; $\geq 85\%$ OpenAI-compatible	B2
Duplicate external effects	Across all chaos runs	0	E3, R7 tests

25.2 Supporting metrics

Area	Metric	Target
Reliability	Runs ended by transient-class failures	0
Reliability	Time to automatic resume after a crash	≤ 60 s
Reliability	Resume from snapshot for a 100-hour log	≤ 2 s
Reliability	Owner interventions per 100 run-hours, excluding genuine approvals	≤ 1
Context	Input tokens per solved task	$\geq 25\%$ below W0
Tokens	Resident startup tokens	$\leq 2,000$
Tokens	Token estimate error after calibration	$< 3\%$
Tools	Wall time per multi-read turn	$\geq 30\%$ below W0
Orchestration	Merge-queue landings that later fail verify	0
Orchestration	Fork subagent cache-read share of prefix	$\geq 90\%$
Memory	Repeat-question rate on seeded facts	$\leq 5\%$
Memory	Correction persistence over 5 sessions	$\geq 95\%$
Policy	Approval prompts per 100 calls in per-action mode	≤ 2
Policy	Decisions with a complete explanation	100%
Security	Injection attack success rate per corpus category	No regression; trending down
Security	Kill-switch latency to every agent	≤ 2 s
Automation	Silent workflow failures	0
Automation	Scheduled completions within window over 30 days	$\geq 99\%$

Area	Metric	Target
Leanness	Release binary	≤ 15 MB
Leanness	Cold start to first render	≤ 50 ms
Leanness	Idle TUI RSS	≤ 30 MB

25.3 How a number becomes a claim

A metric is reported publicly only with its method: harness commit, model and version, seeds, task set and version, environment digest, and the confidence interval, in the rig's report card format. Competitor numbers are reported only from runs on the rig with their recommended settings recorded, never from vendor claims.

26. Decisions for the Owner, and the Risk Register

26.1 Decisions needed, in the order they block work

#	Decision	Options	Recommendation	Blocks
D1	Lift the eval hold for a capped baseline	Keep the hold; lift with a spend cap; lift for free models only	Lift with a small spend cap plus the free model; without it every “better” in this roadmap is a guess	E1 and every measured gate (W0 onward)
D2	First priced live run	Approve a capped run on Anthropic; defer	Approve; the spend gate has never seen real prices	K12 (W0)
D3	Live computer-use session with owner present	Schedule; defer	Schedule; nothing in computer use is proven live	U1 (W0)
D4	Rotate the provider key from 2026-09-20	Rotate now	Rotate now	S7 (W0)
D5	Move the repository out of iCloud-synced Desktop	Move; mark target/unsynced; leave	Move	Z8; risk before W2’s long runs
D6	Stop the failing PR release job	<code>pr-run-mode = "skip"</code> ; keep <code>[skip actions]</code> on every PR	<code>pr-run-mode = "skip"</code>	Z3 (W0)
D7	Default rate budgets for long runs	Owner sets per run (required); global defaults	Required per run the first time, then remembered per project	R10 (W3)
D8	Keep the machine awake during long runs on mains power	Never; opt-in; default on	Opt-in	R12 (W2)
D9	Away-policy default on timeout	<code>deny_once</code> ; park; wait	<code>deny_once</code>	R8 (W4)
D10	Trust model for third-party AGENTS.md	Ask once per content hash; always untrusted; always trusted	Ask once per hash	P1 (W4)
D11	Full-automation sensitive-surface defaults at onboarding	Owner chooses each category; offer a preset	Owner chooses, with a suggested preset shown	P6 (W4)
D12	Order of W5 and W6	Context and tokens first; orchestration first	Context and tokens first	W5/W6
D13	Workflow definition format	TOML; JSON (like gateway config today)	TOML for human-authored files	A1 (W7)
D14	Apple Developer enrollment	Enroll now; later	Enroll when convenient; it gates signing, the computer helper and Life’s browser entitlements	Z2, U6, S5 (W10); Life phases
D15	Remote executor	Build R18; skip	Build after W9, one session at a time	R18 (W10)
D16	Close PR #50	Close; keep	Close; fully contained in #51	Hygiene

Overseer Life’s own pending owner items stay as recorded in its plan: the X live spike (developer app, a few dollars of credit, owner present), bank and email providers for the money spike, and the Apple Developer account.

26.2 Risk register

The highest-impact risks from Chapter 22 with owners and triggers for review.

ID	Risk	Likelihood	Impact	Owner	Mitigation	Review trigger
K-1	Auto-resume repeats a side effect	M	H	engine	R7 before R4 auto-resume	Any in-doubt resolution bug
K-2	Long run drifts while looking busy	M	H	engine	R5, R9, O9	Goal audit drift verdicts above 5% of audits
K-3	Cost runaway on long runs	M	H	engine	R10 rate budgets, B4 forecasts	Any run exceeding forecast by 50%
K-4	Provider or model change mid-run	M	H	engine	L3, L4, L8	Provider announcements; E5 regressions
K-5	Prompt injection late in a run	M	H	security	Latches, invariant 19, S6	Corpus regression
K-6	Upgrade breaks in-flight runs	M	H	engine	R13, Z3	Any golden replay failure
K-7	Disk exhaustion	M	H	engine	R11	Free space alerts
K-8	Parallel writers conflict	M	M	engine	O6	Repeated conflict re-plans
K-9	Leanness erodes	H	M	engine	Z1 budgets in every gate	Any budget exceedance
K-10	The roadmap stalls	H	M	owner	One wave at a time; W1–W3 first	A wave open longer than its predecessor took twice over
K-11	Evaluation unaffordable	M	M	owner	Free models, small priced subsets	Spend cap reached
K-12	Full automation causes harm	L	H	owner	P6, S4, S10	Any highlighted effect in the audit digest
K-13	iCloud sync corrupts the repository	M	H	owner	Z8	Any conflict file under <code>.git</code>
K-14	Scheduled automation fails silently	M	H	automation	A12 dead-man's switches and independent channels	Any missed window



Appendices

Appendix A — New Event Kinds

Every new kind follows invariant 21: new fields default, unknown kinds are skippable by older readers, and each kind lands with a golden-log replay test.

Event	Introduced by	Rehydrated into the model view?
RunState { state, reason }	R14	No (audit and supervisor)
Failure { class, detail } (or a class field on existing failure events)	R1	No
Resumed { cause, after_sleep, binary_version }	R4, R12	Through the resume briefing only
Snapshot { covers_event, covers_hash, state }	R11	No (replay cache)
JournalGoal, JournalPlan, JournalNote, JournalMilestone, JournalNext	R5	Through the resume briefing and edge block (C5)
EffectIntent, EffectOutcome	R7	In-doubt notices only
ApprovalRequested, ApprovalResolved	R8	As pending results and resolution notices
Failover { from, to, reason }	L4	Journal note
GraphNode { id, state, task_id }	O2	Graph summary notice
ArtifactWritten { name, version, hash, taint }	O3	As inputs to consumers
BoardEntry { kind, text, taint }	O4	In briefs, capped
MailboxMessage { from, to, kind, reply_to, deadline }	O7, O17	Yes, at the loop top
Handoff { brief }	C9	Starts the new view
PolicyCompiled { fingerprint, sources, diff }	P2	No
ShadowVerdict { would_have, rule, class, taint }	S4	No
CacheMiss { cause, segment }	K10	No
ContextBreakdown (field on model requests)	C10	No
Workflow instance events: InstanceStarted, StepScheduled, StepStarted, StepCompleted, StepFailed, StepCompensated, InstanceCompleted	A2	Not part of any agent view

Appendix B — Configuration Surface

Reliability keys are listed in §5.5. All configuration flows through the policy compiler (P2) from these files:

File	Scope	Holds
<code>~/.overseer/policy.toml</code>	User	Rules, grants defaults, consent mode, sensitive-surface permissions, budgets, away policy, retry, power
<code><repo>/.overseer/policy.toml</code>	Project (trusted by hash)	Project rules within what the owner delegates
<code>~/.overseer/AGENTS.md</code> , <code><repo>/AGENTS.md</code> , nested <code>AGENTS.md</code>	User, project, directory	Instructions; optional frontmatter for verify and heavy commands, protected paths, never-auto-repeat commands
<code>~/.overseer/mcp.json</code> , <code><repo>/.overseer/mcp.json</code>	User, project (trusted by hash)	MCP servers
<code>~/.overseer/modes/*.toml</code>	User	User-defined modes
<code>~/.overseer/agents/<name>/agent.toml</code>	User	Persistent agents
<code>~/.overseer/workflows/*.toml</code>	User	Workflow definitions
<code>~/.overseer/plugins/<name>/plugin.toml</code>	User	Plugins
<code>~/.overseer/rules</code>	User (legacy)	Migrated into <code>policy.toml</code> , then read-only

Appendix C — Workstream Index

Every workstream in this roadmap, with its chapter and the wave that carries it.

ID	Workstream	Chapter	Wave
R1	Failure taxonomy	5	W1
R2	Provider retry, backoff and resumable streams	5	W1
R3	Close every timeout hole	5	W1
R4	The supervisor	5	W2
R5	Durable intent: the run journal	5	W3
R6	Crash-consistency audit of every durable write	5	W2
R7	The side-effect journal and idempotency	5	W2
R8	Away policy: asking when nobody is there	5	W4
R9	Progress ledger and long-window stall detection	5	W3
R10	Pacing: rate budgets and shared provider limits	5	W3
R11	Resource guards: disk, logs, spills, worktrees	5	W1, W3
R12	Time, sleep, power and networks	5	W2
R13	Upgrade-safe runs and schema evolution	5	W2
R14	Run lifecycle and control surface	5	W2
R15	Fault injection built into the engine	5	W1
R16	Environment drift	5	W3
R17	Process hygiene for very long lives	5	W2
R18	Optional remote executor	5	W10
O1	Depth and fan-out limits	6	W6
O2	The task graph	6	W6
O3	Artifacts	6	W6
O4	The blackboard	6	W6
O5	Detach and attach	6	W6
O6	The merge queue	6	W6
O7	Persistent agents	6	W6
O8	Teams and roles: the lead and sidekick protocol	6	W6
O9	Goal audits	6	W3
O10	Best-of-N with executable selection	6	W6
O11	Cross-provider verification	6	W6
O12	Handoff as an alternative to compaction	6	W5 (as C9)
O13	Brief quality	6	W6
O14	Seeing the tree	6	W6
O15	Missions	6	W6
O16	Cache-inheriting forks	6	W6
O17	Messaging with deadlines	6	W6
O18	Scheduling and machine resources	6	W6
M1	History search (memory op=history)	7	W8
M2	Applied learned skills	7	W8

ID	Workstream	Chapter	Wave
M3	The dream pass	7	W8
M4	Mounted memory repositories and sync	7	W8
M5	Outcome memory	7	W8
M6	Journals into memory	7	W8
M7	Cross-project promotion	7	W8
M8	Recurring reminders	7	W8
M9	Memory evaluation	7	W8
M10	Poisoning defences, extended	7	W8
M11	Memory you can see and edit	7	W8
M12	Shared and team memory	7	W10
C1	Token-aware eviction	8	W5
C2	Cross-turn result deduplication	8	W5
C3	Notice and digest aging	8	W5
C4	Shedding old reasoning	8	W5
C5	Edge placement of critical state	8	W3
C6	Retrieval-scoped reading	8	W5
C7	The context planner itself	8	W5
C8	Optional semantic compaction	8	W5
C9	Handoff instead of compaction	8	W5
C10	Context health telemetry	8	W5
T1	Hashline edits, behind a measured gate	9	W5
T2	run_code reach	9	W5
T3	A persistent QuickJS context	9	W5
T4	Native deferred loading on Anthropic	9	W5
T5	Project-level MCP configuration	9	W5
T6	Diagnostics beyond cargo	9	W5
T7	Availability that follows reality	9	W5
T8	Parallel read-only tool calls	9	W5
T9	Shaped results	9	W5
T10	Web fetch and search	9	W5
T11	Tool latency budgets	9	W5
T12	Tool descriptions as tested artifacts	9	W5
K1	The fourth Anthropic breakpoint	10	W5
K2	Time-to-live policy	10	W5
K3	Cache keepalive	10	W5
K4	Accurate token counts without a heavy tokenizer	10	W5
K5	Adaptive compaction threshold	10	W5
K6	Cost-aware routing inside a run	10	W5
K7	Effort that comes back down	10	W5
K8	Output discipline	10	W5
K9	One view of spend and cache across everything	10	W5
K10	Cache-miss forensics	10	W5
K11	Provider-native context features	10	W5

ID	Workstream	Chapter	Wave
K12	The first priced live run	10	W0
P1	The AGENTS.md loader	11	W4
P2	The instruction compiler	11	W4
P3	Rules engine v2	11	W4
P4	Scoped, expiring, revocable grants	11	W4
P5	Consent modes	11	W0, W4
P6	Dedicated permissions for secrets and sensitive surfaces	11	W4
P7	User-defined modes	11	W10
P8	Stream rules	11	W10
P9	Contextual rules (microagents, version 2)	11	W4
P10	Hooks, version 2	11	W10
P11	Explain any decision	11	W4
P12	Managing permissions	11	W4
P13	Principles enforced by the build	11	W4
P14	The instruction hierarchy as defence in depth	11	W4
P15	Git safety defaults	11	W4
A1	The workflow definition	12	W7
A2	The durable executor	12	W7
A3	Step kinds	12	W7
A4	Retries, timeouts and compensation	12	W7
A5	Schedules and time	12	W7
A6	Taint through workflows	12	W7
A7	Inbox, version 2	12	W7
A8	The workflow library	12	W7
A9	Versioning and migration	12	W7
A10	Testing workflows	12	W7
A11	Triggers, version 2	12	W7
A12	Seeing automations	12	W7
S1	The egress proxy	13	W4 (S1a macOS), W10 (S1b Linux)
S2	Finish the credential broker	13	W4
S3	A threat model for long runs and automation	13	W2
S4	The full-automation audit trail	13	W4
S5	Signed builds and provenance	13	W10
S6	The injection corpus, extended	13	Every wave
S7	Secrets hygiene	13	W0
S8	Sandbox hardening	13	W10
S9	A standing red-team programme	13	Every wave close
S10	The kill switch everywhere	13	W2
S11	Data at rest, retention and backup	13	W4
U1	Live validation on the owner's Mac	14	W0
U2	Consent modes for computer use	14	W0, W4
U3	Progress guard	14	W4

ID	Workstream	Chapter	Wave
U4	Settle waits	14	W4
U5	Clipboard	14	W4
U6	Overseer's own computer helper	14	W10
U7	Revocation on lock, sleep and session switch	14	W4
U8	The browser	14	W10
U9	Computer-use evaluation	14	W9
U10	GUI scripts through <code>run_code</code>	14	W5
L1	Error classification in every adapter	15	W1
L2	A living profile registry	15	W1
L3	Capability probes	15	W1
L4	Failover ladders	15	W1
L5	Local and open-weight models	15	W10
L6	Native provider features	15	W5
L7	Routing by task class	15	W5
L8	Deprecation watch	15	W1
B1	Traces from events	16	W3
B2	Global rollups	16	W3
B3	The run timeline	16	W3
B4	Forecasting	16	W3
B5	Alerts	16	W3
B6	Export	16	W9
B7	Replay debugger	16	W9
B8	Long-run health panel	16	W2
E1	Lift the hold: the first measured baseline	17	W0
E2	The long-horizon suite	17	W9
E3	The chaos suite	17	W9
E4	Cost per solved task, published	17	W9
E5	Regression gates per wave	17	Every wave
E6	Memory evaluation	17	W9
E7	Orchestration evaluation	17	W9
E8	Competitors on the same model	17	W9
E9	Contamination hygiene	17	W2 (authoring), W9
X1	The long-run cockpit	18	W2, W6
X2	The inbox frontend	18	W7
X3	ACP and the editor bridge	18	W10
X4	Approvals and status from the phone	18	W4
X5	Notifications that respect attention	18	W7
X6	Steering long runs	18	W10
X7	Web UI parity	18	W10
X8	Onboarding	18	W10
X9	Accessibility	18	W10
X10	Documentation as part of the product	18	Every wave
X11	Importing from other harnesses	18	W10

ID	Workstream	Chapter	Wave
Y1	A plugin model	19	W10
Y2	Versioning and trust tiers	19	W10
Y3	Project-level MCP	19	W5 (as T5)
Y4	An embedding SDK and peer agents	19	W10
Y5	The hooks protocol	19	W10
Y6	Sharing	19	W10
Z1	Budgets enforced by the build	20	Every wave
Z2	Signed and notarized builds	20	W10
Z3	Installers and safe self-update	20	W0 decision, W10
Z4	iOS gating	20	W10
Z5	Linux parity	20	W10
Z6	Windows plan	20	W10
Z7	Soak and leak testing	20	W2 onward
Z8	Move the repository out of iCloud	20	W0
Z9	Licences and attribution	20	Every wave
F1	Engine APIs for Life	21	W10
F2	Specialist agents on persistent-agent infrastructure	21	W10
F3	Life automations as workflows	21	W10
F4	One permissions model	21	W10
F5	Telemetry into the AI accounts module	21	W10

Appendix D — Glossary

Term	Meaning
Artifact	A typed, versioned, taint-carrying output one agent hands another (O3)
Away policy	What happens to an approval request when the owner does not answer in time (R8)
Blackboard	A run's shared, append-only scratchpad of short entries (O4)
Cache boundary	The point in the prompt above which nothing volatile may appear (invariant 2)
Compaction	Replacing old events in the model's view with a mechanical summary; never mutates the log (invariant 8)
Context planner	The single deterministic function that builds each request's view under a token budget (C7)
Detached run	A run owned by the supervisor rather than by the frontend that started it (R4)
Effect journal	Intent and outcome records around every side effect (R7)
Failover ladder	An ordered list of equivalent models to continue on when the primary is unavailable (L4)
Full automation	The owner-chosen consent mode that never asks (P5)
Handoff	Starting a fresh context inside the same run, seeded with the journal and a brief (C9)
In-doubt effect	A side effect with an intent record and no outcome record after a crash (R7)
Journal	The durable record of a run's goal, plan, decisions, milestones and next step (R5)
Merge queue	The serialized path by which parallel writers' branches are rebased, verified and landed (O6)
Persistent agent	An agent with an identity, memory scope and mailbox that outlives one task (O7)
Policy compiler	The single pipeline that turns every instruction and permission source into the compiled policy (P2)
Rejuvenation	Restarting a long run's process at a safe boundary to shed slow leaks (R17)
Rule of Two	Overseer's taint model: untrusted content plus sensitive data arms the exfiltration gate
Shadow verdict	The verdict a call would have received under per-action consent, recorded in full automation (S4)
Supervisor	The daemon component that owns run liveness and restarts (R4)
Taint	A latch recording that untrusted content entered a run, propagated with data (invariant 19)
Wave	A unit of sequencing that closes only on its exit gate (Chapter 23)

Appendix E — Sources

In this repository:

- `docs/agent-harness-playbook.pdf` (145 pages, September 2026): the twelve theses, harness tear-downs, the Alfred architecture (Ch. 11 §7), the build plan (Ch. 12).
- `docs/LLM_Harness_Arsenal.pdf`.
- `docs/research/2026-09-29-pillars.md`: caching and token economics, resident footprints, the harness distillation table.
- `docs/research/2026-09-30-memory-v2.md`, `2026-10-06-memory-v3.md`: memory design and the H2/H3 specifications referenced by M1–M4.
- `docs/research/2026-09-30-tool-economy.md`: deferral and code-mode evidence and decisions.
- `docs/research/2026-09-30-subagent-tiers.md`: tiers, budgets, verification evidence, refusals.
- `docs/research/2026-10-05-digital-life.md`, `2026-10-05-life-security.md`, `2026-10-05-life-phase0-results.md`: Overseer Life plan, threat model and Phase 0 results.
- `docs/LEAVING-OFF.md`, `AGENTS.md`, `eval/README.md`, `eval/LEDGER.md`.
- The baseline code at the commits listed in Part 0, read for Chapter 3.

External work cited through those notes (tagged `[src]` in the text): Anthropic’s multi-agent research system and context engineering posts; Cognition’s “Don’t build multi-agents”; MAST (Cemri et al. 2025); Huang et al. 2023 on self-correction; Panickssery et al. 2024 on self-preference; CRITIC (Gou et al. 2023); Lightman et al. 2023; Agentless (Xia et al. 2024); CodeMonkeys (Ehrlich et al. 2025); CodeAct (Wang et al. 2024); RAG-MCP (Gan et al. 2025); Lindenbauer et al. 2025 on observation masking; RULER, NoLiMa and Context Rot on effective context; MINJA and AgentPoison on memory poisoning; LongMemEval, LoCoMo and MemoryAgentBench; FrugalGPT and RouteLLM on routing; the agentic-browser security studies listed in the Life plan.

Appendix F — How This Edition Was Reviewed

The owner asked for this roadmap to be thought through “twice, thrice” before its final edition. This appendix records what each pass changed.

Pass 1: the author’s own review

A full re-read against the contract, the baseline audit and the document’s own invariants, looking for gaps, contradictions and designs that would not work. It changed:

Area	Change
Identifiers	Automation workstreams renamed from W to A so they never collide with wave numbers; finding numbers from the fix2 reviews marked as such so they cannot be read as workstreams
R7 idempotency keys	The draft derived keys from the attempt number, which would have given every retry a new key and defeated deduplication. Keys now come from the session (or instance) and call (or step) only
R10 shared limits	Per-call admission through the daemon would have added a round trip to every request on a single-threaded loop. Replaced by background-renewed capacity leases
R10 pacing	Added subscription quota windows, fed by Overseer Life’s usage readers
R4	Interactive runs register with the supervisor and are offered, not forced, a resume
R16	Workspace leases so two top-level runs never write the same tree
A2	Workflow tool steps run outside the credential-free daemon; agent steps keep durable sessions instead of <code>--bare</code> temp sessions
O2	Graph events go through the run’s single log writer
P1	Owner can trust a whole repository (Overseer’s own AGENTS.md changes constantly)
New workstreams	P15 git safety defaults, S11 data at rest and backup, X10 documentation, Z9 licences and attribution
Premortems	Lessons from the three premortems folded back into R7, R9, R10 and A12
Sequencing	Every workstream given a wave (L3, L5–L8, T3 and U10 had none); the coverage matrix completed
Failure atlas	Memory lock contention under many concurrent runs; workspace collisions

Pass 2: independent adversarial review

A separate reviewer, given only the owner’s request, the document and the repository, checked more than 60 factual claims against the code and read every chapter for consistency, soundness and coverage, then inspected all 125 rendered pages. It reported 23 findings: 3 material, 11 minor, 9 nits. Every finding was investigated; all were confirmed and fixed:

Finding	Fix
Eval corpus is 40 public tasks, not 35 (the repository README is stale)	Corrected in Chapters 3 and 17; README correction added to W0
R7 could not resolve effects with no read-back, and parking for them would wedge unattended runs	In-doubt policy declared at intent time (<code>verify</code> , <code>replay_safe</code> , <code>model_verify</code> , <code>unverifiable</code>); unverifiable effects block only their dependent plan item while the run continues; invariant 13 reworded
The egress proxy’s sandbox rule is not expressible with <code>bwrap</code> on Linux	S1 split into S1a (macOS, W4) and S1b (Linux transport, W10); Linux keeps deny-all until then
Stuck detector has five patterns, not four	Corrected
Resident spec “current pin” was the target	Corrected (pin 6,136 plus 5%, target 6,000)

Finding	Fix
Wave ranges swept in workstreams placed elsewhere	Exclusions written into Chapter 23 and its graph
“No single failure ends a run” overstated	Scoped to transient and infrastructure failures
Long-horizon targets depended on an undefined “solvable”	Solvability declared before scoring, never after (E2, metrics)
No true private holdout until the wave that consumes it	Holdout authored in W2
No equivalence policy for competitor comparisons	Published equivalence and “not comparable” policy (E8)
Model refusals had no failure class	Refused class added to R1
Workflow migration ignored semantic and engine-version changes	A9 refuses semantic changes without confirmation and checks engine ranges
Keepalive mechanics unstated	Out-of-band, never logged, suppressed while throttled
Invariant 25 contradicted by the pasted-key incident	Scoped to secrets the harness holds; pasted secrets handled by S7
Repository trust grant keyed on values the repository controls	Bound to the repository’s git directory identity
No relative sizing	Relative size per wave added (no dates, deliberately)
No import path from incumbent harnesses	X11 added
U2 missing from wave contents; M8 name collision; fix2 wording	Corrected
Display headings hyphenated	Template fixed
This appendix was a placeholder	Filled

Pass 3: final consistency and render check

After the repairs: every workstream reference in the text resolves to a defined workstream (checked by script), Appendix C was regenerated from the chapter headings so it cannot drift from them, and the final build was rendered and every page inspected again.